PulseTrack

Data Warehousing Capstone Project

Student Name: Ashini Samarasingha

Student ID: 1503

Programme: Zero to Data Engineer – Data Warehouse Design Fundamentals

Project: Capstone Project

Submission Date: 19/10/2026

Table of Contents

Question 1: Business KPIs, Analytical Requirements and Grain 8

1.1 Business Processes 8

1.2 Key Performance Indicators and Measures 9

1.3 Required Analytical Dimensions 10

1.4 Grain of the Analytical Processes 11

1.5 Historical Requirements 12

1.6 Analytical Questions 12

1.7 Data-Freshness and Retention Requirements 13

1.8 KPI-to-Source Mapping 13

Question 2 - Source Forensics and Enterprise Data Warehouse Design 16

2.1 Source Inventory and Data Formats 16

2.1.1 Relational CSV Sources 16

2.1.2 JSONL Sources 17

2.1.3 Kafka-Style Conversion Event Log 17

2.2 Source Keys, Relationships and Record Grain 18

2.2.1 Business Keys 18

2.2.2 Source Relationships 18

2.3 Source Profiling and Data-Quality Findings 19

2.3.1 Duplicate Conversion Records 19

2.3.2 Missing and Invalid Relationships 20

2.3.3 Partner Identity Issues 20

2.3.4 Revenue Adjustments Issue 21

2.3.5 Event-Stream Quality 21

2.3.6 Other Data-Quality Issues 22

2.4 Why Directly Joining the Raw Tables Is Unsafe 22

2.5 Proposed Enterprise Data Warehouse Dimensional Model 22

2.5.1 Dimension Tables 23

2.5.2 Fact Tables and Bridge Table 24

2.5.3 Fact-to-Dimension Relationships 25

2.6 Measures and Additive Behaviour 27

2.7 Surrogate Keys, Business Keys and Conformed Dimensions 28

2.8 Historical Preservation Using SCD Type 2 29
2.9 How the Dimensional Model Supports the Required KPIs 30
Question 3 – Data Platform Architecture and ETL/ELT Tool Selection 32
3.1 Technology Selection 32
3.2 Weighted Technology Evaluation 32
Evaluation Findings 33
Preferred Technology Approach 34
3.3 Final Technology Decision and Responsibilities 34
Selected Technologies 34
Proposed Data Processing Approach 35
Operational Considerations 35
3.4 Proposed Data Platform Architecture 36
3.4.1 Architecture Layers 36
3.4.2 Pipeline Orchestration 36
3.4.3 Monitoring, Recovery and Data Quality 37
3.4.4 Security and Data Traceability 37
Question 4 – ETL/ELT Pipeline Implementation 39
4.1 Implementation Overview 39
4.2 Bronze Layer - Source Ingestion and Raw Data Preservation 39
4.2.1 Bronze Ingestion Results 40
4.3 Silver Layer – Data Cleansing and Standardisation 42
4.3.1 Reference Data Processing 42
4.3.2 Transactional Data Processing 43
4.3.3 Conversion Event-Log Processing 43
4.3.4 Silver Layer Validation 44
4.4 Gold Layer – Enterprise Data Warehouse Implementation 45
4.4.1 Dimension Table Implementation 45
4.4.2 Historical Advertiser Processing – SCD Type 2 46
4.4.3 Fact and Bridge Table Implementation 46
4.4.4 Business Rules and Data Integrity 47
4.4.5 Gold Loading Validation 47
4.5 Apache Airflow Pipeline Orchestration 48

4.5.1 DAG Design and Task Dependencies 48
4.5.2 Execution Configuration 49
4.5.3 Successful End-to-End Execution 49
4.5.4 Monitoring and Automated Validation 50
4.5.5 Database Connectivity and Credential Handling 51
4.6 Implementation Summary and Validation 51
Question 5 – Incremental Processing, Idempotency, Late-Arriving Data and Recovery 52
5.1 Processing Strategy 52
5.2 Duplicate Delivery and Idempotency 53
5.2.1 Idempotency Across the Processing Layers 53
5.2.2 Controlled 50,000-Record Gold Replay Test 53
5.2.3 Test Scope and Limitation 54
5.2.4 Automated CSV Incremental Ingestion and Idempotency Testing 54
5.3 Late-Arriving Revenue Adjustments 57
5.3.1 Adjustment Processing Strategy 58
5.3.2 Controlled Late-Adjustment Test 58
5.3.3 Historical Accuracy and Limitations 59
5.4 Historical Corrections and SCD Type 2 59
5.4.1 Historical Processing Strategy 60
5.4.2 Controlled Historical Correction Test 60
5.4.3 Transaction Rollback and Verification 61
5.5 Pipeline Failure and Recovery 61
5.5.1 Recovery Strategy 62
5.5.2 Failure Between Dimension and Fact Loading 62
5.5.3 Airflow Failure Detection Test 62
5.5.4 Successful Recovery and Revalidation 63
5.5.5 Recovery Outcome 64
5.6 Processing Limitations and Summary 64
Question 6 – Data Quality, Reconciliation and Audit 66
6.1 Data Quality Framework 66
6.1.1 Main Data Quality Checks 66
6.2 Source-to-Bronze-to-Silver-to-Gold Reconciliation 67

6.2.1 Transactional Data Reconciliation 67
6.2.2 Event-Log Reconciliation 68
6.2.3 Reference Data and JSONL Reconciliation 69
6.2.4 Reconciliation Outcome 69
6.3 Duplicate Detection, Malformed Records and Referential Integrity 69
6.3.1 Duplicate Detection and Validation Results 70
6.3.2 Malformed-Record Handling 70
6.3.3 Missing and Unresolved Relationships 70
6.3.4 Data Quality Outcome 71
6.3.5 PostgreSQL Foreign-Key Constraints 71
6.3.6 Foreign-Key Validation Results and Unresolved References 72
6.4 Business-Rule, Attribution and Analytical Reconciliation 73
6.4.1 Conversion and Revenue Validation 73
6.4.2 Attribution Validation 73
6.4.3 Gold-to-Mart Reconciliation 74
6.4.4 OLAP Cache Consistency 75
6.4.5 Validation Outcome 76
6.5 Pipeline Audit and Error Tracking 76
6.5.1 Audit Table Structure 76
6.5.2 Audit Implementation and Verification Results 77
6.5.3 Audit Counters and Interpretation 79
6.5.4 Failure Tracking and Recovery Evidence 79
6.6 Data Quality, Reconciliation and Audit Summary 80
Question 7 - Data Mart Design and Implementation 81
7.1 Data Mart Overview 81
7.2 Data Mart Tables and Verification 81
7.3 Data Mart Transformation and Refresh Logic 83
7.3.1 Data Transformation 83
7.3.2 Refresh Execution 83
7.3.3 Refresh Verification 84
7.4 Business Reporting and Data Mart Usage 84
7.4.1 Campaign and Advertiser Performance 84

7.4.2 Creative and Partner Analysis 84
7.4.3 Account Manager and Financial Reporting 85
7.5 Data Mart Summary and Limitations 85
Question 8 - OLAP Implementation and Analytical Performance 86
8.1 OLAP Implementation Overview 86
8.2 Monthly Analytical View and Materialized Cache 86
8.2.1 OLAP Data Consistency Verification 86
8.2.2 Query Performance Comparison 87
8.3 Multidimensional Analysis and OLAP Operations 88
8.3.1 Analytical Dimensions and Measures 88
8.3.2 Roll-Up and Drill-Down 89
8.3.3 Slice and Dice 90
8.3.4 Ranking and Comparative Analysis 90
8.3.5 OLAP Roll-Up Verification 91
8.4 Period-over-Period Analytical Comparisons 92
8.4.1 Month-over-Month Analysis 93
8.4.2 Quarter-over-Quarter Analysis 93
8.4.3 Year-over-Year Analysis 94
8.4.4 Analytical Interpretation and Limitations 94
8.5 OLAP Performance Considerations and Limitations 95
High-Cardinality Dimensions and Scalability 95
8.6 OLAP Implementation Summary 96
Question 9 - Business Analytics and Insights 97
9.1 Business Analytics Overview 97
9.2 Analytical Questions and KPI Traceability 98
9.2.1 KPI Calculation and Data Lineage 98
9.2.2 Interpretation of Analytical Results 99
9.3 Business Analytics Results and Insights 99
9.3.1 Analysis 1 – Top Advertisers by Quarter 99
9.3.2 Analysis 2 – High CTR but Low Conversion Rate 100
9.3.3 Analysis 3 – Top Revenue-Attributed Partners 101
9.3.4 Analysis 4 – Month-over-Month Revenue Changes 101

9.3.5 Analysis 5 – Campaign Revenue Concentration 102
9.3.6 Analysis 6 – Creative Underperformance 102
9.3.7 Analysis 7 – Historical Account-Manager Assignment Changes 103
9.3.8 Analysis 8 – Unresolved Partner Revenue Attribution 104
9.3.9 Analysis 9 – Increasing Impressions but Declining Conversions 105
9.3.10 Analysis 10 – Delayed Revenue Adjustments 106
9.4 KPI Lineage and Business Decision Support 106
9.4.1 Business Analytics SQL Implementation 106
9.4.2 KPI Traceability Across Warehouse Layers 107
9.4.3 Business Decision Support 109
9.5 Business Analytics Summary and Limitations 110
Question 10 - Architecture Scenarios and Evaluation 111
10.1 Overview 111
10.2 Scenario-Based Architecture Evaluation 111
10.2.1 Scenario 1 – Source Data Volume Increases Tenfold 111
10.2.2 Scenario 2 – A JSON Source Changes Its Schema 112
10.2.3 Scenario 3 – The Same Source File Is Delivered Three Times 113
10.2.4 Scenario 4 – A Revenue Adjustment Arrives Seven Days Late 114
10.2.5 Scenario 5 – A Historical Account-Manager Assignment Is Corrected 115
10.2.6 Scenario 6 – The ETL Pipeline Fails Halfway Through Processing 116
10.2.7 Scenario 7 – A BI User Requests Five Years of Analytical Data 117
10.2.8 Scenario 8 – The Finance Team Disputes a Reported Revenue Amount 118
10.3 Overall Architecture Evaluation and Recommendations 120
10.4 Final Conclusion 121

Question 1: Business KPIs, Analytical Requirements and Grain

1.1 Business Processes

PulseTrack is a performance marketing platform that manages advertising campaigns, partner activities and customer conversions. To design the data warehouse, the main business processes must first be identified, as they determine what data needs to be collected and analysed.

The five main processes are:

1. Impression tracking

Records when an advertisement is displayed to a user. This data helps measure advertisement exposure and compare the performance of different campaigns and creatives.

2. Click tracking

Captures user interactions with advertisements. Click data is used to measure engagement and calculate click-through rates (CTR). Some clicks may occur without a recorded impression, so these need to be handled correctly during processing.

3. Conversion tracking

Records customer actions resulting from marketing activities, such as completing a purchase or registration. A conversion may have multiple status updates, but it should only be counted once when calculating business KPIs.

4. Revenue and revenue-adjustment tracking

Tracks revenue generated by conversions and any later financial adjustments. Adjustments may increase or decrease the original revenue, so both values must be considered when reporting financial performance.

5. Partner attribution

Identifies which marketing partners contributed to a conversion. Since multiple clicks may be involved in a single conversion, attribution information is needed to distribute conversion credit and revenue accurately.

Advertisers, campaigns, offers, creatives and account managers provide additional business context for these processes. Their information supports performance comparisons across different reporting periods. Historical changes, particularly account-manager reassignments, must also be preserved to ensure that past results remain accurate.

1.2 Key Performance Indicators and Measures

The PulseTrack data warehouse will support KPIs that measure advertising engagement, conversion performance, revenue and partner contributions. These measures will help compare advertisers, campaigns and partners across different reporting periods.

The main KPIs are defined below.

KPI	Calculation	Purpose
Impressions	Count of valid impression events	Measures advertisement exposure.
Clicks	Count of valid click events	Measures user engagement with advertisements.
Click-through Rate (CTR)	(Clicks / Impressions) × 100	Shows the percentage of impressions that result in clicks.
Conversions	Count of distinct eligible conversions	Measures successful customer actions.
Conversion Rate	(Conversions / Clicks) × 100	Shows how effectively clicks lead to conversions.
Revenue	Sum of eligible conversion revenue	Measures revenue generated by marketing activities.
Adjusted Revenue	Original revenue + valid revenue adjustments	Shows revenue after financial adjustments.
Revenue per Conversion	Revenue / Conversions	Measures average revenue earned per conversion.
Revenue per Click	Revenue / Clicks	Measures revenue generated relative to click volume.
Attributed Conversions	Count of distinct eligible conversions associated with a partner	Identifies conversions linked to each partner.
Weighted Conversion Credit	Sum of valid attribution weights assigned to a partner	Measures the partner's share of conversion credit.
Attributed Revenue	Sum of eligible conversion revenue × valid attribution weight	Measures revenue credited to each partner.
Attribution Percentage	(Weighted conversion credit / Total eligible conversions) × 100	Shows a partner's share of eligible conversions.

The warehouse will also support campaign rankings and percentage contribution analysis. Campaigns can be ranked using KPIs such as revenue, conversions and CTR. Percentage contribution will show how much an advertiser, campaign or partner contributes to the corresponding total.

Calculation Considerations

Some calculations require additional checks to ensure accurate results.

- **Duplicate conversions:** Multiple records with the same conversion ID must not be counted as separate business conversions.
- **Revenue adjustments:** Adjustments must be linked to the correct conversion without increasing the conversion count.
- **Partner attribution:** A conversion may involve several partners. Distinct conversion counts and weighted attribution credit must be kept separate to avoid double counting.
- **Aggregation:** Rates and percentages must be recalculated from aggregated totals rather than summed or averaged directly.
- **Zero denominators:** Percentage and revenue-per-event calculations must handle cases where the denominator is zero.

The final eligibility rules for conversion statuses, test traffic, revenue and attribution will be confirmed during source analysis and applied consistently throughout the warehouse and reporting layers.

1.3 Required Analytical Dimensions

Analytical dimensions allow PulseTrack to examine marketing performance from different perspectives. They provide the information needed to group, filter and compare KPIs across campaigns, advertisers, partners and reporting periods.

The following dimensions are identified for the data warehouse.

Dimension	Analytical Purpose
Date and Time	Analyse daily, monthly, quarterly and yearly performance and identify trends over time.
Advertiser	Compare revenue, conversions and overall marketing performance between advertisers.
Campaign	Evaluate campaign performance using measures such as impressions, clicks, conversions and revenue.
Creative	Compare advertisement engagement and conversion performance across different creatives.
Partner	Analyse partner contributions, including attributed conversions and revenue.
Account Manager	Evaluate performance under different account managers while preserving historical assignments.
Offer	Analyse the performance of marketing offers associated with advertisers and campaigns.
Geography	Compare marketing performance across available geographical locations.
Device	Examine engagement and conversions across different device types.
Conversion Status	Analyse conversions according to their recorded status, such as approved, pending or reversed.

These dimensions will be used across the relevant fact tables to support consistent reporting. Their final attributes and relationships will be confirmed using the supplied source data during Question 2.

Historical account-manager assignments will require special attention so that past conversions remain linked to the correct manager, even when an advertiser's assignment changes.

1.4 Grain of the Analytical Processes

Grain defines what a single record in a fact table represents. Each business process in PulseTrack has a different level of detail, so the grain must be defined before designing the data warehouse.

The proposed grain for each process is shown below.

Business Process	Proposed Grain	Reason
Impressions	One row per unique impression event	Allows impressions to be counted without duplication.
Clicks	One row per unique click event	Supports accurate click counts, including clicks without a recorded impression.
Conversions	One row per distinct business conversion	Prevents repeated conversion records and status updates from being counted as new conversions.
Revenue Adjustments	One row per unique adjustment event	Keeps individual revenue changes available for financial calculations and auditing.
Partner Attribution	One row per conversion-to-click touchpoint	Supports multi-touch attribution without duplicating the original conversion or its revenue.

The main challenge is that one conversion may be linked to multiple clicks and revenue adjustments. If these records are combined without considering their grain, conversion counts and revenue totals could be overstated.

To avoid this, impressions, clicks, conversions and revenue adjustments will be modelled at their individual grains. A bridge table will preserve the relationship between conversions and their attribution touchpoints.

Account-manager performance will be analysed using the historical assignment valid at the time of each conversion. This allows past results to remain associated with the correct manager without creating additional conversion records.

These are the proposed logical grains. The final fact tables and relationships will be established in the dimensional model in Question 2.

1.5 Historical Requirements

PulseTrack needs to preserve historical information so that past reports remain accurate when business details change. One important example is an advertiser being reassigned to a different account manager.

If an advertiser moves from Manager A to Manager B, conversions recorded before the reassignment should still belong to Manager A. Assigning all past conversions to the new manager would produce incorrect historical performance reports.

To handle this, the warehouse will use Slowly Changing Dimension Type 2 (SCD Type 2) for advertiser records that require historical tracking. Instead of overwriting the previous account-manager assignment, a new dimension record will be created with its own effective dates and surrogate key.

When processing conversions, the warehouse will use the conversion date to identify the account-manager assignment that was valid at that time.

The system must also handle late-arriving events and revenue adjustments. These changes should update the relevant analytical results without creating duplicate conversions or losing the original transaction history.

1.6 Analytical Questions

The PulseTrack data warehouse should help the business understand campaign performance, partner contributions and revenue trends. The following questions will guide the warehouse design and reporting requirements.

1. Which advertisers generate the highest revenue each month, quarter and year?
2. Which campaigns have the highest conversion rates, and which receive many impressions but relatively few clicks?
3. Which creatives perform best in terms of engagement and conversions?
4. Which partners contribute the most attributed conversions and revenue?
5. How does revenue change from month to month, quarter to quarter and year to year?
6. What percentage of total revenue or conversions comes from each advertiser, campaign or partner?
7. How do campaigns and partners rank based on selected KPIs such as revenue, conversions and CTR?
8. How does performance differ between account managers, and what changes can be observed before and after an advertiser is reassigned?
9. Are there periods when impressions and clicks increase without a similar increase in conversions?
10. How do revenue adjustments affect the final revenue reported for campaigns and advertisers?

These questions will be addressed through the warehouse fact tables, dimensions and analytical models. The final reports should provide measurable results that support business decisions.

1.7 Data-Freshness and Retention Requirements

PulseTrack needs timely data for performance reporting while keeping enough historical information for trend analysis, auditing and recovery.

The following requirements are proposed for the data warehouse.

Requirement	Proposed Target	Reason
Data Freshness	Process new data daily, making it available for reporting within 24 hours of delivery.	Supports regular campaign monitoring and performance reporting.
Historical Data	Retain five years of detailed warehouse data.	Supports year-over-year comparisons and historical account-manager analysis.
Raw Data Retention	Retain raw source records and ingestion metadata for at least one year.	Allows recent data to be checked or reprocessed when problems occur.
Audit Records	Retain pipeline logs, processing results and error records for one year.	Supports troubleshooting, reconciliation and monitoring.
Data Quality	Run validation and reconciliation checks during every pipeline execution.	Helps detect missing, duplicate or incorrect records before reporting.

Late-arriving conversions and revenue adjustments should be included in the next successful processing run. The pipeline should also support reprocessing historical data when corrections are required.

These targets are proposed design requirements, as the capstone brief does not specify exact freshness or retention periods. The final processing schedule and retention approach will be reviewed when selecting the technologies and designing the data platform in Question 3.

1.8 KPI-to-Source Mapping

The KPIs defined earlier must be linked to the relevant PulseTrack source data. This mapping identifies which datasets will be used for each measure and highlights the main processing requirements.

The source data contains duplicate conversions, inconsistent partner identifiers and revenue adjustments that may arrive after the original conversion. These issues must be resolved before the KPIs are calculated.

Table - KPI-to-Source Mapping

KPI / Analysis	Required Measure	Main Dimensions	Grain	Source Data
Impressions	Valid impression count	Date, advertiser, campaign, creative	Unique impression event	`impressions_raw.csv`
Clicks	Valid click count	Date, advertiser, campaign, creative, partner	Unique click event	`clicks_raw.csv`
CTR	Clicks / impressions × 100	Date, campaign, creative, advertiser	Compatible aggregated click and impression totals	`clicks_raw.csv`, `impressions_raw.csv`
Conversions	Distinct eligible conversion count	Date, advertiser, campaign, partner, status	Unique business conversion	`conversions_raw.csv`, conversion event log
Conversion Rate	Conversions / clicks × 100	Date, advertiser, campaign, creative	Compatible aggregated conversion and click totals	`conversions_raw.csv`, `clicks_raw.csv`
Revenue	Sum of eligible conversion revenue	Date, advertiser, campaign, account manager	Unique business conversion	`conversions_raw.csv`
Adjusted Revenue	Base revenue + valid adjustments	Date, advertiser, campaign	Conversion-level revenue after aggregating adjustments	`conversions_raw.csv`, `revenue_adjustments_raw.csv`, event log
Revenue per Conversion / Click	Eligible revenue / corresponding count	Date, advertiser, campaign	Compatible aggregated measures	Conversion and click sources

KPI / Analysis	Required Measure	Main Dimensions	Grain	Source Data
Attributed Conversions	Distinct eligible conversions linked to each partner	Date, partner, campaign	Unique conversion–partner pair	`conversion_touchpoints_raw.csv`, `clicks_raw.csv`
Weighted Conversion Credit	Sum of attribution weights	Date, partner, campaign	Conversion-to-click touchpoint	`conversion_touchpoints_raw.csv`
Attributed Revenue	Eligible revenue × attribution weight	Date, partner, campaign	Conversion-to-click touchpoint	Conversion, touchpoint and partner sources
Attribution Percentage	Weighted credit / total eligible conversions × 100	Date, partner, campaign	Aggregated partner attribution	Conversion and touchpoint sources
Campaign Ranking and Top-N	Selected KPI and rank	Date, campaign, advertiser	Campaign per reporting period	Campaign and relevant fact sources
Percentage Contribution	Entity measure / corresponding total × 100	Date, advertiser, campaign, partner	Aggregated reporting group	Relevant conversion, revenue and attribution sources
Account-Manager Performance	Conversions and revenue	Date, account manager, advertiser, campaign	Historical conversion context	`advertiser_accounts.jsonl`, `advertisers_raw.csv`, `conversions_raw.csv`
Period Comparisons and Trends	KPI totals and period changes	Day, month, quarter, year	Selected reporting period	Relevant event and financial sources

Note: The mapping represents the intended analytical design. Source validation, conversion eligibility and attribution rules will be finalised during warehouse implementation. The same definitions will be used across the warehouse, Data Mart and reporting layers.

This mapping will guide the warehouse design in Question 2. During implementation, the source fields and transformation rules will be linked to the corresponding fact tables, dimensions and validation checks.

Question 2 - Source Forensics and Enterprise Data Warehouse Design

2.1 Source Inventory and Data Formats

PulseTrack provides data in three formats: relational CSV files, JSONL documents and a Kafka-style event log. These sources contain information about advertisers, campaigns, partners, marketing activities and conversions.

Before designing the warehouse, the datasets need to be examined to understand their structure, relationships and data-quality issues.

The supplied package contains 11 relational tables with 774,344 rows, two JSONL collections with 660 documents and an event log containing 183,162 lines. These are raw source counts, before cleaning or deduplication.

2.1.1 Relational CSV Sources

The relational data contains the main business records and marketing activities.

Source Table	Records	Description
`account_managers_raw`	25	Account-manager details.
`advertisers_raw`	60	Advertiser information and current manager assignments.
`partners_raw`	600	Partner reference information.
`offers_raw`	150	Marketing offers associated with advertisers.
`campaigns_raw`	400	Campaign details and offer relationships.
`creatives_raw`	1,200	Advertisement creatives linked to campaigns.
`impressions_raw`	300,000	Advertisement impression events.
`clicks_raw`	150,000	Advertisement click events.
`conversions_raw`	126,380	Conversion records, including repeated snapshots.
`conversion_touchpoints_raw`	183,529	Click touchpoints associated with conversions.
`revenue_adjustments_raw`	12,000	Revenue changes associated with conversions.
Total	**774,344**	

The raw tables contain inconsistent values, duplicate conversion records and some missing or invalid relationships. These issues must be addressed before loading the cleaned analytical data.

2.1.2 JSONL Sources

Two JSONL collections provide additional information about advertisers and partners.

Source Collection	Documents	Description
`advertiser_accounts.jsonl`	60	Advertiser CRM details, billing information and historical account-manager assignments.
`partner_profiles.jsonl`	600	Partner profiles, including contact, channel and payment information.

These collections contain nested fields that are not fully available in the relational data.

The account-manager history is particularly important for historical reporting, while the partner profiles help match partner identifiers across different sources.

2.1.3 Kafka-Style Conversion Event Log

The `pulsetrack_conversions.log` file contains 183,162 lines representing conversion-related events.

The main event types are:

- `conversion.created` – records the creation of a conversion.
- `conversion.status_changed` – records changes to a conversion's status.
- `adjustment.applied` – records revenue adjustments.

Unlike the relational conversion table, which contains snapshots of conversion records, the event log captures individual changes over time.

This information can help reconstruct conversion history and identify adjustments that occurred after the original conversion.

The log also includes duplicate deliveries, malformed records and changes in event structure. These issues will be examined in the data-quality analysis in Section 2.3.

The supplied log is a Kafka-style export rather than evidence of a running Kafka service. The ingestion approach will be decided during the architecture and implementation stages.

2.2 Source Keys, Relationships and Record Grain

Before designing the dimensional model, the keys and relationships between PulseTrack's source tables must be identified. These keys help connect records across the different data sources and prevent incorrect joins during processing.

2.2.1 Business Keys

A business key identifies an entity or event in the source system. The main source keys are listed below.

Source Entity	Business Key	Purpose
Account Manager	`am_id`	Identifies an account manager.
Advertiser	`advertiser_id`	Links advertiser records across CSV and JSONL sources.
Partner	Canonical five-digit partner ID	Identifies partners after standardising different source formats.
Offer	`offer_id`	Identifies a marketing offer.
Campaign	`campaign_id`	Identifies an advertising campaign.
Creative	`creative_id`	Identifies a campaign creative.
Impression	`impression_id`	Identifies an impression event.
Click	`click_id`	Identifies a click event.
Conversion	`conversion_id`	Identifies a business conversion.
Conversion Touchpoint	`touchpoint_id`	Identifies a conversion-to-click touchpoint.
Revenue Adjustment	`adjustment_id`	Identifies an individual revenue adjustment.
Stream Event	`event_id`	Identifies an event in the conversion stream.

These are candidate keys and must be validated before use in the warehouse.

One important issue is that `conversion_id` is repeated in `conversions_raw`. Some records represent updated versions of the same conversion, so they cannot all be treated as separate transactions.

Similarly, multiple stream deliveries may contain the same `event_id`. Duplicate deliveries must be identified without removing legitimate status changes or revenue adjustments.

2.2.2 Source Relationships

The main business relationships in PulseTrack are shown below.

Business hierarchy:

Account Manager → Advertiser → Offer → Campaign → Creative

An account manager is responsible for advertisers, while advertisers are associated with offers, campaigns and creatives. Account-manager assignments may change over time, so their historical relationships must be considered.

Marketing activity:

Impression → Click → Conversion

This represents the general marketing journey. However, not every click has a recorded impression, and some conversions may not have a directly linked click. These records should not automatically be treated as invalid.

Two additional relationships are important:

- **Conversion and Touchpoints:** A conversion can have multiple associated clicks, and a click may contribute to multiple conversions. The `conversion_touchpoints_raw` table records these relationships and supports multi-touch attribution.
- **Conversion and Revenue Adjustments:** A conversion may have several revenue adjustments. These must be linked to the original conversion without increasing the conversion count.

Partner identifiers also appear in different formats across the CSV files, JSONL profiles and event data. They must be standardised and matched to a common partner identifier before partner-level KPIs are calculated.

These relationships will guide the fact tables, dimensions and conversion-touchpoint bridge in the Enterprise Data Warehouse.

2.3 Source Profiling and Data-Quality Findings

The raw PulseTrack datasets were examined to identify duplicate records, missing values, inconsistent formats and relationship problems. These findings help determine which cleaning and validation rules are needed before loading data into the warehouse.

The following results describe the original source data, before cleaning or deduplication.

2.3.1 Duplicate Conversion Records

The `conversions_raw` table contains 126,380 rows but only 90,000 distinct conversion IDs. Of these, 31,672 conversion IDs appear more than once, resulting in 36,380 additional records.

These repeated records can represent different snapshots of the same conversion. Counting every row as a separate conversion would therefore overstate the conversion totals.

The proposed solution is to retain the latest valid record for each `conversion_id`, based on `updated_at`. Timestamp formats must first be standardised to ensure that the correct record is selected.

Original records will remain in Bronze, while the cleaned conversion records will be maintained in Silver.

2.3.2 Missing and Invalid Relationships

Several missing or inconsistent relationships were identified during source profiling.

Finding	Raw Records Affected	Proposed Handling
Clicks without an impression ID	22,591	Retain as valid direct clicks where permitted.
Impressions without a creative ID	14,472	Use an appropriate unknown creative reference.
Conversions without a click ID	12,654	Preserve valid conversions and investigate available touchpoints.
Conversions referencing missing clicks	12,591	Record unresolved click references for reconciliation.
Campaigns with invalid or mismatched advertiser references	12	Investigate and apply a documented correction rule.
Revenue adjustments referencing conversions absent from the batch export	721	Preserve for reconciliation with other source records.

These figures were measured from raw records, including repeated conversion snapshots.

Not every missing reference represents an error. For example, PulseTrack allows direct clicks without a recorded impression. Such records should be preserved rather than rejected automatically.

2.3.3 Partner Identity Issues

Partner identifiers are not consistent across the source systems.

The relational partner table and JSONL partner profiles each contain 600 records, but only 560 partner identifiers overlap. Some partner information is therefore available in one source but missing from the other.

Marketing records also contain different identifier formats, including compound IDs, five-digit partner IDs, platform-generated IDs and missing values.

To address this, valid identifiers will be standardised to a common five-digit partner ID and matched against the combined partner reference data.

Unresolved identifiers will be recorded separately rather than assigned to an incorrect partner. Original values will also be retained for tracing and validation.

2.3.4 Revenue Adjustments Issue

The `revenue_adjustments_raw` table contains 12,000 records. Profiling identified the following issues:

- 600 records have missing adjustment amounts.
- 372 records have missing adjustment timestamps.
- 721 records reference conversions not found in the batch conversion data.

Some missing adjustment values may be recovered from matching conversion-stream events.

The proposed approach is to match adjustment records using `adjustment_id` and `conversion_id`, validate the amounts and timestamps, and reconcile them with available stream events.

Adjustments must be applied to their original conversions without creating additional conversion records.

2.3.5 Event-Stream Quality

The Kafka-style conversion log contains 183,162 lines. The initial profiling results are summarised below.

Finding	Record Count
Total log lines	183,162
Malformed or truncated lines	184
Parseable records	182,978
Duplicate event deliveries	4,378
Distinct events after deduplication	178,600

The log also contains a schema change where some newer events use `revenue_amount` instead of `revenue`.

During processing, malformed records will be separated for investigation, duplicate events will be identified using `event_id`, and both revenue field names will be supported.

The original stream records will be retained for traceability.

2.3.6 Other Data-Quality Issues

The source files contain several additional inconsistencies that must be considered during transformation:

- **Dates and timestamps:** Different date formats and timezone representations.
- **Monetary values:** Currency symbols, text values, blanks and negative amounts.
- **Conversion statuses:** Different spellings and casing of status values.
- **Boolean values:** Different representations such as `true`, `false`, `Y`, `N`, `1` and `0`.
- **Names and descriptions:** Inconsistent capitalisation and spelling.
- **Audit timestamps:** Missing values or inconsistent creation and update times.
- **Sensitive information:** Some click records contain IP addresses and user-agent details that should not be included in analytical outputs without a valid need.

These findings will guide the cleaning and validation rules used in the Silver layer. Records that cannot be resolved automatically will be retained for investigation, while accepted records will be prepared for dimensional modelling.

2.4 Why Directly Joining the Raw Tables Is Unsafe

Directly joining the raw PulseTrack tables could produce incorrect KPI results because the datasets contain different types of records and one-to-many relationships.

For example, a single conversion may have three attribution touchpoints and two revenue adjustments. Joining all three tables without handling these relationships correctly could produce six rows for that conversion. If the original revenue is summed from these rows, it would be counted multiple times.

The raw conversion data also contains repeated snapshots of the same conversion. These duplicates could further increase conversion counts and revenue totals if they are not resolved before reporting.

To prevent this, each business process will be stored at its appropriate grain. Conversions will be deduplicated, revenue adjustments will be handled separately, and a bridge table will link conversions to their attribution touchpoints.

This design will allow the warehouse to calculate impressions, clicks, conversions and revenue without accidentally multiplying records during joins.

2.5 Proposed Enterprise Data Warehouse Dimensional Model

The PulseTrack Enterprise Data Warehouse will use a dimensional model to organise marketing activities, conversions and revenue data for analysis.

The design separates measurable business events into fact tables and supporting business information into dimension tables. Shared dimensions will allow KPIs to be compared consistently across campaigns, advertisers, partners and reporting periods.

The proposed warehouse includes dimension tables, fact tables and a bridge table for multi-touch attribution.

2.5.1 Dimension Tables

The following dimensions are proposed for the warehouse.

Dimension	Main Information	Historical Handling
dim_date	Date, day, month, quarter and year	No historical tracking required.
dim_advertiser	Advertiser ID, name, vertical and account-manager assignment	SCD Type 2 for historical manager assignments.
dim_account_manager	Manager ID, name, team and region	Preserve relevant history where required.
dim_partner	Standardised partner ID and profile details	Type 1 initially.
dim_offer	Offer ID, name, advertiser and offer details	Historical tracking if required.
dim_campaign	Campaign ID, name, offer, advertiser and campaign details	Historical tracking where justified.
dim_creative	Creative ID, name, format, size and associated campaign	Type 1 initially.
dim_country	Standardised country ID and name	Type 1.
dim_device	Standardised device classification	Type 1.
dim_conversion_status	Conversion status and description	Type 1.

Each dimension will use a surrogate key generated within the warehouse. The original business identifier will also be retained to support source matching and traceability.

The advertiser dimension requires special attention because account-manager assignments can change over time. SCD Type 2 will preserve previous assignments by creating a new dimension version instead of overwriting historical information.

Dimensions shared by multiple fact tables will follow consistent definitions, allowing marketing and financial measures to be analysed together without conflicting business information.

The final dimension attributes and historical rules will be validated against the source data during implementation.

2.5.2 Fact Tables and Bridge Table

The PulseTrack warehouse will use four fact tables to store marketing events, conversions and revenue adjustments. A separate bridge table will handle multi-touch attribution.

Each table has a defined grain to ensure that records are counted and aggregated correctly.

Table	Grain	Main Measures and Purpose
`fact_impression`	One row per unique valid impression	Stores impression events for measuring advertisement exposure and calculating CTR.
`fact_click`	One row per unique valid click	Stores click events for analysing engagement and conversion rates.
`fact_conversion`	One row per distinct business conversion	Stores conversion count, base revenue, payout and conversion status after deduplication.
`fact_revenue_adjustment`	One row per unique adjustment event	Stores positive or negative revenue adjustments linked to conversions where possible.
`bridge_conversion_touchpoint`	One row per valid unique conversion-to-click touchpoint	Links conversions to their contributing clicks and stores the attribution weight.

Handling Conversions and Revenue

The raw conversion data contains repeated records for the same conversion. Before loading `fact_conversion`, these records will be processed to identify the latest valid snapshot for each `conversion_id`.

Revenue adjustments will be stored separately in `fact_revenue_adjustment`. This allows the warehouse to preserve individual financial changes and calculate adjusted revenue without creating additional conversion records.

Handling Multi-Touch Attribution

A conversion may be associated with several clicks, potentially involving different marketing partners.

The `bridge_conversion_touchpoint` table will preserve these relationships and their attribution weights. These weights can then be used to calculate each partner's share of conversion credit and revenue.

Attribution weights and relationships must be validated before they are used in KPI calculations. Conversion revenue must not be counted multiple times when joining the bridge table.

Together, these tables will support accurate marketing performance analysis while keeping the original business events and financial adjustments traceable.

2.5.3 Fact-to-Dimension Relationships

The fact tables will connect to the relevant dimension tables using surrogate keys. These relationships will allow PulseTrack to analyse marketing performance across advertisers, campaigns, partners and reporting periods.

The proposed relationships are outlined below.

Fact / Bridge Table	Main Relationships
`fact_impression`	Links to date, campaign, creative, advertiser, country and device dimensions where valid source information is available.
`fact_click`	Links to date, advertiser, campaign, creative, partner, country and device dimensions. The advertiser reference will use the historically valid dimension version, while the original impression identifier will be retained where available.
`fact_conversion`	Links to date, advertiser, campaign, conversion status and the historically correct account-manager information.
`fact_revenue_adjustment`	Links to the adjustment date and its original conversion. Advertiser and campaign information can be obtained through the matched conversion.
`bridge_conversion_touchpoint`	Connects conversions to their associated clicks and attribution weights. Partner information is resolved through the relevant click and partner records.

The relationships must be validated before dimension keys are assigned. For example, an advertiser's historical account-manager assignment will be identified using the conversion date rather than the advertiser's current manager.

Some records may contain missing or unresolved references. Where the business event is otherwise valid, these records will be preserved using an appropriate unknown or unresolved reference while retaining their original source identifiers.

For revenue adjustments, the adjustment date must be kept separate from the original conversion date. This allows the warehouse to analyse when revenue was generated and when it was later adjusted.

The conversion-touchpoint bridge will support multi-touch attribution without duplicating conversion revenue. Attribution weights will be applied when calculating partner-level conversion credit and attributed revenue.

These relationships will be represented in the dimensional model diagram, including primary keys, foreign keys and table cardinalities.

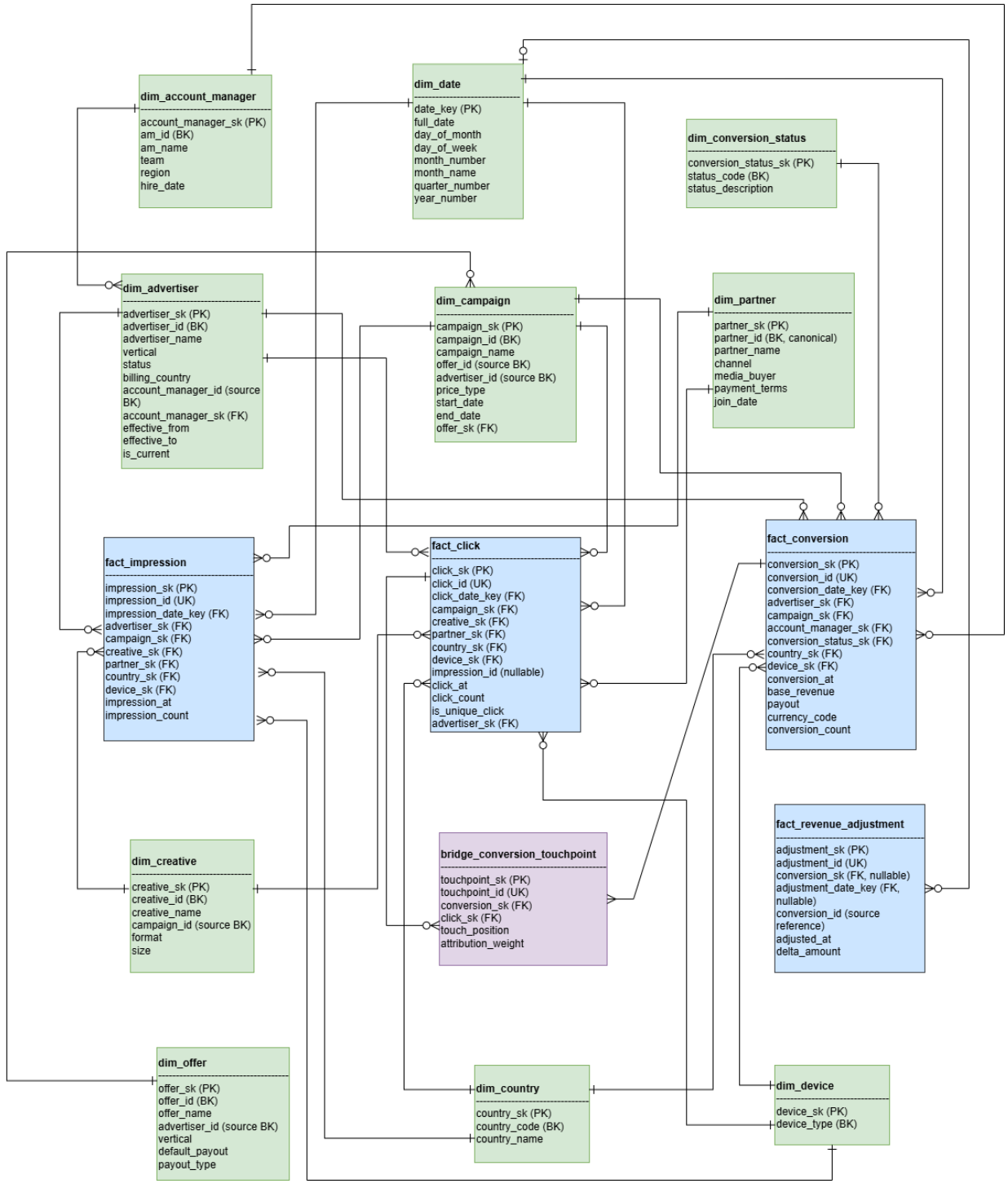


Figure 1: Proposed PulseTrack Enterprise Data Warehouse Dimensional Model

The proposed dimensional model includes four fact tables, ten dimensions and a bridge table for multi-touch attribution. Each fact table has a defined grain to support accurate KPI calculations. Shared dimensions provide consistent reporting across marketing activities, while the advertiser dimension uses SCD Type 2 to preserve historical account-manager assignments.

2.6 Measures and Additive Behaviour

PulseTrack uses different types of measures to evaluate marketing and financial performance. Some measures can be summed directly, while others must be calculated from aggregated values to avoid incorrect results.

The main measures and their aggregation rules are shown below.

Measure	Additive Behaviour	Calculation / Handling
Impressions	Additive	Count each valid impression once.
Clicks	Additive	Count each valid click once.
Conversions	Additive after deduplication	Count each distinct eligible conversion once.
Base Revenue	Additive after validation	Sum eligible conversion revenue using consistent currency and status rules.
Payout	Additive after validation	Sum valid payout amounts.
Revenue Adjustments	Additive	Sum valid positive and negative adjustment amounts without duplication.
Adjusted Revenue	Derived, then additive	Calculate base revenue plus valid adjustments for each conversion before aggregation.
Weighted Conversion Credit	Additive across valid attribution records	Sum the attribution weights assigned to each partner.
Attributed Revenue	Derived, then additive	Multiply eligible conversion revenue by valid attribution weights before aggregation.
CTR	Non-additive	Total clicks ÷ total impressions × 100.
Conversion Rate	Non-additive	Total eligible conversions ÷ total valid clicks × 100.
Revenue per Conversion	Non-additive	Total eligible revenue ÷ total eligible conversions.
Revenue per Click	Non-additive	Total eligible revenue ÷ total valid clicks.
Attribution Percentage	Non-additive	Weighted conversion credit ÷ total eligible conversions × 100.
Percentage Contribution	Non-additive	Selected entity's measure ÷ corresponding overall total × 100.

Revenue Calculations

Revenue adjustments must be applied to the correct conversion before calculating the final revenue.

Adjusted Revenue = Base Revenue + Sum of Valid Revenue Adjustments

For example, if a conversion generates $100 and later receives a -$20 adjustment, its adjusted revenue becomes $80.

The warehouse will preserve the original conversion revenue and individual adjustment records so that financial changes remain traceable.

Attribution Calculations

A conversion may involve several click touchpoints. The attribution weights determine how conversion credit and revenue are distributed among contributing partners.

For example, if a $100 conversion has two valid touchpoints with weights of 0.60 and 0.40, the attributed revenue would be $60 and $40 respectively.

The attribution weights must be validated before use. Distinct conversion counts will remain separate from weighted conversion credit to prevent double counting.

Aggregation Rules

Ratios such as CTR, conversion rate and revenue per click must be recalculated using aggregated totals rather than by averaging individual percentages.

Calculations must also handle zero denominators, use consistent reporting periods and apply the agreed conversion-status and currency rules.

These rules will be applied consistently across the warehouse, Data Mart and analytical reporting layers.

2.7 Surrogate Keys, Business Keys and Conformed Dimensions

The PulseTrack warehouse will use business keys to identify records from the source systems and surrogate keys to manage relationships within the warehouse.

A **business key** is an identifier from the original source, such as `advertiser_id`, `campaign_id` or `conversion_id`. A **surrogate key** is generated within the warehouse and is used to link fact records to the correct dimension records.

This distinction is especially important for historical data. For example, when an advertiser changes account managers, the original `advertiser_id` remains the same, but SCD Type 2 creates a new `advertiser_sk` for the updated version. This allows historical conversions to remain linked to the advertiser information that was valid at the time.

Conformed Dimensions

Dimensions such as `dim_date`, `dim_campaign` and `dim_partner` will be shared across relevant fact tables. Using consistent dimension definitions allows impressions, clicks, conversions and revenue to be compared without conflicting identifiers or business information.

Source Traceability and Missing References

Original event identifiers, including `impression_id`, `click_id`, `conversion_id` and `adjustment_id`, will be retained in their respective fact tables. These identifiers will help trace warehouse records back to their sources during validation and troubleshooting.

Some source records may have missing or unresolved relationships. Where the business event is valid, the warehouse will preserve the record using an appropriate unknown or unresolved dimension member rather than discarding it.

The original source identifier and validation status will also be retained to support later reconciliation.

2.8 Historical Preservation Using SCD Type 2

PulseTrack needs to maintain historical account-manager assignments so that past marketing performance is reported under the correct manager.

The `advertisers_raw` table contains current account-manager information, while `advertiser_accounts.jsonl` provides historical assignments through the `account_manager_history` field.

To preserve these changes, the warehouse will use Slowly Changing Dimension Type 2 (SCD Type 2) in `dim_advertiser`.

When an advertiser is assigned to a new account manager, the existing dimension record will be closed by updating its effective end date and current-record indicator. A new record will then be created with a different surrogate key, the updated manager assignment and a new effective start date.

The original `advertiser_id` will remain unchanged across both versions.

Historical Conversion Matching

When loading conversions, the warehouse will use the conversion timestamp to identify the advertiser record that was valid when the conversion occurred.

For example, if an advertiser moves from Manager A to Manager B on 1 September, conversions recorded before that date should remain associated with Manager A. Conversions from the new assignment period should be linked to Manager B.

The manager information available in the conversion source will also be compared with the CRM assignment history to identify possible inconsistencies.

Late-Arriving Historical Changes

Some historical assignments or conversion records may arrive after the relevant period has already been processed.

The pipeline must therefore support correcting affected historical dimension versions and fact references without creating duplicate conversions. This approach allows account-manager performance to be analysed accurately across different periods, even when advertiser assignments change.

2.9 How the Dimensional Model Supports the Required KPIs

The proposed dimensional model is designed to support the business KPIs identified in Question 1. Each measure will be calculated using the relevant fact tables, while shared dimensions will allow results to be compared across advertisers, campaigns, partners and reporting periods.

The table below shows how the warehouse will support the main analytical requirements.

Analytical Requirement	Warehouse Support
Impressions and Clicks	`fact_impression` and `fact_click` store valid marketing events separately to support accurate counts.
Click-Through Rate (CTR)	Calculate clicks divided by impressions using aggregated values at the same reporting level.
Conversions and Conversion Rate	`fact_conversion` stores distinct business conversions after deduplication. Conversion rates use eligible conversions and compatible click counts.
Revenue and Adjusted Revenue	`fact_conversion` stores base revenue, while `fact_revenue_adjustment` stores financial changes. Adjusted revenue is calculated without duplicating conversions.

Revenue per Conversion / Click	Calculate eligible revenue divided by the corresponding conversion or click count.
Partner Attribution	`bridge_conversion_touchpoint` links conversions to contributing clicks. Valid attribution weights support weighted conversion credit and attributed revenue.
Campaign Rankings	Shared campaign and date dimensions support comparisons and rankings based on selected KPIs.
Percentage Contribution	Compare each advertiser's, campaign's or partner's selected measure with the corresponding total.
Account-Manager Performance	SCD Type 2 in `dim_advertiser` preserves historical assignments, allowing conversions to be reported under the correct manager.
Period Comparisons	`dim_date` supports daily, monthly, quarterly and yearly analysis, including comparisons between reporting periods.
Late Revenue Adjustments	Adjustment records remain linked to their original conversions so that financial results can be corrected without increasing conversion counts.

The model keeps marketing events, conversions and revenue adjustments at their appropriate grains. This reduces the risk of double counting when calculating KPIs across different business dimensions.

During implementation, these relationships and calculations will be validated using the cleaned source data. The same KPI definitions will then be used in the Data Mart, semantic model and analytical reports.

Question 3 – Data Platform Architecture and ETL/ELT Tool Selection

3.1 Technology Selection

PulseTrack requires a data platform that can process relational CSV data, JSONL documents and conversion event logs. The platform must also handle duplicate records, inconsistent identifiers, historical changes and late-arriving revenue adjustments.

The selected technologies should support reliable ingestion, data cleaning, warehouse loading, scheduling, monitoring and recovery. The design should also be practical to implement and maintain using the available environment.

Three technology approaches were considered:

Approach	Technology Stack	Main Considerations
A	Python, PostgreSQL SQL and Apache Airflow	Supports custom data processing and SQL-based transformations with scheduled pipeline execution.
B	Apache NiFi, PostgreSQL SQL and Apache Airflow	Provides visual data ingestion and routing alongside database transformations and workflow orchestration.
C	Apache Spark, dbt, Apache Airflow and PostgreSQL	Provides distributed data processing and structured SQL transformation workflows, but introduces additional operational complexity.

The approaches will be evaluated using weighted criteria based on PulseTrack's technical requirements. The final selection will consider processing capability, reliability, maintainability and scalability, rather than familiarity with a particular tool.

3.2 Weighted Technology Evaluation

The three technology approaches were compared based on PulseTrack's data processing and operational requirements. The evaluation considers source compatibility, data quality, incremental processing, scalability and ease of maintenance.

A qualitative scoring method was used, with each criterion assigned a priority based on its importance to the project.

Evaluation Scale

- **Priority:** High = 3, Medium = 2
- **Suitability:** Strong = 3, Suitable = 2, Limited = 1

The ratings represent an assessment of each approach's suitability for PulseTrack, rather than results from performance testing.

Technology Comparison

Evaluation Criteria	Priority	Approach A	Approach B	Approach C
CSV, JSONL and event-log ingestion	High	Strong	Strong	Strong
Data cleaning, transformation and reconciliation	High	Strong	Suitable	Strong
Incremental processing, replay and recovery	High	Strong	Strong	Strong
Scheduling and monitoring	Medium	Strong	Strong	Strong
Scalability	Medium	Suitable	Strong	Strong
Implementation and maintenance simplicity	High	Strong	Suitable	Limited
Weighted Total		**46**	**42**	**42**

Approaches:

- **A:** Python + PostgreSQL SQL + Apache Airflow
- **B:** Apache NiFi + PostgreSQL SQL + Apache Airflow
- **C:** Apache Spark + dbt + Apache Airflow + PostgreSQL

The weighted total is calculated by multiplying each criterion's priority value by its suitability rating and adding the results. The maximum possible score is 48.

Evaluation Findings

Approach A – Python, PostgreSQL and Airflow

This approach is suitable for processing PulseTrack's CSV, JSONL and event-log data. Python supports custom validation and cleaning, PostgreSQL handles warehouse storage and SQL transformations, and Airflow manages pipeline scheduling and monitoring. It also requires fewer services to maintain compared with the other approaches.

Approach B – NiFi, PostgreSQL and Airflow

NiFi provides visual ingestion, data routing and error-handling capabilities. However, combining NiFi with Airflow introduces additional configuration and maintenance. Complex data reconciliation may also require more coordination between NiFi and SQL transformations.

Approach C – Spark, dbt, Airflow and PostgreSQL

Spark provides distributed processing capabilities, while dbt supports SQL transformations and testing. This approach is suitable for larger workloads but introduces additional tools and operational complexity that may not be necessary for the supplied PulseTrack dataset.

Preferred Technology Approach

Based on this preliminary evaluation, **Approach A – Python, PostgreSQL and Apache Airflow – is selected for the proposed architecture**.

This combination provides the main capabilities needed for data ingestion, cleaning, transformation, orchestration and warehouse loading while remaining practical to implement and maintain.

Although NiFi and Spark offer advantages in particular areas, the current dataset does not require their additional infrastructure. The selected approach will be validated during implementation, and its limitations will be documented where relevant.

3.3 Final Technology Decision and Responsibilities

Based on the technology evaluation, PulseTrack will use **Python, PostgreSQL and Apache Airflow** as the main components of the proposed data platform.

The selected approach follows an ELT-oriented design, supported by Python-based data extraction and preparation. Raw data will be preserved before cleaning, while SQL transformations will prepare the warehouse and analytical datasets.

Selected Technologies

Technology	Responsibility in PulseTrack
Python	Read CSV, JSONL and conversion event-log files; validate source formats; standardise selected fields; and identify malformed or duplicate records.
PostgreSQL	Store staging data, cleaned datasets, dimensional warehouse tables and analytical Data Marts. SQL will support deduplication, joins, historical lookups and KPI calculations.
Apache Airflow	Schedule and coordinate pipeline tasks, manage dependencies, support retries and record pipeline execution results.
Local File Storage	Preserve original source files and ingestion metadata for traceability and reprocessing.
SQL and Python Validation	Perform data-quality checks, source-to-target reconciliation and verification of warehouse measures.

Proposed Data Processing Approach

The platform will organise data processing into three main layers:

Bronze – Raw Data

Original CSV, JSONL and event-log records will be retained without losing their source information. Ingestion metadata will be recorded to support auditing and replay.

Silver – Cleaned Data

Python and PostgreSQL transformations will prepare consistent datasets by handling duplicate conversions, standardising partner identifiers, validating references and identifying problematic records.

Gold – Enterprise Data Warehouse

Cleaned records will be loaded into the fact tables, dimensions and attribution bridge defined in Question 2. The warehouse will preserve historical account-manager assignments using SCD Type 2.

Analytical Data Marts will then provide business-ready datasets for KPI calculations and reporting.

Operational Considerations

Airflow will coordinate the processing stages and ensure that dependent tasks run in the correct order.

The pipeline will be designed to support incremental loading, safe reprocessing and recovery from failed runs. Source identifiers and processing logs will help prevent duplicate records and support reconciliation.

The platform will also apply appropriate database access controls and avoid exposing unnecessary sensitive source fields in analytical outputs.

This architecture is intended to provide a reliable and maintainable solution for the supplied PulseTrack dataset. Its operational behaviour and data-quality results will be verified during implementation.

3.4 Proposed Data Platform Architecture

The proposed PulseTrack data platform follows a layered architecture using Python, PostgreSQL and Apache Airflow. It separates raw data ingestion, cleaning, warehouse processing and analytical reporting so that each stage can be managed and validated independently.

The architecture is designed around the Bronze, Silver and Gold approach, with an additional Data Mart and reporting layer.

3.4.1 Architecture Layers

Layer	Main Responsibility
Source Systems	Relational CSV files, PostgreSQL source tables, advertiser and partner JSONL documents, and the Kafka-style conversion event-log export. The ingestion process will support both file-based loading and extraction from PostgreSQL.
Ingestion	Python scripts automatically load the eleven CSV datasets into the existing `pulsetrack_raw` PostgreSQL tables, extract relational records into Bronze, and ingest JSONL documents and conversion-event logs directly. CSV loading includes header validation, incremental insertion of additional row occurrences, repeated-load protection and transactional error handling.
Bronze	Original source data is preserved for auditing, troubleshooting and reprocessing. Parsed records are also loaded into PostgreSQL staging tables.
Silver	Data is cleaned and standardised. This includes conversion deduplication, partner identifier matching, timestamp conversion and data-quality validation.
Gold – Enterprise Data Warehouse	Cleaned data is loaded into the fact tables, dimensions and conversion-touchpoint bridge defined in Question 2. Historical advertiser assignments are preserved using SCD Type 2.
Data Mart	Business-focused tables and views provide prepared measures for campaign, advertiser, partner and revenue analysis.
OLAP and Reporting	Analytical queries and dashboards support KPI monitoring, filtering, period comparisons and performance rankings.

3.4.2 Pipeline Orchestration

Apache Airflow 3.3.0 was implemented to orchestrate the complete PulseTrack data-processing pipeline through eleven sequential tasks.

The first task, `load_source_csv`, automatically loads or verifies the eleven original CSV datasets in the `pulsetrack_raw` PostgreSQL schema. This is followed by PostgreSQL-to-Bronze ingestion, JSONL ingestion, event-log ingestion, Silver processing, Gold warehouse loading, Data Mart and OLAP refresh, and automated validation.

Task dependencies ensure that downstream processing begins only after the preceding task completes successfully.

The DAG is configured for manual execution using `schedule=None`, with a maximum of one active run, one retry per failed task and a two-minute retry delay. Automatic daily scheduling was not enabled during the capstone testing.

This demonstrates automated processing from available CSV, JSONL and event-log sources through the analytical reporting layer, once the required PostgreSQL database schemas and execution environment have been prepared.

3.4.3 Monitoring, Recovery and Data Quality

The architecture will include processing logs, validation results and source-to-target reconciliation checks.

Malformed records and unresolved references will be recorded for investigation rather than silently discarded.

The implemented pipeline supports configured retries, idempotent reruns and append-only incremental CSV ingestion. Historical reprocessing is supported through repeatable transformations and Data Mart refreshes, while automatic date-specific backfill remains a future improvement.

Source identifiers and processing metadata will be used to prevent duplicate records when tasks are rerun.

3.4.4 Security and Data Traceability

PostgreSQL access will be controlled through appropriate database roles and permissions. Connection credentials will be managed separately from application code.

Original source identifiers, ingestion timestamps and transformation records will support data lineage and troubleshooting. Sensitive source information, such as IP addresses, will not be exposed unnecessarily through analytical datasets.

The overall architecture is intended to provide a maintainable platform that supports accurate KPI calculations, historical reporting and reliable recovery when source data or pipeline execution requires correction.

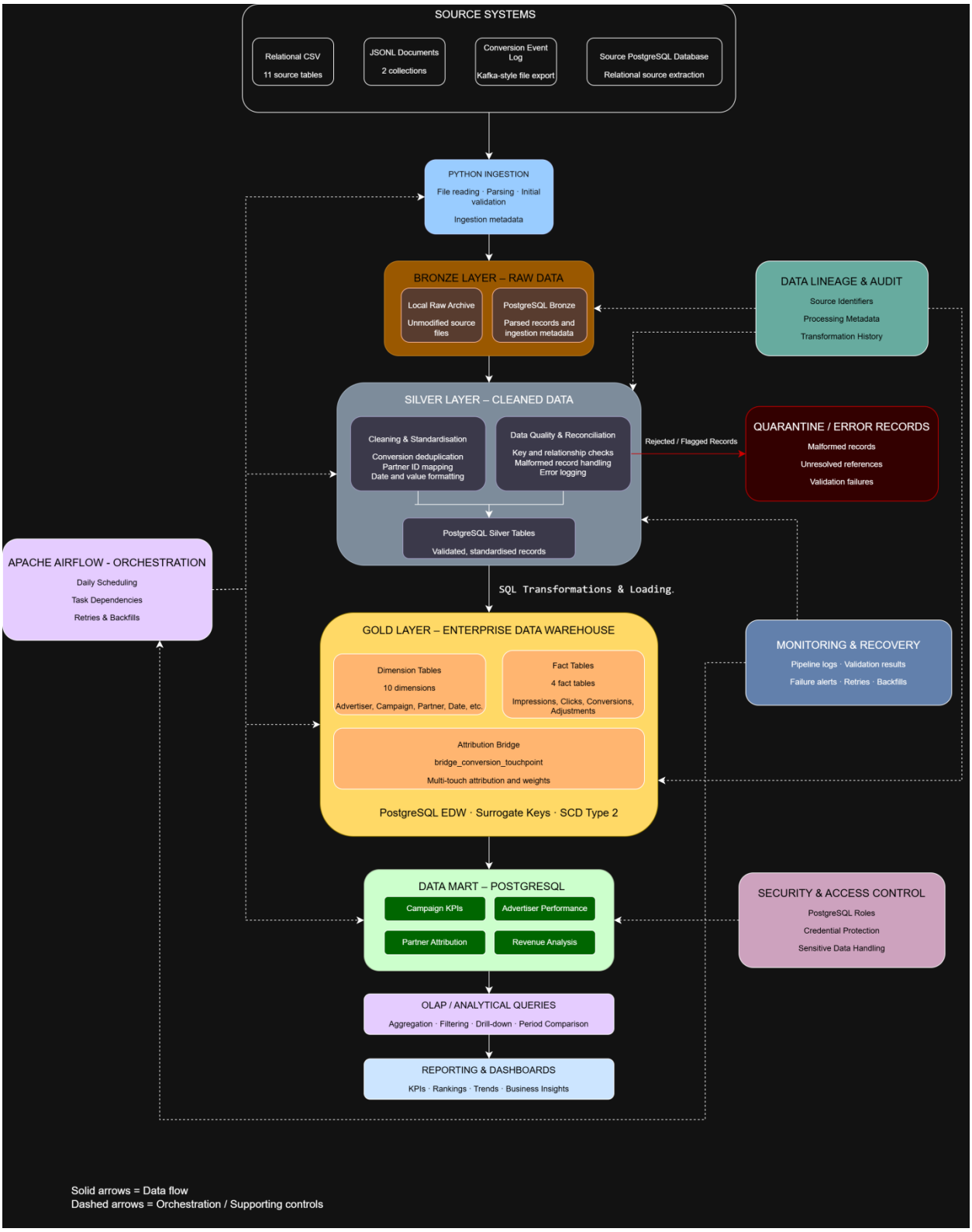


Figure 2: PulseTrack Data Platform Architecture

Question 4 – ETL/ELT Pipeline Implementation

4.1 Implementation Overview

The PulseTrack ETL/ELT pipeline was implemented using Python, PostgreSQL and Apache Airflow, following the architecture selected in Question 3.

The pipeline processes relational source tables, JSONL documents and a Kafka-style conversion event-log export. Data is organised into Bronze, Silver and Gold layers to separate raw data preservation, cleansing and dimensional warehouse loading.

Python handles ingestion, source parsing and pipeline execution, while PostgreSQL performs the main transformations and stores the warehouse tables. Apache Airflow coordinates the processing stages, manages task dependencies and records execution results.

The implementation follows the dimensional model developed in Question 2, including the fact tables, shared dimensions, conversion-touchpoint bridge and SCD Type 2 advertiser history.

4.2 Bronze Layer - Source Ingestion and Raw Data Preservation

The Bronze layer preserves source records before business transformations are applied. This allows the original data to be examined, reconciled and reprocessed when necessary.

Four Python scripts support source preparation and Bronze ingestion:

Script	Responsibility
`load_source_csv.py`	Automatically loads the eleven CSV datasets into `pulsetrack_raw`, validates CSV headers and supports append-only incremental loading and idempotent reruns.
`ingest_postgres.py`	Extracts the eleven PostgreSQL raw source tables into Bronze while preventing repeated ingestion of unchanged source data.
`ingest_jsonl.py`	Loads advertiser-account and partner-profile JSONL records while preserving their original lines.
`ingest_event_log.py`	Loads conversion event-log records, including malformed and duplicate deliveries, without discarding the original records.

The eleven relational datasets were originally supplied as CSV files. An automated Python loader, `scripts/load_source_csv.py`, is used to load these datasets into the existing PostgreSQL `pulsetrack_raw` tables.

The loader validates source-file availability and CSV headers against the destination table structures. It uses PostgreSQL bulk loading through temporary staging tables and `EXCEPT ALL` comparisons to identify additional source-row occurrences.

When existing source records match the CSV files, no additional rows are inserted. When the CSV contains additional records without removing existing ones, the loader inserts only the additional row occurrences.

The eleven source tables are processed within a single database transaction. If validation or loading fails, the transaction is rolled back instead of committing a partially completed import.

The implementation supports initial loading, append-only incremental ingestion and safe repeated execution. Corrections or deletions of existing source records are not automatically applied and require a separate resolution process.

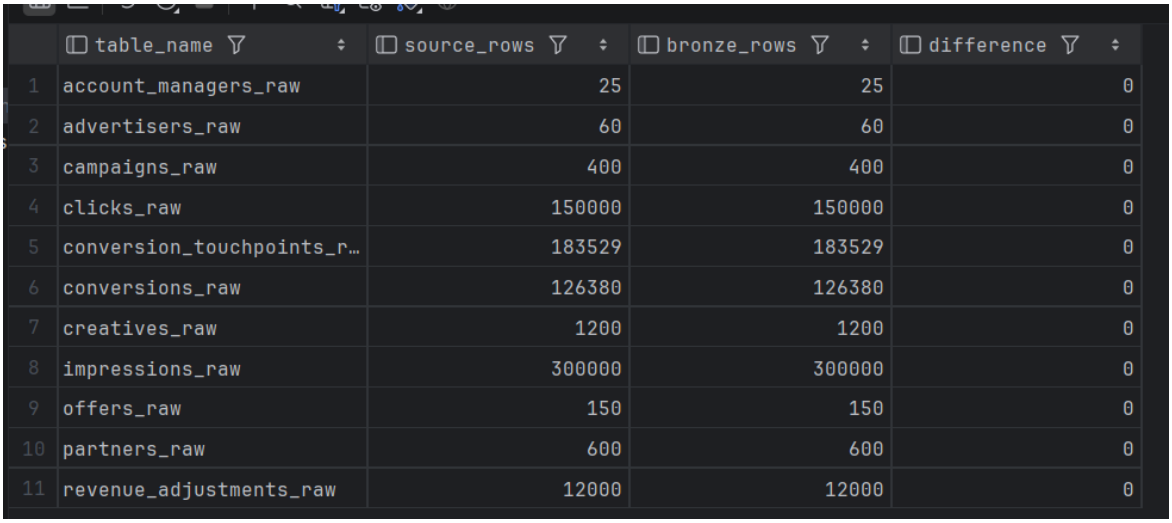
After CSV loading, `ingest_postgres.py` transfers the PostgreSQL source data into Bronze. The JSONL documents and conversion-event log are ingested directly from their respective files.

The initial PostgreSQL schemas and tables must already exist before executing the CSV loader. The loader automates data population but does not create the database schema.

4.2.1 Bronze Ingestion Results

The initial ingestion produced the following results:

Source category	Bronze records
Relational source tables (11)	774,344
JSONL documents (2 files)	660
Conversion event-log lines	183,162
Total	**958,166**



	table_name	source_rows	bronze_rows	difference
1	account_managers_raw	25	25	0
2	advertisers_raw	60	60	0
3	campaigns_raw	400	400	0
4	clicks_raw	150000	150000	0
5	conversion_touchpoints_r…	183529	183529	0
6	conversions_raw	126380	126380	0
7	creatives_raw	1200	1200	0
8	impressions_raw	300000	300000	0
9	offers_raw	150	150	0
10	partners_raw	600	600	0
11	revenue_adjustments_raw	12000	12000	0

Figure 3: Source-to-Bronze Reconciliation Results

Source-to-Bronze reconciliation confirmed that the number of loaded records matched the available source records.

For the JSONL and event-log datasets, the original text, source filename and line number were retained. These fields support traceability and allow problematic records to be investigated without losing the original input.

The ingestion scripts were also tested for idempotency. Reprocessing unchanged source data did not create additional Bronze records, ensuring that repeated execution does not inflate source counts.

Automated CSV Ingestion Testing

The automated CSV loader was tested using a separate PostgreSQL database, `pulsetrack_csv_test`, to avoid modifying the validated capstone database.

The first execution successfully imported all 774,344 records from the eleven CSV files into the corresponding raw tables.

A second execution reported `ALREADY MATCHES` for all eleven tables, confirming that repeated processing did not create additional source records.

Incremental insertion was tested by removing one record from the isolated test database while retaining the original CSV file. The next loader execution detected and restored exactly one missing record. A subsequent execution confirmed that the source tables again matched the CSV files.

Changed-data detection was also tested using a temporary PostgreSQL table. The `EXCEPT ALL` comparison reported no difference for identical records and detected a deliberately modified record. The simulated modification was rolled back.

The loader was also executed against the main `pulsetrack_capstone` database and verified that all 774,344 existing source records matched their corresponding CSV files.

These tests demonstrate initial automated source loading, repeated-load idempotency, recovery of a missing source row and record-comparison logic. A complete end-to-end test involving newly appended historical records or corrected source records was not performed.

4.3 Silver Layer – Data Cleansing and Standardisation

The Silver layer prepares the raw Bronze data for use in the Enterprise Data Warehouse. Its main responsibilities include standardising inconsistent values, identifying duplicates, validating relationships and preserving records needed for historical analysis.

Python scripts execute the processing stages, while PostgreSQL stores the cleaned datasets in the `silver` schema.

4.3.1 Reference Data Processing

The `process_dimensions.py` script processes the reference entities required for dimensional modelling.

These include account managers, advertisers, partners, offers, campaigns and creatives. During processing, inconsistent date formats and selected source attributes are standardised.

The completed Silver reference tables contain:

Silver table	Records
`silver.account_managers`	25
`silver.advertisers`	60
`silver.partners`	600
`silver.offers`	150
`silver.campaigns`	400
`silver.creatives`	1,200
Total	**2,435**

The `process_jsonl_profiles.py` script also processes advertiser-account and partner-profile information from the JSONL sources.

Of the 600 partner profiles, 560 identifiers matched the relational partner reference data, while 40 could not be matched. Unmatched information was preserved rather than assigned to an incorrect partner.

Historical account-manager information from advertiser profiles was prepared for SCD Type 2 processing in the Gold layer.

4.3.2 Transactional Data Processing

The `process_transactions.py` script processes impressions, clicks, conversion history, conversion touchpoints and revenue adjustments.

The transactional results are summarised below.

Silver dataset	Records
Impressions	300,000
Clicks	150,000
Conversion history	126,380
Conversion touchpoints	183,529
Revenue adjustments	12,000

An important issue was the presence of repeated conversion IDs.

The conversion source contained 126,380 records but only 90,000 distinct conversion IDs. The additional 36,380 rows represented repeated conversion records or historical snapshots.

Instead of deleting these records, the implementation preserves conversion history in `silver.conversion_history`. The Gold processing stage subsequently selects one current record per business conversion.

This approach prevents duplicate conversion counts while retaining the information needed to investigate historical changes.

The pipeline also handles inconsistent conversion statuses, timestamps, currency values and missing relationships. Ambiguous values are preserved for investigation rather than silently treated as valid business values.

4.3.3 Conversion Event-Log Processing

The `process_event_log.py` script processes the Kafka-style conversion event-log export.

The script parses event records, identifies repeated deliveries using event identifiers and separates malformed JSON records.

The processing results were:

Processing outcome	Records
Original event-log lines	183,162
Valid unique events	178,600
Duplicate event deliveries	4,378
Malformed records	184



Figure 4: Silver Event-Log Processing and Reconciliation Results

The 184 malformed records were preserved in `silver.event_log_errors`, while the 178,600 unique valid events were stored in `silver.conversion_events`.

This allows valid events to be processed without losing evidence of duplicate deliveries or invalid source records.

The original event-log lines remain available in Bronze for traceability and reprocessing.

4.3.4 Silver Layer Validation

The Silver processing results were verified using SQL checks for record counts, uniqueness and processing outcomes.

The event-log reconciliation confirmed that all 183,162 Bronze lines were accounted for as valid unique events, duplicate deliveries or malformed records.

These results demonstrate that the pipeline can identify duplicates and isolate malformed event records without discarding the original source information.

The cleaned Silver datasets provide the foundation for the Gold warehouse, including historical advertiser processing, fact loading and multi-touch attribution.

4.4 Gold Layer – Enterprise Data Warehouse Implementation

The Gold layer is the Enterprise Data Warehouse (EDW) of PulseTrack. It stores cleaned and validated business data in a dimensional model designed to support accurate marketing performance analysis.

The implementation follows the warehouse design established in Question 2, consisting of ten dimension tables, four fact tables and a conversion-touchpoint bridge. Surrogate keys are used to link business events with their corresponding dimensions, while original source identifiers are preserved for traceability.

The `load_gold.py` script coordinates the Gold loading process. Dimensions are processed before facts so that the required dimensional references are available when transactional records are loaded.

4.4.1 Dimension Table Implementation

Ten dimensions were created in the PostgreSQL `gold` schema to support analysis by date, advertiser, account manager, partner, offer, campaign, creative, country, device and conversion status.

The implemented dimension tables contain the following records:

Gold dimension	Records
dim_date	589
dim_account_manager	25
dim_advertiser	127
dim_partner	600
dim_offer	150
dim_campaign	400
dim_creative	1,200
dim_country	10
dim_device	6
dim_conversion_status	6

The advertiser dimension contains more records than the 60 source advertisers because it preserves historical versions using Slowly Changing Dimension Type 2 (SCD Type 2).

Shared dimensions support consistent grouping and filtering across impressions, clicks, conversions and revenue adjustments.

4.4.2 Historical Advertiser Processing – SCD Type 2

Historical account-manager assignments were implemented in `gold.dim_advertiser` using SCD Type 2.

The resulting dimension contains 127 records representing 60 advertisers. These include 60 current records, 14 previous versions supported by documented assignment history and 53 inferred earlier versions.

Where the source did not identify the manager responsible before the earliest recorded assignment, the earlier version was preserved with an unknown manager rather than assigning an unsupported historical value.

Each historical version has its own surrogate key and effective period. Fact records are matched to the version valid at the event timestamp.

This approach enables historical performance to remain associated with the correct documented account-manager assignment even after changes occur.

4.4.3 Fact and Bridge Table Implementation

Four fact tables and one attribution bridge were populated from the cleaned Silver datasets.

Gold table	Records	Grain
`fact_impression`	300,000	One unique impression
`fact_click`	150,000	One unique click
`fact_conversion`	90,000	One distinct business conversion
`fact_revenue_adjustment`	12,000	One revenue adjustment
`bridge_conversion_touchpoint`	183,529	One conversion-to-click touchpoint

The conversion fact contains 90,000 records rather than the 126,380 Silver conversion-history records. This is intentional: repeated conversion snapshots are retained in Silver, while Gold represents each distinct business conversion once.

Revenue adjustments are stored separately so that changes in revenue do not increase the number of conversions. The 721 adjustments without a matched conversion are retained for reconciliation but are not assigned to another conversion.

The attribution bridge preserves multiple touchpoints for each conversion. During analytical aggregation, attribution weights are normalised where necessary to prevent rounding differences from overstating conversion credit or revenue.

4.4.4 Business Rules and Data Integrity

The Gold loading process applies business rules before records contribute to financial KPIs.

Revenue eligibility requires an approved, non-test conversion with a recognised USD amount and a non-negative base revenue value. Eligible matched USD adjustments are applied to the corresponding conversion without creating another conversion record.

Unresolved source relationships are not automatically treated as valid matches. For example, clicks with unverified partner identifiers are retained, but their partner references remain unresolved until supported by reliable source evidence.

The pipeline also preserves records with missing event dates, while excluding them from date-based aggregates where the reporting date cannot be established.

These decisions prioritise accurate reporting and traceability over artificially complete relationships.

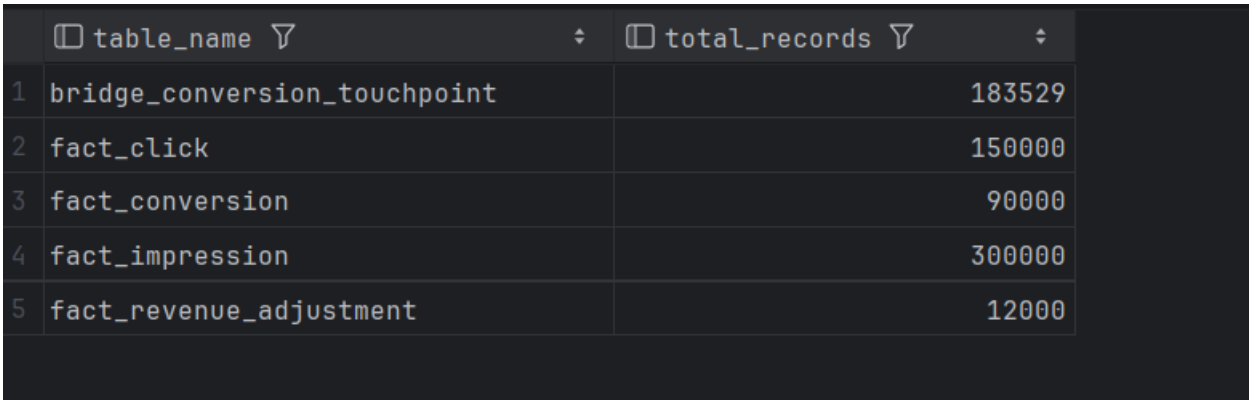
4.4.5 Gold Loading Validation

Gold loading was tested for record counts, conversion uniqueness, historical advertiser assignment and safe repeated execution.

The verification results confirmed that the intended fact and bridge row counts were maintained. Re-running the Gold loading process did not create duplicate business events.

Gold dimension and fact loading also uses transactional processing and audit records, allowing failures to be investigated and affected stages to be safely rerun.

The completed Gold warehouse provides the foundation for the business-focused Data Mart implemented in Question 7.



	table_name	total_records
1	bridge_conversion_touchpoint	183529
2	fact_click	150000
3	fact_conversion	90000
4	fact_impression	300000
5	fact_revenue_adjustment	12000

Figure 5: Gold Enterprise Data Warehouse Fact and Bridge Record Counts

4.5 Apache Airflow Pipeline Orchestration

Apache Airflow was implemented to coordinate the PulseTrack data-processing pipeline. Instead of executing each Python script manually, Airflow manages the order of execution, task dependencies, retries and monitoring.

The implementation uses Apache Airflow 3.3.0 with LocalExecutor in an Ubuntu environment running through Windows Subsystem for Linux (WSL 2). The existing PulseTrack Python scripts and PostgreSQL database were reused without rebuilding the processing logic.

A Directed Acyclic Graph (DAG) named `pulsetrack_pipeline` was created to represent the complete processing workflow.

4.5.1 DAG Design and Task Dependencies

The PulseTrack Airflow DAG contains eleven sequential tasks, beginning with automated CSV source loading.

The verified execution order is shown below.

Task	Responsibility
`load_source_csv`	Automatically loads or verifies the eleven CSV source datasets in PostgreSQL.
`ingest_postgres`	Extracts relational source records into Bronze.
`ingest_jsonl`	Ingests advertiser and partner JSONL records.
`ingest_event_log`	Preserves conversion event-log records.
`process_dimensions`	Cleans and standardizes reference data.
`process_jsonl_profiles`	Processes advertiser and partner profile information.
`process_transactions`	Processes impressions, clicks, conversions, touchpoints and revenue adjustments.
`process_event_log`	Validates and deduplicates conversion events.
`load_gold`	Loads warehouse dimensions, facts and attribution bridge records.
`load_mart`	Refreshes the six Data Marts and monthly OLAP cache.
`validate_pipeline`	Executes automated reconciliation and analytical validation checks.

Airflow uses `BashOperator` to execute the existing Python scripts through the configured project Python environment.

The `load_source_csv` task runs before PostgreSQL-to-Bronze ingestion. Task dependencies ensure that downstream processing begins only after the preceding task completes successfully.

The sequential architecture simplifies dependency management and prevents later stages from proceeding when an earlier task fails.

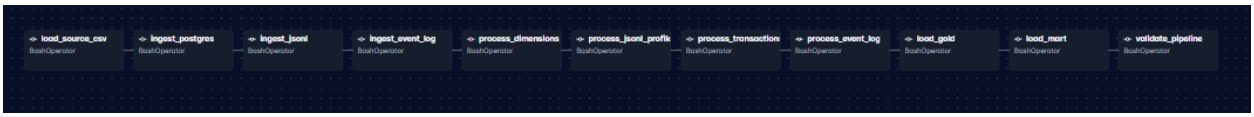


Figure 6: PulseTrack Airflow DAG Showing Eleven Processing and Validation Tasks

4.5.2 Execution Configuration

The DAG is configured for manual triggering using `schedule=None`, allowing controlled execution and testing.

The configuration also includes a maximum of one active DAG run, one retry for failed tasks and a two-minute retry delay.

These settings help reduce overlapping executions and provide a basic recovery mechanism for temporary failures.

Although the architecture supports future scheduled execution, automatic daily scheduling was not enabled during the demonstrated capstone tests.

4.5.3 Successful End-to-End Execution

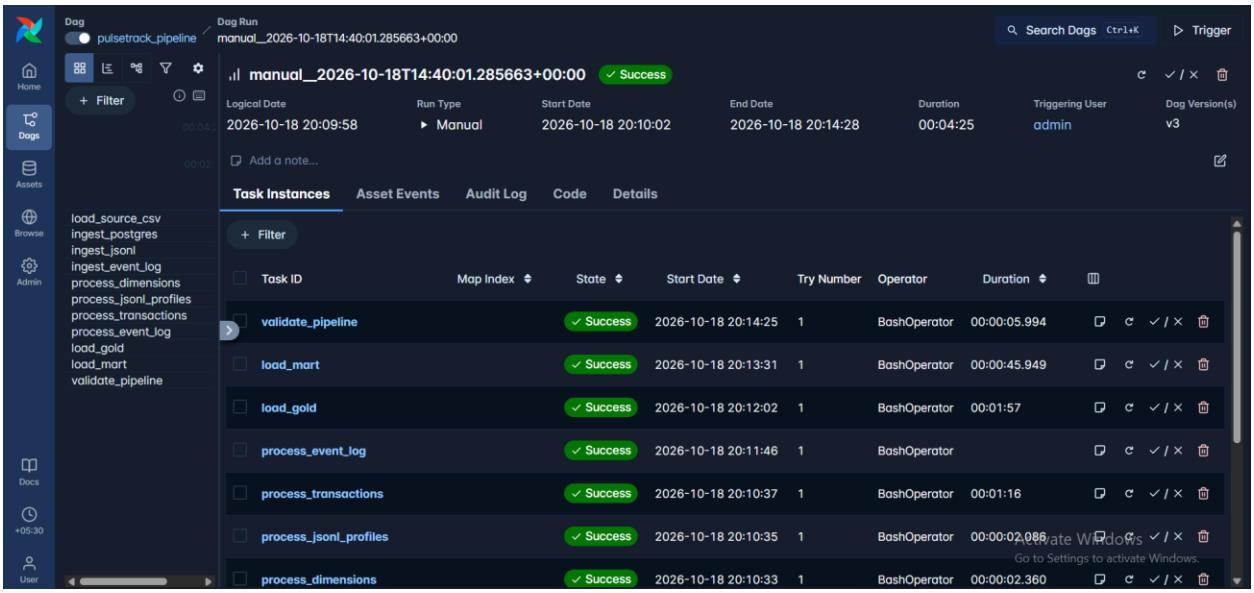


Figure 7: Successful Execution of the Eleven-Task PulseTrack Airflow Pipeline

The eleven-task PulseTrack pipeline was executed successfully through the Apache Airflow web interface on 18 October 2026.

The verified execution used DAG version 3 and completed in approximately four minutes and twenty-five seconds.

All eleven tasks reported successful execution, including the automated CSV-loading task and the final validation task.

During this execution, the CSV loader verified that all 774,344 existing relational source records matched their corresponding CSV files without inserting duplicate records.

The downstream pipeline processed the available source data through the Bronze, Silver, Gold, Data Mart and semantic analytical layers.

The final validation confirmed that all fourteen Gold-to-Mart reconciliation checks passed. Attribution reconciliation and OLAP materialized-cache validation also passed.

Airflow reported a successful DAG execution, and the final validation command exited with return code 0.

This demonstrates that automated CSV ingestion, ETL/ELT processing, warehouse loading and analytical validation operate together within one orchestrated workflow.

The successful run used an already initialized PostgreSQL environment. Initial database schema creation and a complete end-to-end historical backfill were not part of this execution.

4.5.4 Monitoring and Automated Validation

Airflow provides task execution status, timestamps, durations and logs for monitoring pipeline activity.

The final `validate_pipeline` task automatically checks the consistency of the analytical outputs after the Data Mart and OLAP cache have been refreshed.

The validation includes:

- Fourteen Gold-to-Mart reconciliation checks.
- Attribution credit and revenue reconciliation.
- Comparison of the OLAP monthly materialized cache against its source semantic view.

The successful execution reported all fourteen reconciliation checks as PASS, together with successful attribution and OLAP cache validation.

If an essential check fails, the validation script exits with a non-zero status, allowing Airflow to identify the failed task.

The implementation therefore combines pipeline execution with automated verification, rather than relying solely on successful SQL execution as evidence of data correctness.

4.5.5 Database Connectivity and Credential Handling

Airflow executes the PulseTrack Python scripts through the project's existing Python environment.

PostgreSQL credentials are maintained separately from the source code using a user-specific PostgreSQL password file. This configuration was tested successfully without supplying a temporary `PGPASSWORD` environment variable.

The local setup demonstrates separation of credentials from application code. For production deployment, a dedicated database account, restricted permissions and centralised secrets management would provide stronger security.

The working Airflow implementation provides the operational foundation for the incremental processing, recovery, reconciliation and audit mechanisms discussed in Questions 5 and 6.

4.6 Implementation Summary and Validation

The PulseTrack ETL/ELT pipeline integrates CSV loading, PostgreSQL extraction, JSONL ingestion and conversion event-log processing through eleven sequential Apache Airflow tasks. The implementation follows a Bronze, Silver and Gold architecture, supported by analytical Data Marts and an OLAP semantic layer.

Bronze preserves the supplied source records, while Silver performs data cleansing, standardisation, deduplication and relationship validation. Gold maintains ten dimensions, four fact tables and a conversion-touchpoint bridge, supporting consistent KPI calculations and historical advertiser assignments. Six Data Mart tables provide business-oriented reporting datasets.

The pipeline was executed successfully on 18 October 2026 using DAG version 3. All eleven tasks completed, and the final validation passed fourteen Gold-to-Mart reconciliation checks, attribution reconciliation and OLAP cache consistency checks. Separate ingestion tests also confirmed idempotent CSV loading and recovery of a missing source record.

The implementation demonstrates an integrated data-processing and analytical reporting workflow. Initial database-schema creation remains a prerequisite, while automated historical backfill and corrections to existing CSV records remain outside the verified scope. These processing considerations are examined further in Question 5.

Question 5 – Incremental Processing, Idempotency, Late-Arriving Data and Recovery

5.1 Processing Strategy

The PulseTrack pipeline must support repeated processing without creating duplicate business records or producing incorrect analytical results. It must also handle changes to existing records, delayed revenue adjustments, historical corrections and recovery from failed processing stages.

To achieve this, the implementation uses different processing strategies across the warehouse layers.

The pipeline begins with automated CSV ingestion into the `pulsetrack_raw` PostgreSQL schema. The `load_source_csv.py` script uses temporary staging tables and `EXCEPT ALL` comparisons to identify additional source-row occurrences while preventing repeated insertion of unchanged records. Existing records that disappear from the CSV cause the import to stop rather than silently overwrite data.

In Bronze, source records are preserved with identifiers and ingestion metadata. PostgreSQL-based ingestion uses comparisons to avoid adding duplicate source rows during repeated execution, while JSONL and event-log landing tables use source filenames and line numbers to identify previously loaded records.

In Silver, conversion history is preserved while repeated event deliveries are identified separately from valid business changes. This prevents duplicate messages from being treated as new business events.

In Gold, database upsert operations allow existing warehouse records to be updated or restored without creating duplicate fact records. Historical advertiser assignments use SCD Type 2 to preserve the correct business context for earlier conversions.

Gold loading also uses database transactions so that incomplete changes can be rolled back if a processing stage fails.

The Data Mart uses a transactional full-refresh approach, followed by the refresh of the monthly OLAP materialized cache. Although this is not incremental mart loading, it provides a consistent analytical result after successful execution.

Apache Airflow coordinates the processing stages, records task status and supports configured retries. The following sections present the actual tests conducted to verify these behaviours.

5.2 Duplicate Delivery and Idempotency

Idempotency means that executing the same processing operation multiple times produces the same final analytical result without creating unintended duplicate records.

This is important for PulseTrack because source files may be delivered more than once, event messages may be repeated, and pipeline tasks may need to be rerun after a failure.

5.2.1 Idempotency Across the Processing Layers

The implementation applies duplicate-prevention mechanisms at different stages.

In Bronze, the relational ingestion process uses PostgreSQL comparisons to identify records already loaded. JSONL and event-log ingestion use source filenames and line numbers as unique identifiers, preventing the same unchanged file lines from being inserted repeatedly.

In Silver, conversion event records are deduplicated using event identifiers. The original event-log dataset contained 4,378 duplicate deliveries, which were identified separately from the 178,600 unique valid events. Conversion history is preserved so that repeated snapshots do not result in lost historical information.

In Gold, unique business identifiers and upsert operations allow existing fact records to be updated or restored without introducing duplicate business events.

These mechanisms allow completed processing stages to be rerun while maintaining consistent record counts.

5.2.2 Controlled 50,000-Record Gold Replay Test

A controlled replay test was conducted using 50,000 existing impression records from the Gold warehouse.

The test temporarily removed these records within a database transaction and executed the Gold loading process to restore the missing impressions.

After the first execution, the Gold impression count returned to 300,000 records. The loading process was executed a second time, and the count remained 300,000.

The result demonstrated that replaying the Gold loading operation did not create additional impression records or increase the analytical count beyond its expected value.

Test stage	Gold impression count	Result
Original warehouse	300,000	Baseline
After temporary removal of 50,000 records	250,000	Test preparation
After first Gold reload	300,000	Missing records restored
After second Gold reload	300,000	No duplicate increase

```
>>
Original impressions: 300000
Test batch size: 50000
Temporarily removed: 50000
After first replay: 300000
After second replay: 300000
Additional rows after second replay: 0
50,000-record Gold replay: PASS
Test transaction rolled back.
Original impression count preserved: True
(.venv) PS C:\Users\Ashini\Downloads\PulseTrack Capstone>
```

Figure 8: Controlled Replay of 50,000 Gold Impression Records Without Duplicate Growth

The test changes were rolled back after verification, leaving the original warehouse unchanged.

5.2.3 Test Scope and Limitation

The earlier Gold replay test confirmed idempotent fact restoration for 50,000 existing impressions. This Gold-only experiment is distinct from the subsequently completed source-level duplicate-delivery and downstream reprocessing test documented in Section 5.2.5.

On its own, the earlier Gold-only test did not simulate duplicate source delivery. A later, separate controlled test delivered the same 50,000 additional source impressions twice to an isolated database and verified all downstream layers (Section 5.2.5).

However, separate Bronze ingestion reruns also confirmed that unchanged relational source records, JSONL lines and event-log lines were not inserted again.

Taken together, the Gold replay and the additional source-delivery test establish duplicate prevention at the source-loader stage and correct propagation of an incremental historical-period batch through the implemented analytical pipeline. Different-filename deliveries and changed existing source records were not independently tested.

The implemented approach reduces the risk of inflated impression, click and conversion counts when pipelines are re-executed.

5.2.4 Automated CSV Incremental Ingestion and Idempotency Testing

The PulseTrack pipeline uses `load_source_csv.py` as the first task in its eleven-task Apache Airflow DAG to load the CSV datasets into the PostgreSQL source tables.

The loader compares staged CSV records against the existing PostgreSQL source tables using `EXCEPT ALL`. This comparison preserves duplicate occurrences while identifying source rows not yet present in the destination.

The implementation was verified using the isolated PostgreSQL database `pulsetrack_csv_test`.

Test	Verified Result
Initial CSV ingestion	774,344 records successfully imported across eleven source tables
Repeated CSV ingestion	All eleven tables reported `ALREADY MATCHES`; no additional records were inserted
Missing-record recovery	One record removed from a test source table was detected and restored
Repeated execution after recovery	Source tables matched the CSV files without further inserts
Changed-data comparison	Identical records returned FALSE; a deliberately modified temporary record returned TRUE
Main database verification	All 774,344 existing source records matched their original CSV files

The tests confirmed initial source loading, idempotent repeated execution and incremental insertion of a missing row.

The modified-record comparison was performed using a temporary SQL table and rolled back. It verified the comparison logic rather than a complete changed-source-file processing workflow.

The eleven-task Airflow pipeline was executed successfully on 18 October 2026. All tasks completed, and the final Gold-to-Mart, attribution and OLAP cache validations passed.

The loader supports append-only changes to the supplied CSV datasets. Subsequent testing on 19 October 2026 demonstrated repeated delivery of 50,000 added impressions and full manual propagation through Bronze, Silver, Gold, Mart and the semantic validations (Section 5.2.5). Automatic corrections or deletions of existing source records and fully automated backfill scheduling remain outside the verified scope.

5.2.5 Isolated Duplicate CSV Delivery and End-to-End Historical-Period Replay

To strengthen source-level idempotency evidence, a separate PostgreSQL database, pulsetrack_backfill_test, was created from the completed 938 MB pulsetrack_capstone database. Python and DataGrip confirmed that the isolated copy initially contained 300,000 impressions each in Source, Bronze and Gold. The original database and CSV files were not modified.

Eleven original CSV files were copied into an external Q5 test directory. A controlled Python test created 50,000 additional impression rows by reusing valid source-record attributes with new, unique impression_id values. The copied impressions CSV therefore contained 350,000 records. The additional records retain historical business-date values from existing impressions; this tests historical-period replay, not a newly dated real-world event.

The first run of load_source_csv.py against pulsetrack_backfill_test imported exactly 50,000 new impressions and reported 824,344 total source records across eleven CSV tables. The second run with the same source file reported ALREADY MATCHES for all eleven tables,

imported zero additional records and retained 350,000 distinct impression IDs with zero missing IDs.

```
>> python scripts/load_source_csv.py --database pulsetrack_backfill_test
>>
Connected to: pulsetrack_backfill_test

account_managers_raw: 25 rows - ALREADY MATCHES
advertisers_raw: 60 rows - ALREADY MATCHES
partners_raw: 600 rows - ALREADY MATCHES
offers_raw: 150 rows - ALREADY MATCHES
campaigns_raw: 400 rows - ALREADY MATCHES
creatives_raw: 1,200 rows - ALREADY MATCHES
impressions_raw: 350,000 rows - ALREADY MATCHES
clicks_raw: 150,000 rows - ALREADY MATCHES
conversions_raw: 126,380 rows - ALREADY MATCHES
conversion_touchpoints_raw: 183,529 rows - ALREADY MATCHES
revenue_adjustments_raw: 12,000 rows - ALREADY MATCHES

All 11 CSV sources verified: 824,344 rows.
Transaction committed successfully.
(.venv) PS C:\Users\Ashini\Downloads\PulseTrack Capstone>
```

Figure 9: Duplicate CSV Delivery Prevention — No Additional Records Inserted on Reprocessing

Manual downstream processing then executed ingest_postgres.py, process_transactions.py, load_gold.py, load_mart.py and validate_pipeline.py in sequence with PGDATABASE set to pulsetrack_backfill_test. Bronze ingested 50,000 new impressions; Silver processed 50,000 without impression-record quarantine; Gold loaded 50,000 new fact_impression records and no changed Gold records. The complete final impression reconciliation was Source = 350,000; Bronze = 350,000; Silver = 350,000; Gold = 350,000.

```
SELECT
    current_database() AS database_name,
    (SELECT COUNT(*) FROM pulsetrack_raw.impressions_raw)
        AS source_records,
    (SELECT COUNT(*) FROM bronze.impressions_raw)
        AS bronze_records,
    (SELECT COUNT(*) FROM silver.impressions)
        AS silver_records,
    (SELECT COUNT(*) FROM gold.fact_impression)
        AS gold_records,
    (SELECT COUNT(*)
     FROM pulsetrack_raw.impressions_raw
     WHERE impression_id LIKE 'Q5IMP%')
        AS new_test_records;
```

Output | Result 5

	database_name	source_records	bronze_records	silver_records	gold_records	new_test_records
1	pulsetrack_backfill_test	350000	350000	350000	350000	50000

Figure 10: Source-to-Bronze-to-Silver-to-Gold Reconciliation After 50,000-Record Duplicate-Delivery Test

The Data Mart refreshed 687,318 output rows. The final validation reported PASS for all 14 Gold-to-Mart KPI comparisons, attribution reconciliation and OLAP-cache consistency. The mart advertiser-impression reconciliation reported 339,534 eligible impressions, which is a business-rule-filtered measure, rather than an assertion that all 350,000 warehouse impressions are eligible.

```
>> python scripts/validate_pipeline.py
>> env) PS C:\Users\Ashini\Downloads\PulseTrack Capstone>
PASS: Advertiser clicks (expected=145604, actual=145604)
PASS: Advertiser impressions (expected=339534, actual=339534)
PASS: Campaign all conversions (expected=87277, actual=87277)
PASS: Campaign clicks (expected=145604, actual=145604)
PASS: Campaign eligible adjustment USD (expected=3280.4600, actual=3280.4600)
PASS: Campaign eligible base revenue USD (expected=466632.7300, actual=466632.7300)
PASS: Campaign eligible conversions (expected=3640, actual=3640)
PASS: Campaign impressions (expected=339534, actual=339534)
PASS: Creative impressions (expected=323113, actual=323113)
PASS: Dated orphan adjustments (expected=702, actual=702)
PASS: Dated revenue adjustments (expected=11628, actual=11628)
PASS: Historical manager conversions (expected=69426, actual=69426)
PASS: Partner-group clicks (expected=145604, actual=145604)
PASS: Posted eligible adjustment USD (expected=3239.9400, actual=3239.9400)
Gold-to-Mart reconciliation: PASS
Attribution reconciliation: PASS
Creative attributed revenue: 445363.2154
OLAP cache validation: PASS

All pipeline validations passed.
(.venv) PS C:\Users\Ashini\Downloads\PulseTrack Capstone>
```

Figure 11: Successful Gold-to-Mart, Attribution and OLAP Validation After Incremental Processing

Conclusion and scope: The experiment verifies that an unchanged 50,000-record CSV delivery can be repeated without source duplication and that the added historical-period impression records flow through the implemented downstream layers with consistent analytical validation. It was an isolated, manually executed test; it does not establish automatic Airflow backfill scheduling, changed-record correction, or every form of late-arriving adjustment ingestion. The 50,000 generated records are synthetic test data and are not part of the production/source baseline.

5.3 Late-Arriving Revenue Adjustments

Revenue adjustments may be recorded after the original conversion because of refunds, commissions, corrections or other financial changes. The warehouse must apply these adjustments to the correct conversion without creating a duplicate conversion record.

5.3.1 Adjustment Processing Strategy

PulseTrack stores conversions and revenue adjustments in separate Gold fact tables.

The `gold.fact_conversion` table maintains one record per distinct business conversion, while `gold.fact_revenue_adjustment` preserves individual financial changes.

An adjustment is associated with its original conversion using the conversion identifier. Eligible adjustments are incorporated into adjusted-revenue calculations without altering the number of conversions.

The implementation also preserves the original conversion timestamp and the adjustment timestamp separately. This allows financial reporting to distinguish when revenue was generated from when a subsequent adjustment was recorded.

The Data Mart is refreshed after Gold processing so that changes to eligible adjustments are reflected in the analytical results.

5.3.2 Controlled Late-Adjustment Test

A controlled test was performed using an existing conversion and a corresponding revenue adjustment.

The test introduced an additional adjustment amount of $43.52 into the calculation while preserving the original conversion record.

The results were:

Measure	Before adjustment	After adjustment
Adjusted revenue	$56.51	$100.03
Revenue change	---	+$43.52
Conversion count	1	1

```
>>
Adjustment tested: ADJ0000045
Conversion tested: CV0055177
Adjustment amount: 43.5200
Revenue before replay: 56.5100
Revenue after replay: 100.0300
Conversion rows before: 1
Conversion rows after: 1
Late adjustment replay: PASS
Test transaction rolled back.
Original adjustment preserved: True
(.venv) PS C:\Users\Ashini\Downloads\PulseTrack Capstone>
```

Figure 12: Late Revenue Adjustment Test Showing Updated Revenue Without Conversion Duplication

The test confirmed that the adjusted revenue increased by the expected amount while the business conversion remained counted once.

This demonstrates that the warehouse can incorporate a subsequent financial correction without duplicating the underlying conversion.

5.3.3 Historical Accuracy and Limitations

Keeping revenue adjustments separate from conversions preserves the original business event and allows its financial value to change over time.

The implementation also supports analysis based on the adjustment posting date, which is important when corrections affect financial results in later reporting periods.

The controlled test verified the revenue calculation and unchanged conversion count. It did not independently demonstrate a new adjustment file arriving several days later through the complete upstream ingestion pipeline.

The distinction between a delayed business adjustment and delayed ingestion is important when interpreting the test evidence.

Overall, the implemented model supports financial corrections while preserving conversion uniqueness and historical traceability.

5.4 Historical Corrections and SCD Type 2

Historical corrections are important because an advertiser's account-manager assignment may change or be reported after the relevant business events have already occurred.

PulseTrack must preserve the manager responsible at the time of each conversion rather than assigning all historical conversions to the advertiser's latest manager.

5.4.1 Historical Processing Strategy

The Gold warehouse implements Slowly Changing Dimension Type 2 (SCD Type 2) in `gold.dim_advertiser`.

Each advertiser history version has a separate surrogate key and effective period. When an account-manager assignment changes, the previous version is closed and a new version is created.

Conversions are associated with the advertiser version valid at their event timestamp. This allows earlier and later conversions to be reported under their corresponding historical assignments.

Historical information is obtained from the advertiser-account profiles and related source records. Where reliable earlier assignments are unavailable, the warehouse retains an unknown historical manager instead of inventing a value.

5.4.2 Controlled Historical Correction Test

A controlled database transaction was used to test a historical account-manager correction for advertiser `ADV0001`.

The original account manager was `AM025`. The test introduced a corrected assignment to `AM001`, effective from 28 January 2025.

The Gold loading logic was then evaluated to determine whether the historical advertiser versions and related conversion assignments were updated correctly.

The test produced the following results:

Test measure	Observed result
Advertiser	`ADV0001`
Original account manager	`AM025`
Corrected account manager	`AM001`
Change effective date	28 January 2025
Earlier conversions preserved	6
Later conversions reassigned	1,522
Advertiser dimension versions	2 → 3
Historical correction result	PASS

```
>>
Advertiser tested: ADV0001
Original manager: AM025
New test manager: AM001
Change effective date: 2025-01-28
Earlier conversions preserved: 6
Later conversions reassigned: 1522
Advertiser versions: 2 -> 3
Historical SCD Type 2 correction: PASS
Test transaction rolled back.
Original history preserved: True
Original dimension versions preserved: True
(.venv) PS C:\Users\Ashini\Downloads\PulseTrack Capstone>
```

Figure 13: Controlled SCD Type 2 Historical Account-Manager Correction and Conversion Reassignment

The six earlier conversions retained their historical assignment, while 1,522 later conversions were reassigned to the corrected advertiser history version.

The advertiser dimension increased from two to three versions during the test, demonstrating that the correction was represented using historical versions rather than overwriting the existing assignment.

5.4.3 Transaction Rollback and Verification

After the test, the database transaction was rolled back to avoid permanently changing the original warehouse data.

The verification confirmed that the original account-manager history and the original number of advertiser dimension versions were preserved.

This demonstrates that the historical correction can be tested without permanently modifying the production-like warehouse state.

The test specifically validates SCD Type 2 correction behaviour and historical conversion reassignment in Gold. It does not independently demonstrate receipt of a new historical correction through the complete upstream ingestion pipeline.

Overall, the results demonstrate that the implemented historical processing logic can preserve earlier business context while applying corrections to the appropriate later conversions.

5.5 Pipeline Failure and Recovery

Pipeline failures may occur because of database connection problems, invalid source records, transformation errors or interruptions during warehouse loading.

PulseTrack uses PostgreSQL transactions, processing audit records and Apache Airflow task monitoring to support failure detection and controlled recovery.

5.5.1 Recovery Strategy

The implemented pipeline separates processing into distinct stages, including source ingestion, Silver transformations, Gold dimension loading, Gold fact loading and analytical refreshes.

PostgreSQL transactions protect database changes within the relevant processing stages. If an operation fails before its transaction is committed, its incomplete changes can be rolled back.

Processing audit records retain information about successful and failed stages, allowing the affected operation to be identified.

Airflow also monitors task execution and prevents dependent tasks from continuing when an upstream task fails. The DAG is configured with one retry and a two-minute retry delay.

Because the loading operations are designed to support repeated execution, failed stages can be rerun without intentionally duplicating existing business records.

5.5.2 Failure Between Dimension and Fact Loading

A controlled failure was tested after Gold dimension processing but before successful completion of fact loading.

The failure was recorded in the pipeline audit information. The affected processing stage was then rerun, allowing fact loading to complete successfully.

This test demonstrated that a failure between warehouse-loading stages could be identified and recovered from without rebuilding the entire warehouse.

The result also highlights an important design consideration: independently committed processing stages do not automatically form one transaction across the complete pipeline. Recovery therefore depends on identifying the failed stage and safely rerunning the necessary operations.

5.5.3 Airflow Failure Detection Test

An additional controlled test was performed using the Airflow `validate_pipeline` task.

An intentionally incorrect PostgreSQL password was supplied for the test execution to simulate an authentication failure.

The connection attempt failed with a PostgreSQL authentication error, and the Python process returned exit code 1.

Airflow recognised the unsuccessful execution and recorded the task failure in its logs.

This confirmed that a database connection error was not silently treated as successful pipeline execution.

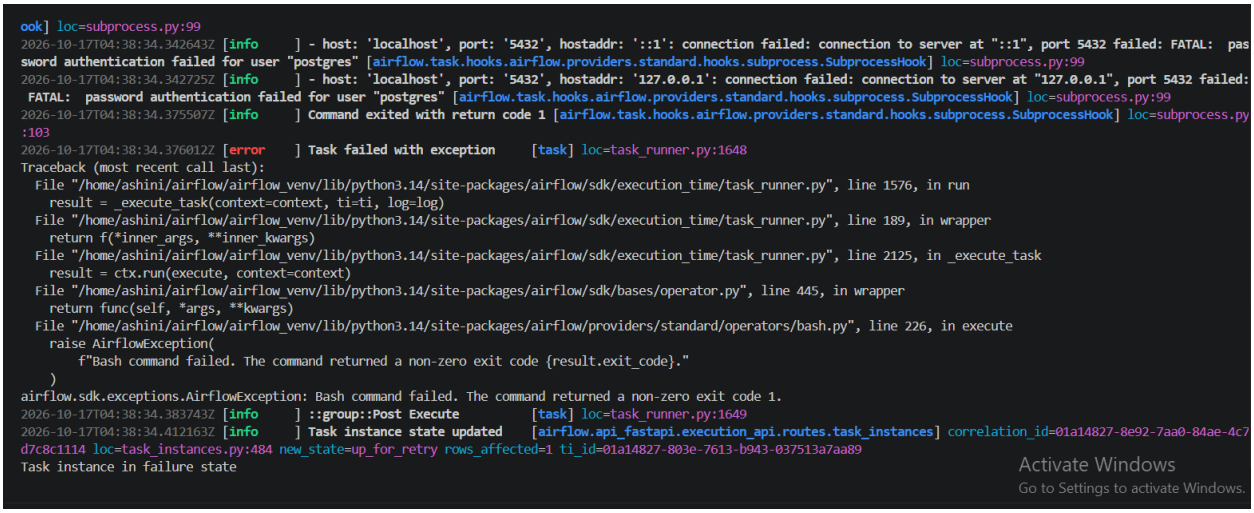


Figure 14: Airflow Detection of a Controlled PostgreSQL Authentication Failure

5.5.4 Successful Recovery and Revalidation

After the controlled authentication failure, the invalid password configuration was removed and the validation task was executed again.

The task successfully connected to PostgreSQL and completed its reconciliation checks.

The recovery execution confirmed:

Validation	Recovery result
Gold-to-Mart reconciliation	14 checks PASS
Attribution reconciliation	PASS
OLAP materialized-cache validation	PASS
Python process exit code	0
Airflow task execution	SUCCESS

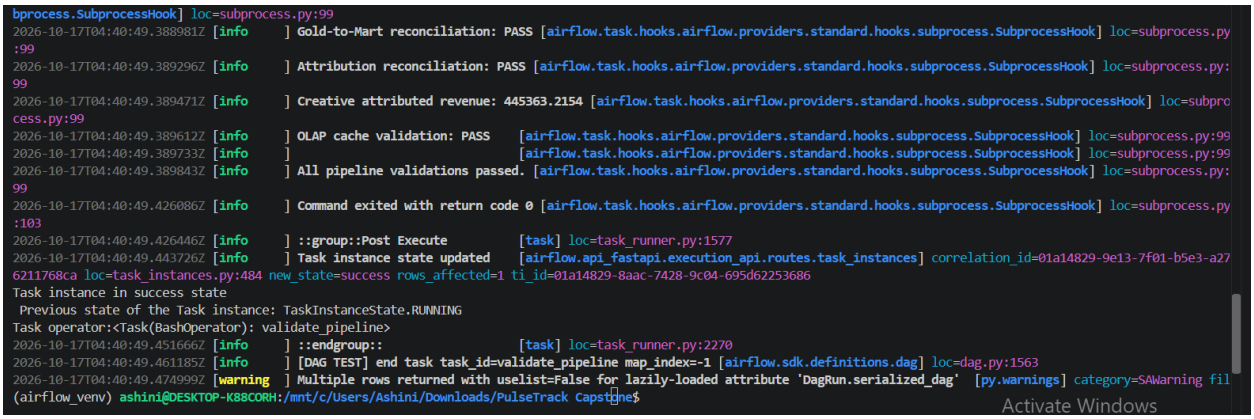


Figure 15: Successful Airflow Validation Following Recovery from the Controlled Failure

These results demonstrated that the validation process could be restored after correcting the connection problem.

The authentication-failure test and successful recovery were executed separately using Airflow's task-testing command. Therefore, this evidence demonstrates failure detection and manual recovery, not an automatically completed retry sequence.

5.5.5 Recovery Outcome

The tests demonstrated three important behaviours: database-stage failures can be recorded and investigated, failed processing stages can be rerun, and Airflow recognises unsuccessful task execution through its exit status.

The successful recovery validation also confirmed that the analytical reconciliation checks continued to pass after the simulated connection problem was resolved.

For a production deployment, recovery would additionally require operational alerts, controlled retry policies, consistent batch identifiers and procedures for handling partially completed multi-stage pipeline runs.

5.6 Processing Limitations and Summary

The PulseTrack pipeline was tested for automated CSV ingestion, repeated source loading, a controlled 50,000-record duplicate CSV delivery, manual end-to-end historical-period reprocessing, Gold fact replay, revenue corrections, historical account-manager changes and failure recovery. The original eleven-task Airflow workflow also completed successfully; the additional Q5 test used manual processing of the isolated database, not another DAG run.

The results demonstrated that repeated Gold loading does not create duplicate impression records, revenue adjustments can change analytical values without increasing conversion counts, and SCD Type 2 processing can preserve historical account-manager assignments.

The recovery tests also confirmed that processing failures can be detected and the affected stages can be successfully rerun.

However, several limitations remain:

- **Duplicate delivery and incremental ingestion: An isolated database test loaded 50,000 additional source impressions on the first delivery and zero on the repeated delivery, then reconciled 350,000 impression records across Source, Bronze, Silver and Gold. The same content arriving under different filenames and arbitrary corrections to already-loaded CSV rows were not separately tested.**
- **Late-arriving data:** Revenue correction tests verified analytical updates but did not simulate a new source file arriving several days after the original ingestion.
- **Historical backfills: Manual source-to-report replay of an additional historical-period impression batch was demonstrated, including transactional Mart refresh and downstream validations. Fully automated source-file backfill scheduling, late-arriving corrections to existing records and an independently timed delayed-revenue-file scenario were not demonstrated.**
- **Data Mart loading:** The Data Mart uses transactional full refresh rather than incremental row-level loading.
- **Automatic recovery:** Airflow retry settings were configured, but the controlled recovery test demonstrated manual re-execution rather than an automatically completed retry.

These limitations identify areas for further development without reducing the value of the behaviours already verified.

Overall, the implementation provides a working foundation for reliable reprocessing, historical preservation and recovery. Together with the data-quality checks and audit records, these mechanisms help maintain consistent analytical results across repeated pipeline executions.

Question 6 – Data Quality, Reconciliation and Audit

6.1 Data Quality Framework

Data quality is important in PulseTrack because the supplied datasets contain duplicate conversions, malformed event records, inconsistent identifiers, missing relationships and invalid or ambiguous business values.

The implemented pipeline applies validation at different processing stages rather than relying on a single check at the end.

In the Bronze layer, original source records are preserved with ingestion metadata. This allows the pipeline to compare loaded records against the supplied data and investigate processing errors.

In the Silver layer, records are cleaned and standardised. The processing includes date conversion, status normalisation, duplicate detection, partner identifier matching and malformed-record identification.

The Gold layer applies dimensional relationships and business rules before records contribute to analytical measures. This includes preserving distinct conversion counts, matching historical advertiser versions, applying revenue eligibility rules and handling unresolved references.

The Data Mart and semantic layers are then checked using reconciliation queries to confirm that aggregated measures remain consistent with the warehouse.

6.1.1 Main Data Quality Checks

Validation category	Implemented approach
Source completeness	Compare available source records with Bronze ingestion counts.
Duplicate detection	Identify repeated conversion IDs and duplicate event deliveries.
Malformed records	Preserve invalid event-log records in a separate error table.
Date validation	Standardise supported date formats and retain missing or unresolved dates for investigation.
Monetary validation	Apply defined revenue eligibility and currency rules.
Status validation	Normalise recognised conversion statuses without automatically accepting ambiguous values.
Referential integrity	Identify unresolved advertiser, campaign, creative, partner and conversion relationships.
Historical validation	Match facts to the applicable SCD Type 2 advertiser version.
Attribution validation	Check weighted conversion credit and attributed revenue.
Analytical reconciliation	Compare Gold warehouse measures with Data Mart outputs and the OLAP cache.

These checks help prevent incorrect business results caused by duplicate records, invalid references or inconsistent calculations.

Records that cannot be safely corrected are preserved where appropriate rather than silently assigned unsupported values.

The following sections present the actual reconciliation results and explain how processing outcomes are recorded for investigation and recovery.

6.2 Source-to-Bronze-to-Silver-to-Gold Reconciliation

Reconciliation was performed to verify that source records were preserved during ingestion and that the transformations produced the expected warehouse results.

The verification compared record counts across four layers: the original PostgreSQL source tables, Bronze, Silver and Gold.

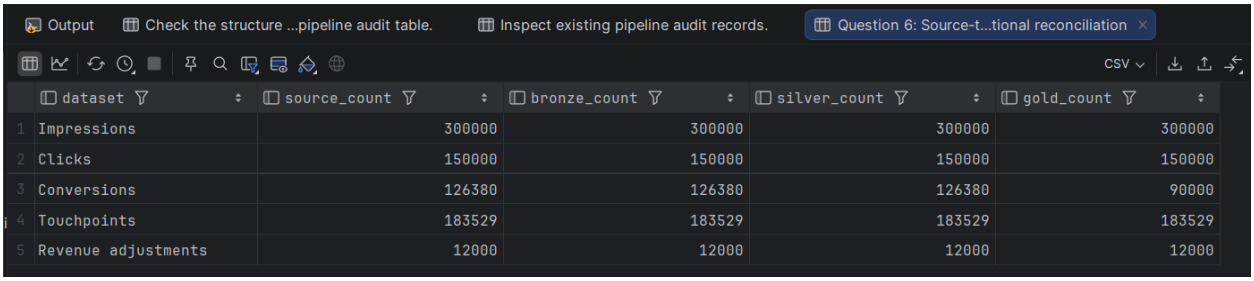
The pipeline validates the eleven relational CSV sources before PostgreSQL-to-Bronze ingestion. CSV-header validation and source-record comparisons help ensure that the PostgreSQL source tables remain consistent with the supplied files.

Automated CSV loading, idempotency and missing-record recovery tests are documented in Sections 4.2.1 and 5.2.4. The reconciliation results below focus on record preservation and expected differences in data grain across the Bronze, Silver and Gold layers.

6.2.1 Transactional Data Reconciliation

The following results were obtained by executing SQL reconciliation queries against the implemented database.

Dataset	Source	Bronze	Silver	Gold
Impressions	300,000	300,000	300,000	300,000
Clicks	150,000	150,000	150,000	150,000
Conversions	126,380	126,380	126,380	90,000
Touchpoints	183,529	183,529	183,529	183,529
Revenue adjustments	12,000	12,000	12,000	12,000



	dataset	source_count	bronze_count	silver_count	gold_count
1	Impressions	300000	300000	300000	300000
2	Clicks	150000	150000	150000	150000
3	Conversions	126380	126380	126380	90000
4	Touchpoints	183529	183529	183529	183529
5	Revenue adjustments	12000	12000	12000	12000

Figure 16: Source-to-Bronze-to-Silver-to-Gold Transactional Record Reconciliation

All five datasets produced their expected counts.

Impressions, clicks, conversion touchpoints and revenue adjustments retained the same number of records throughout the four processing layers.

The conversion count decreased from 126,380 records in Silver to 90,000 in Gold. This difference is expected because the source contains 36,380 additional conversion-history records associated with repeated conversion identifiers.

Silver preserves these records for historical analysis, while Gold maintains one record per distinct business conversion to prevent double counting.

The reconciliation therefore demonstrates that the conversion-count difference results from the intended warehouse grain rather than unexplained record loss.

6.2.2 Event-Log Reconciliation

The conversion event-log export was reconciled separately because it contains event messages rather than the same business-record grain as the Gold conversion fact.

The processing results were:

Event-log classification	Records
Original source lines	183,162
Valid unique events	178,600
Duplicate deliveries	4,378
Malformed records	184
Total accounted for	**183,162**

All original event-log lines were accounted for.

Valid unique events were stored in `silver.conversion_events`, and malformed records were retained in `silver.event_log_errors`.

Duplicate event deliveries were identified without treating them as additional unique events.

This reconciliation confirms that the event-processing classifications account for the entire supplied event log.

6.2.3 Reference Data and JSONL Reconciliation

The source inventory also contains six relational reference tables, two JSONL collections and their associated historical or profile information.

The eleven relational source tables contained 774,344 records in total, and the Bronze ingestion checks confirmed matching source and Bronze counts.

The two JSONL collections contained 660 documents: 60 advertiser-account records and 600 partner-profile records. All 660 original documents were retained in Bronze.

These records were subsequently processed to support reference dimensions, historical advertiser assignments and partner-profile matching.

A direct one-to-one count comparison with Gold is not appropriate for every reference dataset. For example, the Gold advertiser dimension contains multiple historical versions for the same source advertiser because SCD Type 2 is used.

6.2.4 Reconciliation Outcome

The Source-to-Gold record-count checks passed for all five transactional datasets.

The event-log reconciliation also accounted for every source line through valid unique events, duplicate deliveries and malformed records.

These checks demonstrate record-count completeness and explain the expected changes caused by deduplication and historical modelling.

However, matching counts alone do not guarantee that every field value, relationship or business measure is correct. Additional data-quality, referential-integrity and financial reconciliation checks are required.

These validations are discussed in the following sections.

6.3 Duplicate Detection, Malformed Records and Referential Integrity

The PulseTrack source data contains several quality issues that must be handled before reliable business reporting is possible.

SQL validation checks were performed against the source, Silver and Gold layers to verify duplicate detection, malformed-record handling and selected relationship issues.

6.3.1 Duplicate Detection and Validation Results

The following results were obtained from the implemented database.

Quality Check	Records	Finding
Additional source conversion rows	36,380	Repeated conversion identifiers are present in the source history.
Duplicate conversion IDs in Gold	0	Each business conversion is represented once.
Duplicate event IDs in Silver	0	Valid processed events have unique identifiers.
Malformed event records	184	Invalid event records are retained for investigation.
Clicks with unresolved partner references	114,166	Clicks are preserved without unsupported partner assignments.

The source contained 126,380 conversion-history records but only 90,000 distinct conversion identifiers. The 36,380 additional rows were preserved in Silver rather than being counted as separate business conversions.

The Gold conversion fact contained zero duplicate conversion identifiers. This confirmed that the implemented warehouse maintained the intended grain of one row per business conversion.

The Silver event table also contained zero duplicate event identifiers, demonstrating that repeated event deliveries were excluded from the set of unique processed events.

6.3.2 Malformed-Record Handling

The conversion event-log source contained 184 malformed JSON records.

These records were identified during Silver processing and preserved in `silver.event_log_errors` for investigation.

The original event-log lines also remained available in Bronze, allowing invalid records to be traced back to their source file and line number.

Malformed records were not treated as valid conversion events. This prevented parsing errors from introducing incorrect event information into downstream processing.

6.3.3 Missing and Unresolved Relationships

Relationship validation is particularly important because marketing events reference advertisers, campaigns, creatives, partners and conversions from different source systems.

The verification identified 114,166 Gold click records without a resolved partner key.

These clicks were retained because an unresolved partner identifier does not necessarily make the click itself invalid.

However, the implementation does not assign such clicks to an arbitrary partner. This prevents unsupported partner attribution and misleading partner-level revenue reports.

Other relationship issues identified during source profiling include missing click references, unmatched partner profiles and revenue adjustments referencing conversions absent from the supplied conversion data.

The pipeline preserves records requiring investigation where appropriate, while avoiding unsupported relationships.

The unresolved partner count also indicates a limitation in the available source identity mapping. Partner attribution results must therefore be interpreted with this limitation in mind.

6.3.4 Data Quality Outcome

The duplicate-detection checks confirmed that Gold conversions and Silver unique events did not contain repeated business identifiers.

Malformed event records were separately preserved, and unresolved partner relationships remained identifiable without discarding otherwise valid clicks.

These results demonstrate important data-quality controls, particularly for deduplication, malformed-record handling and cautious relationship matching.

However, the five checks do not independently prove that every foreign-key relationship is valid. Additional schema, relationship, monetary, status and attribution checks are needed for comprehensive validation.

6.3.5 PostgreSQL Foreign-Key Constraints

The Gold warehouse defines 25 foreign-key relationships across its dimension tables, fact tables and conversion-touchpoint bridge.

These relationships connect marketing events to their corresponding dimensions and maintain links between conversions, clicks, touchpoints and revenue adjustments.

The implementation includes foreign keys from the impression, click and conversion facts to their relevant dimensions. The conversion-touchpoint bridge references both the click and conversion facts, while revenue adjustments reference their corresponding conversions where a valid match is available.

Foreign-key constraints provide an additional layer of protection against invalid references. However, relationships that cannot be resolved from the available source data may remain NULL where the schema permits this.

For example, clicks with unresolved partner references are retained without assigning an unsupported partner. Similarly, adjustments without a matching conversion are preserved for reconciliation.

Therefore, PostgreSQL constraints are combined with transformation rules and data-quality checks to support both referential integrity and preservation of incomplete source information.

6.3.6 Foreign-Key Validation Results and Unresolved References

PostgreSQL system-catalog queries were used to verify the foreign-key constraints implemented in the Gold warehouse.

The results confirmed that all 25 declared foreign-key constraints were validated, with zero unvalidated constraints.

This provides database-level protection against invalid non-NULL references in the implemented dimensional model.

However, validated foreign keys do not mean that every source relationship has been resolved. Some records contain NULL dimension or fact references because the corresponding relationship could not be established reliably.

The following unresolved references were identified:

Relationship check	Affected records
Impressions without a resolved creative	14,472
Clicks without a resolved partner	114,166
Conversions without a resolved advertiser	2,723
Conversions without a resolved campaign	0
Revenue adjustments without a matched conversion	721

The 14,472 impressions without resolved creative references and 114,166 clicks without resolved partner references were retained rather than linked to unsupported dimension members.

Similarly, 2,723 conversions had no resolved advertiser reference, while all 90,000 Gold conversions had a resolved campaign reference.

The 721 revenue adjustments without matching conversions were also preserved for reconciliation rather than assigned to unrelated conversion records.

These results demonstrate that the warehouse maintains validated foreign-key constraints while retaining information about unresolved source relationships.

The remaining gaps are important data-quality limitations, particularly for advertiser-level analysis, creative performance and partner attribution.

Overall, the implementation prioritises traceability and reliable relationships over artificially complete dimensional references.

6.4 Business-Rule, Attribution and Analytical Reconciliation

In addition to checking record counts and database relationships, PulseTrack validates the business rules used to calculate marketing performance and revenue.

These checks are important because a technically valid record should not automatically contribute to every business KPI. Conversion status, test traffic, currency, missing dates and unresolved attribution relationships must also be considered.

6.4.1 Conversion and Revenue Validation

The implemented warehouse maintains one record per distinct business conversion while preserving repeated conversion history in Silver.

For financial KPIs, the pipeline applies a defined revenue-eligibility rule. An eligible conversion must have an approved status, represent non-test activity, use USD and contain a recognised non-negative base revenue amount.

Conversions that do not satisfy these conditions are retained in the warehouse where appropriate but are excluded from eligible-revenue measures.

Revenue adjustments are processed separately from conversions. Eligible adjustments are matched to their original conversions, allowing adjusted revenue to be calculated without increasing the number of business conversions.

The reporting logic also distinguishes the original conversion date from the adjustment posting date. This prevents financial changes recorded in later periods from being incorrectly interpreted as new conversions.

6.4.2 Attribution Validation

PulseTrack supports multi-touch attribution through `gold.bridge_conversion_touchpoint`.

A conversion may have multiple contributing touchpoints, so the warehouse must distribute conversion credit without counting the original conversion or its revenue multiple times.

The implementation uses attribution weights to calculate weighted conversion credit and attributed revenue. Small rounding differences in the supplied weights are normalised where appropriate.

The validation process checks whether aggregated attribution results agree with the corresponding eligible conversion and revenue measures within defined numerical tolerances.

However, the source contains a substantial number of unresolved partner references. These records are preserved without assigning attribution to an unsupported known partner.

Therefore, attribution reconciliation can pass while partner identification remains incomplete. Both findings must be considered when interpreting partner-level performance.

6.4.3 Gold-to-Mart Reconciliation

The Data Mart is populated from the Gold warehouse and contains aggregated performance measures for campaigns, advertisers, creatives, partners, account managers and revenue adjustments.

To verify these transformations, SQL reconciliation checks compare the Data Mart results with the corresponding Gold warehouse calculations.

The automated validation script executes fourteen reconciliation checks.

The completed pipeline execution returned the following results:

Validation category	Result
Gold-to-Mart reconciliation	14 of 14 checks PASS
Weighted attribution reconciliation	PASS
Attributed revenue reconciliation	PASS
OLAP materialized-cache consistency	PASS

For the date-eligible campaign reporting population, the reconciled measures included:

Measure	Verified value
Impressions	291,032
Clicks	145,604
All conversions	87,277
Revenue-eligible conversions	3,640
Eligible base revenue	$466,632.73
Matched eligible revenue adjustments	$3,280.46
Adjusted revenue	$469,913.19

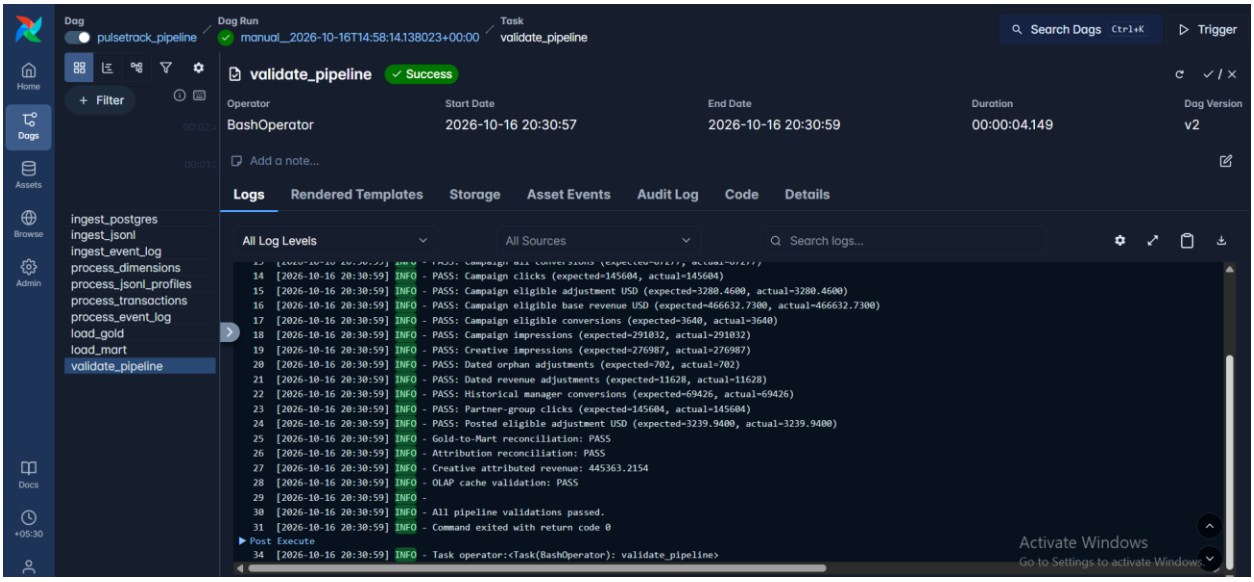


Figure 17: Automated Gold-to-Mart and Attribution Reconciliation Results

These figures relate to the date-eligible campaign reporting population, not every source record.

The difference between the total Gold fact counts and these analytical counts is therefore intentional. Records without a usable reporting date or without the required business eligibility are not automatically included in the corresponding analytical measure.

The reconciliation confirms that the Data Mart reproduces the relevant Gold calculations under the same reporting and eligibility rules.

6.4.4 OLAP Cache Consistency

The semantic analytical layer includes a monthly campaign-performance materialized cache designed to improve repeated analytical queries.

After the Data Mart is refreshed, the monthly cache is refreshed from the semantic model.

An automated comparison checks whether the cached records match the corresponding monthly semantic-view results.

The completed validation returned zero differences, confirming that the materialized cache was consistent with its source view at the time of testing.

This check is important because improved query performance must not come at the cost of inconsistent analytical results.

6.4.5 Validation Outcome

The completed checks demonstrated consistency between the Gold warehouse, Data Mart and monthly OLAP cache.

All fourteen Gold-to-Mart reconciliation checks passed, alongside attribution and cache-consistency validation.

These results provide evidence that the implemented analytical calculations are internally consistent under the defined business rules.

However, successful reconciliation does not establish that all source data is complete or that unresolved partner references have been corrected. These remain documented data-quality limitations.

The validation outcomes are combined with pipeline execution audit records to support investigation and recovery when processing problems occur.

6.5 Pipeline Audit and Error Tracking

The PulseTrack platform includes an audit mechanism to record warehouse-loading activity and support operational monitoring, troubleshooting and recovery.

Audit information is stored in the PostgreSQL table `audit.pipeline_runs`.

6.5.1 Audit Table Structure

The audit table contains fourteen columns for identifying pipeline executions, recording their outcomes and storing processing statistics.

Audit field	Purpose
`run_id`	Unique identifier for an audit record.
`batch_id`	Groups related processing stages within a pipeline execution.
`pipeline_name`	Identifies the pipeline being executed.
`stage_name`	Identifies the individual processing stage.
`started_at`	Records when processing began.
`finished_at`	Records when processing ended.
`status`	Records the execution outcome, such as SUCCESS or FAILED.
`source_count`	Stores the source record count when captured by the stage.
`processed_count`	Stores the processed record count when captured.
`inserted_count`	Stores the number of inserted records when available.
`updated_count`	Stores the number of updated records when available.
`rejected_count`	Stores the number of rejected records when available.
`error_count`	Stores the recorded number of processing errors.
`error_message`	Preserves an error description when a failure occurs.

The audit design supports execution traceability by recording which processing stage ran, when it started and finished, and whether it completed successfully.

It also provides fields for processing statistics, although the completeness and interpretation of these counters depend on the individual pipeline stage.

The audit records complement Apache Airflow task logs. Airflow provides orchestration-level execution information, while the PostgreSQL audit table preserves processing-stage outcomes within the data platform.

6.5.2 Audit Implementation and Verification Results

The PostgreSQL audit mechanism records processing outcomes across Bronze ingestion, Silver processing, Gold warehouse loading and Data Mart refresh operations.

A reusable Python helper, `pipeline_audit.py`, was implemented to record execution details and processing counts in the existing PostgreSQL `audit.pipeline_runs` table.

The audit mechanism was tested by rerunning the individual processing stages and examining their latest records.

Apache Airflow orchestrates eleven pipeline tasks, while the PostgreSQL `audit.pipeline_runs` table records ten internal processing stages across Bronze, Silver, Gold and Data Mart operations.

The counts differ because the Gold loading task produces separate dimension and fact audit records, while CSV loading and final pipeline validation are monitored through Airflow execution logs.

Together, these mechanisms provide workflow-level monitoring and detailed processing-stage audit information.

Processing Layer	Audited Stages	Latest Result
Bronze	3	All SUCCESS
Silver	4	All SUCCESS
Gold	2	All SUCCESS
Data Mart	1	SUCCESS
Total	**10**	**10 SUCCESS**

Output | pulsetrack_capstone.audit.pipeline_runs 3 | Question 6.5: Latest...r each pipeline stage | pulsetrack_capstone.audit.pipeline_runs 2

	run_id	pipeline_name	stage_name	status	source_count	processed_cou…	inserted_count	updated_count
1	22	bronze_ingestion	ingest_postgres	SUCCESS	774344	774344	0	
2	23	bronze_ingestion	ingest_jsonl	SUCCESS	660	660	0	
3	24	bronze_ingestion	ingest_event_log	SUCCESS	183162	183162	0	
4	25	silver_processing	process_dimensions	SUCCESS	2435	2435	<null>	
5	26	silver_processing	process_jsonl_profi…	SUCCESS	660	660	<null>	
6	27	silver_processing	process_transactions	SUCCESS	771909	771909	<null>	
7	28	silver_processing	process_event_log	SUCCESS	183162	183162	0	
8	19	load_gold	load_gold_dimensions	SUCCESS	<null>	<null>	0	
9	20	load_gold	load_gold_facts	SUCCESS	<null>	<null>	0	
10	21	load_mart	refresh_marts	SUCCESS	<null>	687318	<null>	

Output | pulsetrack_capstone.audit.pipeline_runs 3 | Question 6.5: Latest...r each pipeline stage | pulsetrack_capstone.audit.pipeline_runs 2

		status	source_count	processed_cou…	inserted_count	updated_count	rejected_count	error_count
1	es	SUCCESS	774344	774344	0	0	0	0
2		SUCCESS	660	660	0	0	0	0
3	log	SUCCESS	183162	183162	0	0	0	0
4	sions	SUCCESS	2435	2435	<null>	<null>	0	0
5	_profi…	SUCCESS	660	660	<null>	<null>	0	0
6	actions	SUCCESS	771909	771909	<null>	<null>	0	0
7	_log	SUCCESS	183162	183162	0	<null>	184	0
8	ensions	SUCCESS	<null>	<null>	0	0	<null>	0
9	ts	SUCCESS	<null>	<null>	0	0	<null>	0
10		SUCCESS	<null>	687318	<null>	<null>	<null>	0

Figure 18: Consolidated Pipeline Audit Results Across Bronze, Silver, Gold and Data Mart

The Bronze reruns processed 774,344 relational records, 660 JSONL records and 183,162 event-log lines. All three recorded zero new inserts, confirming that the unchanged source data did not create duplicate Bronze records.

The Silver reference-data stage processed 2,435 records, while JSONL profile processing examined 660 records. Silver transactional processing examined 771,909 records without quarantining additional records.

The Silver event-log stage examined 183,162 lines, identified 184 malformed records and skipped 182,978 valid event deliveries already present in Silver. The skipped count includes previously loaded events and should not be interpreted as the number of duplicates in the original source.

The improved Gold audit recorded zero inserted and zero changed records during the verified rerun of the dimension and fact loading stages. Separate controlled tests confirmed that the audit comparison logic could identify both newly appearing records and meaningful updates.

The Data Mart refresh successfully processed 687,318 output rows across six mart tables.

All ten latest stage records reported successful execution with zero pipeline execution errors.

6.5.3 Audit Counters and Interpretation

The audit mechanism records source and processed counts for Bronze and Silver processing stages.

Bronze ingestion also records newly inserted rows. Silver event-log processing records newly inserted valid events and malformed records encountered during each execution.

Gold processing uses before-and-after row comparisons to identify inserted records and existing records whose stored values have changed.

The Data Mart audit records the combined output row count following a successful full refresh.

Not every field is populated for every stage. In particular, the Silver UPSERT functions do not separately measure inserts and updates, while Gold processing does not currently record complete stage-level source and processed counts.

These unavailable metrics remain NULL rather than being assigned misleading values.

The event-log rejection count also requires careful interpretation: the 184 malformed records were encountered during the verified rerun, but they were not 184 newly discovered errors.

The latest stage records were produced by separate verification executions, rather than a single coordinated pipeline run sharing one batch identifier.

6.5.4 Failure Tracking and Recovery Evidence

The audit table also preserves earlier failed executions.

The recorded failures included a division-by-zero error during Gold fact loading, a closed database connection during Data Mart refresh and an SQL syntax error during another Data Mart refresh.

The audit records retained the failed stage, execution timestamps, status, error count and error message.

Subsequent successful executions demonstrated that the affected stages could be corrected and rerun.

Together with Apache Airflow task logs, these records provide operational evidence for identifying processing failures and verifying recovery.

The audit implementation therefore supports execution monitoring, error investigation and processing-count verification, while retaining documented limitations in stage-level metric coverage.

6.6 Data Quality, Reconciliation and Audit Summary

The PulseTrack data-quality framework was verified using source reconciliation, duplicate checks, referential-integrity validation, business-rule checks and processing audit records.

Source-to-Bronze record counts matched across the supplied datasets. Transactional reconciliation confirmed the expected Gold fact counts, including the reduction from 126,380 conversion-history records to 90,000 distinct business conversions.

The Silver event-log reconciliation accounted for 178,600 unique valid events, 4,378 duplicate deliveries and 184 malformed records.

All 25 declared Gold foreign-key constraints were validated. However, unresolved business relationships remain, including missing partner references and unmatched revenue adjustments.

The analytical validation script successfully completed fourteen Gold-to-Mart reconciliation checks, attribution reconciliation and OLAP materialized-cache consistency validation.

PostgreSQL audit records were verified for ten internal processing stages across Bronze, Silver, Gold and Data Mart operations. The latest checked execution for each stage completed successfully.

Some limitations remain, particularly incomplete insert/update breakdowns for Silver processing, unresolved source relationships and the absence of a single shared batch identifier across every independently executed stage.

Overall, the implementation provides verified record-count reconciliation, data-quality controls, analytical consistency checks and operational audit evidence. These mechanisms improve the reliability, traceability and maintainability of the PulseTrack analytical platform.

Question 7 - Data Mart Design and Implementation

7.1 Data Mart Overview

The PulseTrack Data Mart layer was developed to support business reporting and analytical queries using the validated Gold warehouse data.

Instead of repeatedly joining detailed fact and dimension tables for every report, the Data Mart layer provides summarized datasets organized around important business perspectives such as campaigns, advertisers, creatives, partners, account managers and revenue adjustments.

The implementation consists of six daily reporting tables stored under the PostgreSQL `mart` schema. These tables support analytical activities such as performance comparisons, conversion analysis, revenue monitoring and investigation of financial adjustments.

The marts are refreshed using the Python script `load_mart.py`, which executes the Data Mart transformation SQL. This process is also included in the Apache Airflow pipeline, allowing it to run after the Gold warehouse loading stages.

7.2 Data Mart Tables and Verification

Six Data Mart tables were implemented and verified in PostgreSQL.

Data Mart Table	Reporting Purpose	Verified Row Count
`campaign_daily`	Campaign performance and conversion reporting	198,129
`advertiser_daily`	Advertiser-level performance reporting	28,835
`creative_daily`	Creative performance and engagement analysis	357,587
`partner_daily`	Partner performance and attribution reporting	69,063
`account_manager_daily`	Account-manager performance reporting	22,654
`adjustment_daily`	Revenue adjustment and financial monitoring	11,050
Total		**687,318**

The row counts were verified directly in DataGrip using SQL `COUNT(*)` queries against each Data Mart table.

The six Data Mart tables contained a combined total of 687,318 reporting rows. Since each table serves a different analytical purpose, its grain is defined separately. Grain specifies what one row represents and prevents incorrect aggregation or double counting.

The implemented Data Mart grains and measures are summarized below.

Data Mart Table	Grain (One Row Per)	Main Measures
`campaign_daily`	Date and campaign	Impressions, clicks, conversions, eligible base revenue and adjustments
`advertiser_daily`	Date and advertiser dimension version	Impressions, clicks, conversions, eligible base revenue and adjustments
`creative_daily`	Date and creative	Impressions, clicks, attributed conversions, weighted conversion credit and attributed revenue
`partner_daily`	Date and partner group, including UNRESOLVED	Clicks, attributed conversions, weighted conversion credit and attributed revenue
`account_manager_daily`	Date, account manager and advertiser dimension version	Impressions, clicks, conversions, eligible base revenue and adjustments
`adjustment_daily`	Adjustment posting date and campaign group	Adjustment counts, matched and orphan adjustments, eligible adjustment amounts and seven-day-late business adjustment measures

These grains are implemented through the grouping and aggregation logic in `sql/load_mart.sql`.

Campaign, advertiser and account-manager reporting aggregate the applicable Gold business activity at the corresponding reporting date. Creative and partner attribution measures use the relevant conversion dates, while the adjustment mart uses adjustment posting dates.

Revenue adjustments are maintained separately from original conversion counts to avoid double counting. Unresolved partner or campaign references are retained in explicit reporting groups where applicable.

The Data Mart tables therefore provide summarized business measures while preserving the distinctions between reporting entities, event dates and adjustment dates.

The Data Mart refresh audit also confirmed a successful execution with a processed output count of 687,318 rows.

	mart_table	total_rows
1	account_manager_daily	22654
2	adjustment_daily	11050
3	advertiser_daily	28835
4	campaign_daily	198129
5	creative_daily	357587
6	partner_daily	69063

Figure 19: Verified Row Counts of the Six PulseTrack Data Mart Tables

7.3 Data Mart Transformation and Refresh Logic

The PulseTrack Data Mart layer is populated from the Gold warehouse using SQL-based transformations. The transformation logic is maintained in `sql/load_mart.sql` and executed through the Python script `scripts/load_mart.py`.

The six Data Mart tables organize warehouse data into daily reporting structures for campaigns, advertisers, creatives, partners, account managers and revenue adjustments. This reduces the need for reporting queries to repeatedly process detailed warehouse facts and dimension relationships.

7.3.1 Data Transformation

The Data Mart transformations use Gold fact and dimension data to calculate reporting measures such as impressions, clicks, conversions and revenue.

Revenue reporting distinguishes base conversion revenue from applicable revenue adjustments. This allows financial corrections to be reflected in adjusted revenue figures without creating duplicate conversion records.

The reporting tables also support different business perspectives. For example, campaign-level reporting focuses on campaign performance, while partner-level reporting supports attribution analysis. Revenue adjustments are additionally maintained in a dedicated mart for financial investigation.

7.3.2 Refresh Execution

The `load_mart.py` script executes the Data Mart SQL transformations and refreshes the related OLAP materialized cache.

These operations are performed within a database transaction so that the Data Mart refresh and analytical cache refresh are committed together after successful execution.

The refresh process is integrated into the Apache Airflow workflow and runs after Gold warehouse loading.

7.3.3 Refresh Verification

The completed Data Mart refresh produced 687,318 rows across the six reporting tables.

The improved audit mechanism recorded the successful refresh and its processed output count in `audit.pipeline_runs`.

The subsequent validation script confirmed that all 14 Gold-to-Mart reconciliation checks passed. Attribution reconciliation and OLAP cache consistency validation also passed.

These results demonstrate that the reporting tables were refreshed successfully and that the verified analytical measures remained consistent with the underlying Gold warehouse.

7.4 Business Reporting and Data Mart Usage

The PulseTrack Data Marts provide summarized information for different business reporting requirements. They support performance monitoring, comparisons across business entities and investigation of revenue-related issues.

7.4.1 Campaign and Advertiser Performance

The `campaign_daily` and `advertiser_daily` marts support the evaluation of advertising performance over time.

Business users can compare impressions, clicks, conversions and revenue across campaigns and advertisers. These measures help identify strong-performing campaigns, changes in engagement and differences between traffic activity and conversion outcomes.

The validated campaign reporting data contained 291,032 impressions, 145,604 clicks and 87,277 total conversions within the applicable reporting scope. Of these, 3,640 conversions met the defined revenue-eligibility criteria.

Campaign-level revenue reconciliation also confirmed USD 466,632.73 in eligible base revenue and USD 469,913.19 after applicable revenue adjustments.

7.4.2 Creative and Partner Analysis

The `creative_daily` mart supports comparisons between advertising creatives, including engagement and conversion-related performance.

The `partner_daily` mart supports traffic-partner analysis and revenue-attribution reporting. It helps identify the contribution of different partners to advertising outcomes.

However, some partner references remain unresolved in the source data. These are retained for investigation rather than being assigned to an unsupported partner identity.

7.4.3 Account Manager and Financial Reporting

The `account_manager_daily` mart supports performance analysis based on account-manager responsibilities. It also provides a reporting basis for investigating changes in account-manager assignments over time.

The `adjustment_daily` mart supports financial monitoring by organizing revenue-adjustment activity separately from original conversion records.

This separation helps business users investigate corrections, understand their effect on revenue and reconcile adjusted revenue with the original conversion activity.

Together, these six Data Marts provide the reporting foundation for the semantic layer and the business-analysis queries presented in Question 9.

7.5 Data Mart Summary and Limitations

The PulseTrack Data Mart layer was successfully implemented using six daily reporting tables designed for campaign, advertiser, creative, partner, account-manager and revenue-adjustment analysis.

The completed refresh produced 687,318 reporting rows across the six tables. The refresh was recorded successfully in the pipeline audit, and all 14 Gold-to-Mart reconciliation checks passed. Attribution reconciliation and OLAP cache consistency validation also passed.

Several limitations remain. The current implementation uses full Data Mart refreshes rather than a demonstrated incremental-refresh mechanism, which may become less efficient as data volumes increase.

Additionally, unresolved source relationships, particularly partner references and some revenue-adjustment links, can affect the completeness of certain business analyses. These limitations are preserved and reported rather than hidden through unsupported assumptions.

Overall, the Data Mart layer provides a verified foundation for analytical reporting while identifying areas for future improvement in scalability, incremental processing and source-data quality.

Question 8 - OLAP Implementation and Analytical Performance

8.1 OLAP Implementation Overview

The PulseTrack analytical layer was developed using PostgreSQL views, Data Mart tables and a materialized view to support Online Analytical Processing (OLAP).

The implementation allows business users to analyze campaign performance across different reporting periods and business dimensions.

The semantic layer provides reusable analytical structures built on the validated Data Mart and Gold warehouse data. These structures support monthly performance analysis, business KPI calculations and comparisons across reporting periods.

The implementation uses PostgreSQL rather than a separate commercial OLAP platform. Analytical queries and precomputed results provide the required multidimensional reporting capabilities within the existing data warehouse environment.

8.2 Monthly Analytical View and Materialized Cache

The monthly OLAP model is implemented using two PostgreSQL objects:

- `semantic.campaign_performance_monthly` – a standard analytical view that calculates monthly campaign performance.
- `semantic.campaign_performance_monthly_cache` – a materialized view that stores precomputed monthly results.

The analytical view combines reporting data with campaign and date information, performs monthly aggregation and applies window calculations for period-based analysis.

The materialized cache stores the calculated results so that repeated analytical queries do not need to execute the complete aggregation process every time.

8.2.1 OLAP Data Consistency Verification

SQL validation confirmed that both the standard view and the materialized cache contained 7,200 rows.

A bidirectional `EXCEPT ALL` comparison returned zero mismatched rows, confirming that the cached analytical results matched the original view at the time of testing.

8.2.2 Query Performance Comparison

Query performance was measured using PostgreSQL `EXPLAIN (ANALYZE, BUFFERS)`.

Both queries retrieved the same 7,200 analytical rows. The tests were repeated, and the second execution times were used for comparison.

Query	Execution Time
Standard monthly analytical view	959.571 ms
Monthly materialized cache	0.547 ms

The materialized cache was approximately 1,754 times faster, representing a 99.94% reduction in measured execution time.

The execution plan showed that the standard analytical view processed 198,129 daily Data Mart rows using joins, sorting, aggregation and window operations. In contrast, the materialized cache retrieved the previously calculated monthly results directly.

These findings demonstrate that materialized caching can significantly reduce execution time for repeated analytical queries.

However, the measurements represent a small local benchmark using the current dataset and environment. Performance may differ with larger datasets, other queries or different system conditions.

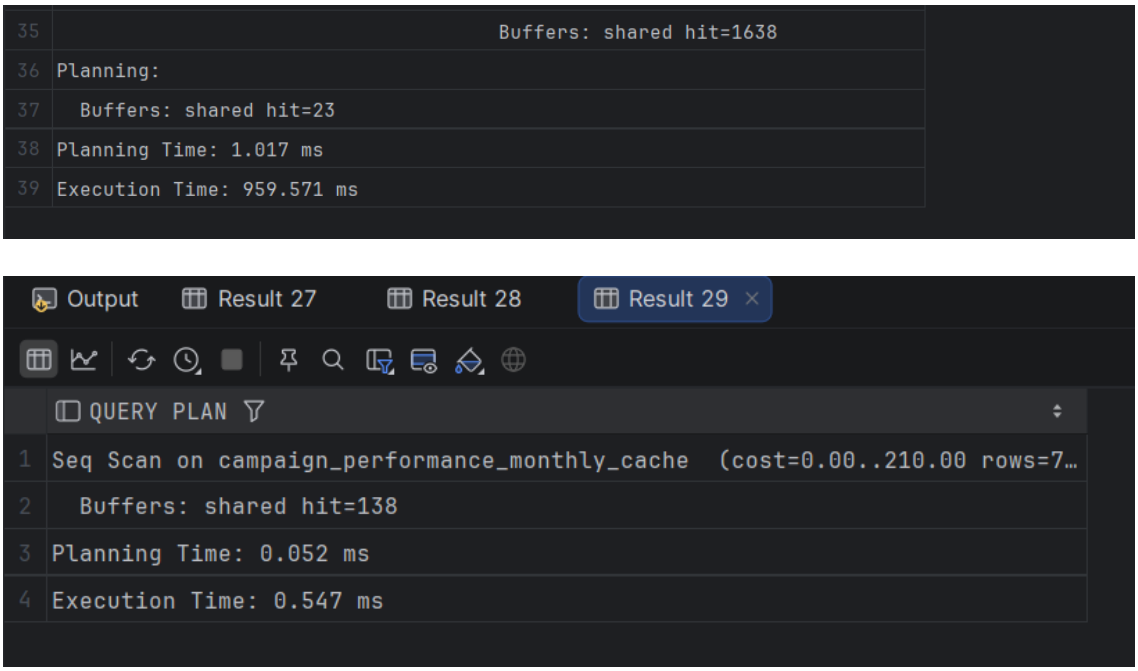


Figure 20: PostgreSQL OLAP Query Performance Comparison Between the Analytical View and Materialized Cache

8.3 Multidimensional Analysis and OLAP Operations

The PulseTrack semantic layer supports multidimensional analysis using PostgreSQL analytical queries over the Data Mart and Gold warehouse data.

The analytical model organizes information by time and business entities, allowing users to examine performance at different levels of detail.

8.3.1 Analytical Dimensions and Measures

The PulseTrack OLAP model supports analysis across time and business dimensions using the existing Gold warehouse, Data Mart and semantic layer.

Analytical Dimensions and Hierarchies

The following hierarchies are supported by the warehouse relationships and reporting structures.

Dimension	Analytical Hierarchy	Purpose
Time	Year → Quarter → Month → Day	Supports period comparisons and drill-down
Advertising	Advertiser → Offer → Campaign → Creative	Supports performance analysis across related advertising entities
Account Manager	Account Manager → Advertiser	Supports historical responsibility and advertiser performance reporting
Partner	Channel → Partner	Supports partner grouping where valid channel information is available

The time hierarchy is implemented through the Gold date dimension, daily reporting marts and monthly semantic model.

The advertising hierarchy follows the relationships between advertisers, offers, campaigns and creatives. Historical account-manager analysis uses the effective advertiser dimension version rather than automatically assigning past performance to the current manager.

Partner analysis preserves unresolved identities instead of assigning them to unsupported partner groups.

These warehouse hierarchies provide the basis for multidimensional SQL analysis. The current implementation does not claim to provide every hierarchy as a separately configured multidimensional cube.

Measures and Additive Classification

The analytical model contains measures such as impressions, clicks, conversions, base revenue, revenue adjustments, adjusted revenue and attributed revenue.

Measures are classified according to how they can be aggregated.

Measure Type	PulseTrack Examples	Aggregation Rule
Additive	Impressions, clicks, eligible conversions, base revenue and applicable revenue adjustments	Can be summed across compatible, non-overlapping reporting groups
Semi-additive	Period-end active advertiser counts, if introduced as snapshot measures	Could be aggregated across business groups but not summed across successive dates
Non-additive	CTR, conversion rate, revenue per click, revenue per conversion, attribution percentage and revenue growth percentage	Must be recalculated from their underlying measures

The current implementation primarily uses event-based additive measures and calculated non-additive KPIs. A separate semi-additive snapshot fact has not been implemented.

Adjusted revenue is calculated by combining eligible base revenue with applicable revenue adjustments according to the reporting scope.

Ratios and percentages must not be summed or directly averaged when moving between reporting levels. For example, CTR is recalculated as total clicks divided by total impressions, multiplied by 100.

Similarly, distinct conversion counts must be handled carefully when analyzing overlapping attribution groups because summing the group counts could double-count the same conversion.

These aggregation rules help maintain consistent KPI results during multidimensional reporting.

8.3.2 Roll-Up and Drill-Down

Roll-up combines detailed information into a higher reporting level. For example, monthly campaign performance can be summarized into quarterly or yearly totals.

Drill-down provides the opposite analysis, allowing a business user to move from yearly or quarterly results toward monthly or daily performance.

PostgreSQL `ROLLUP` and `GROUPING SETS` support these operations using the established date hierarchy.

These capabilities help users identify broad performance trends and investigate the periods responsible for changes.

8.3.3 Slice and Dice

A slice selects a particular value from an analytical dimension. For example, campaign performance can be examined for one reporting month.

Dice applies multiple conditions to create a more focused analytical dataset. For example, users can compare selected advertisers or campaigns across a particular reporting period.

These operations support targeted investigation without requiring changes to the underlying warehouse structure.

8.3.4 Ranking and Comparative Analysis

The PulseTrack semantic layer supports analytical ranking, Top-N selection, percentage contribution and pivot-style reporting using PostgreSQL SQL queries.

Top-N and Ranking

Top-N analysis identifies the highest-performing business entities based on a selected measure, such as adjusted revenue, clicks or conversions.

PostgreSQL window functions such as `RANK()` and `DENSE_RANK()` can assign performance rankings, while `ORDER BY` and `LIMIT` can retrieve the leading entities.

The business analytics implementation demonstrates ranking through the quarterly advertiser analysis and the revenue-attributed partner analysis.

Percentage Contribution

Percentage contribution measures how much an entity contributes to an overall reporting total.

The calculation is:

Percentage Contribution = (Entity Revenue / Total Revenue) × 100

The campaign revenue-concentration analysis in Question 9 applied this approach using a threshold of greater than 10%. No campaign exceeded that threshold within the defined reporting scope.

Percentage contributions must be calculated against the appropriate reporting total to avoid misleading comparisons.

Pivot Analysis

Pivot analysis presents values from one dimension as separate columns, making comparisons between reporting periods easier.

For example, monthly campaign revenue can be displayed with January, February and March as separate columns.

In PostgreSQL, this can be implemented using conditional aggregation with the `FILTER` clause. It allows users to compare performance across reporting periods without modifying the underlying Data Mart tables.

Together, these operations support ranking, contribution analysis and multidimensional reporting using the existing semantic analytical model.

OLAP Pivot, Top-N and Ranking Verification

A practical SQL query was executed against `semantic.campaign_performance_monthly` to demonstrate pivot-style reporting, Top-N selection, ranking and percentage contribution.

The query displayed January, February and March 2025 adjusted revenue in separate columns and ranked campaigns by their total Q1 2025 adjusted revenue.

The highest-ranked campaign was CMP00177, with USD 1,175.76 in adjusted revenue, representing 1.54% of total Q1 adjusted revenue.

The next four campaigns were CMP00063 (USD 765.40), CMP00174 (USD 706.75), CMP00080 (USD 698.77) and CMP00238 (USD 692.00).

The results demonstrated that the semantic layer can support multiple OLAP operations within a single query using conditional aggregation, window functions and calculated contribution percentages.

8.3.5 OLAP Roll-Up Verification

A practical OLAP roll-up was executed using PostgreSQL `GROUPING SETS` to aggregate monthly campaign performance into quarterly and yearly reporting levels.

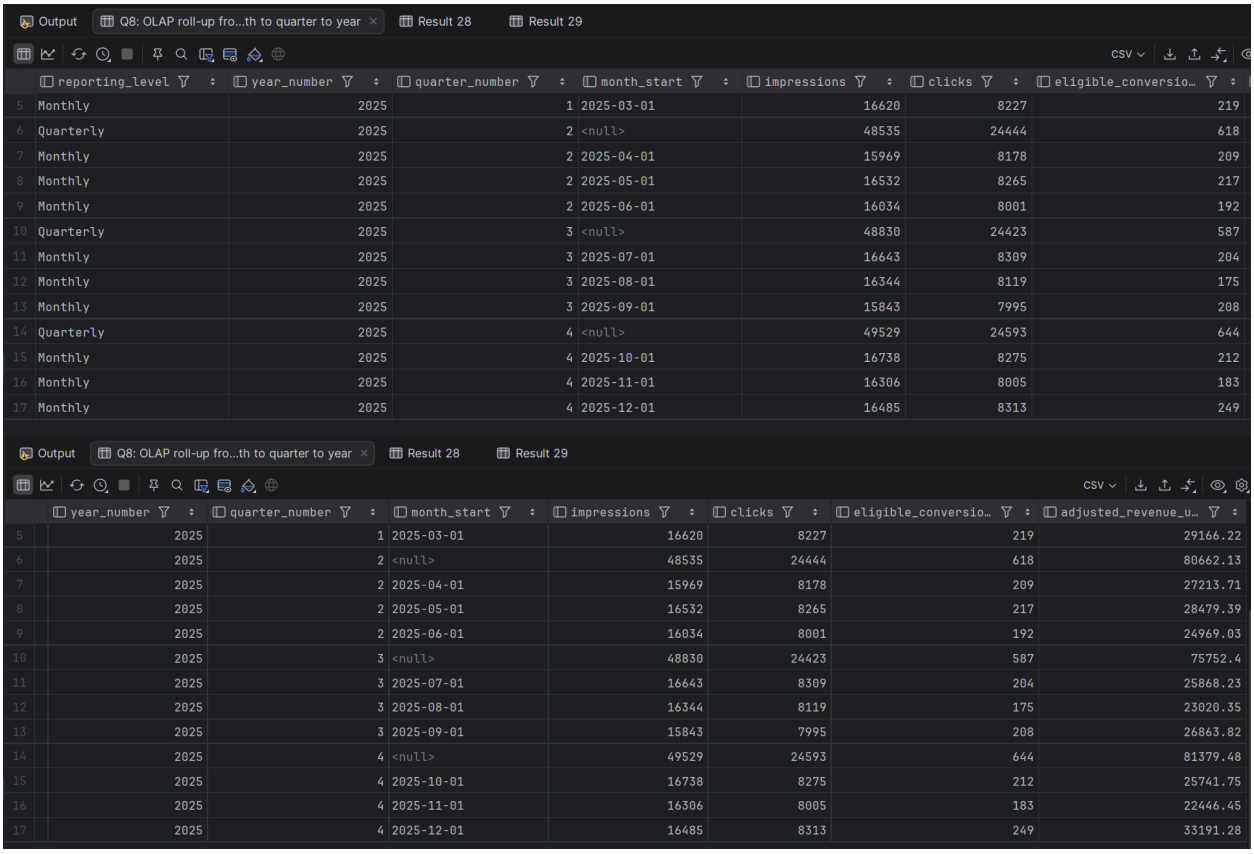
The query returned 12 monthly records, four quarterly summaries and one yearly summary for 2025.

The yearly results contained 195,243 impressions, 97,469 clicks, 2,429 eligible conversions and USD 314,144.44 in adjusted revenue within the analytical reporting scope.

The four quarterly adjusted revenue totals were USD 76,350.43, USD 80,662.13, USD 75,752.40 and USD 81,379.48. Their sum matched the yearly total exactly.

The monthly figures also reconciled with the corresponding quarterly totals, demonstrating consistent aggregation across the time hierarchy.

This test verifies the ability to analyze campaign performance at different time granularities without changing the underlying analytical data.



Output | Q8: OLAP roll-up fro...th to quarter to year | Result 28 | Result 29

CSV

	reporting_level	year_number	quarter_number	month_start	impressions	clicks	eligible_conversio…
5	Monthly	2025	1	2025-03-01	16620	8227	219
6	Quarterly	2025	2	<null>	48535	24444	618
7	Monthly	2025	2	2025-04-01	15969	8178	209
8	Monthly	2025	2	2025-05-01	16532	8265	217
9	Monthly	2025	2	2025-06-01	16034	8001	192
10	Quarterly	2025	3	<null>	48830	24423	587
11	Monthly	2025	3	2025-07-01	16643	8309	204
12	Monthly	2025	3	2025-08-01	16344	8119	175
13	Monthly	2025	3	2025-09-01	15843	7995	208
14	Quarterly	2025	4	<null>	49529	24593	644
15	Monthly	2025	4	2025-10-01	16738	8275	212
16	Monthly	2025	4	2025-11-01	16306	8005	183
17	Monthly	2025	4	2025-12-01	16485	8313	249

Output | Q8: OLAP roll-up fro...th to quarter to year | Result 28 | Result 29

CSV

	year_number	quarter_number	month_start	impressions	clicks	eligible_conversio…	adjusted_revenue_u…
5	2025	1	2025-03-01	16620	8227	219	29166.22
6	2025	2	<null>	48535	24444	618	80662.13
7	2025	2	2025-04-01	15969	8178	209	27213.71
8	2025	2	2025-05-01	16532	8265	217	28479.39
9	2025	2	2025-06-01	16034	8001	192	24969.03
10	2025	3	<null>	48830	24423	587	75752.4
11	2025	3	2025-07-01	16643	8309	204	25868.23
12	2025	3	2025-08-01	16344	8119	175	23020.35
13	2025	3	2025-09-01	15843	7995	208	26863.82
14	2025	4	<null>	49529	24593	644	81379.48
15	2025	4	2025-10-01	16738	8275	212	25741.75
16	2025	4	2025-11-01	16306	8005	183	22446.45
17	2025	4	2025-12-01	16485	8313	249	33191.28

Figure 21: OLAP Roll-Up Verification Across Monthly, Quarterly and Yearly Reporting Levels

8.4 Period-over-Period Analytical Comparisons

Period-over-period analysis was implemented using the PulseTrack monthly semantic model to compare adjusted revenue across different reporting periods.

Three comparison methods were verified: Month-over-Month (MoM), Quarter-over-Quarter (QoQ) and Year-over-Year (YoY).

The SQL queries aggregate adjusted revenue at the required time level and calculate percentage changes using the previous comparable period. Division-by-zero protection is applied using `NULLIF`.

8.4.1 Month-over-Month Analysis

Month-over-Month analysis compares adjusted revenue with the immediately preceding month.

The verified results showed changes in adjusted revenue throughout the first six months of 2026.

Month	Adjusted Revenue (USD)	MoM Growth
January 2026	25,962.67	-21.78%
February 2026	28,143.96	+8.40%
March 2026	28,979.76	+2.97%
April 2026	28,715.63	-0.91%
May 2026	28,218.16	-1.73%
June 2026	15,748.57	-44.19%

The largest monthly decline occurred in June 2026, when adjusted revenue decreased from USD 28,218.16 in May to USD 15,748.57, representing a reduction of 44.19%.

8.4.2 Quarter-over-Quarter Analysis

Quarter-over-Quarter analysis compares the total adjusted revenue of each quarter with the previous quarter.

The monthly semantic data was aggregated into calendar quarters and compared using the same growth calculation.

Quarter	Adjusted Revenue (USD)	QoQ Growth
Q2 2025	80,662.13	+5.65%
Q3 2025	75,752.40	-6.09%
Q4 2025	81,379.48	+7.43%
Q1 2026	83,086.39	+2.10%
Q2 2026	72,682.36	-12.52%

The results showed that Q2 2026 adjusted revenue decreased by USD 10,404.03 compared with Q1 2026, representing a decline of 12.52%.

This demonstrates how OLAP aggregation supports analysis at a higher time hierarchy level.

8.4.3 Year-over-Year Analysis

Year-over-Year analysis compares the same calendar month across different years, allowing users to examine performance while accounting for the reporting period.

The query compared January–June 2026 with the corresponding months in 2025.

Month	YoY Growth
January 2026	+35.03%
February 2026	+0.67%
March 2026	-0.64%
April 2026	+5.52%
May 2026	-0.92%
June 2026	-36.93%

January 2026 showed the strongest positive YoY growth at 35.03%.

June 2026 showed the largest decline, with adjusted revenue decreasing from USD 24,969.03 in June 2025 to USD 15,748.57 in June 2026, a reduction of 36.93%.

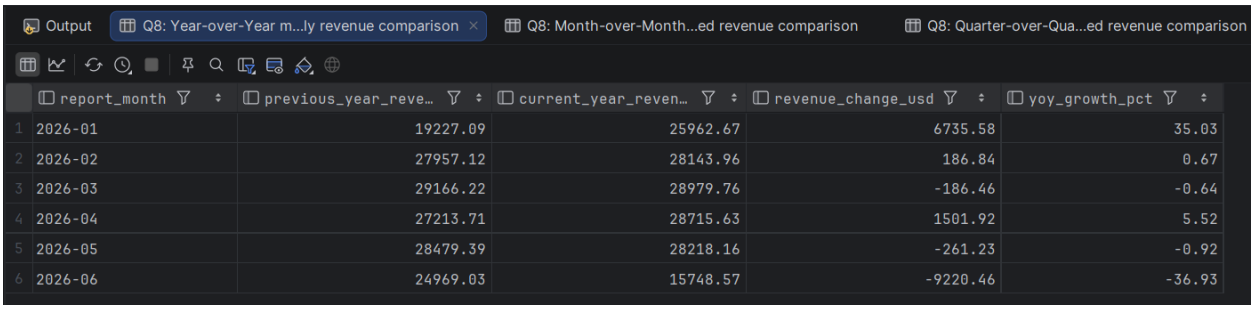
8.4.4 Analytical Interpretation and Limitations

The verified MoM, QoQ and YoY calculations demonstrate that the PulseTrack semantic layer supports analytical comparisons across multiple time granularities.

These comparisons help identify periods that may require further business investigation.

However, changes in reported revenue should not automatically be attributed to campaign performance. Incomplete source deliveries, delayed updates, revenue adjustments and other business factors may influence the observed values.

Therefore, period comparisons should be interpreted alongside data-completeness checks and additional business context.



	report_month	previous_year_reve…	current_year_reven…	revenue_change_usd	yoy_growth_pct
1	2026-01	19227.09	25962.67	6735.58	35.03
2	2026-02	27957.12	28143.96	186.84	0.67
3	2026-03	29166.22	28979.76	-186.46	-0.64
4	2026-04	27213.71	28715.63	1501.92	5.52
5	2026-05	28479.39	28218.16	-261.23	-0.92
6	2026-06	24969.03	15748.57	-9220.46	-36.93

Figure 22: Year-over-Year Adjusted Revenue Comparison for January–June 2026

8.5 OLAP Performance Considerations and Limitations

The PulseTrack OLAP implementation uses PostgreSQL analytical views and materialized caching to support multidimensional reporting without requiring a separate OLAP server.

Performance testing demonstrated that the materialized monthly cache was substantially faster than the corresponding standard analytical view. The standard view required 959.571 milliseconds, while the materialized cache required 0.547 milliseconds to retrieve the same 7,200 rows.

This improvement occurs because the standard view performs joins, aggregation, sorting and window calculations when queried, whereas the materialized cache stores the previously calculated results.

However, the implementation has several limitations.

First, materialized views do not automatically reflect changes to underlying data. The cache must be refreshed after the Data Mart is updated. The PulseTrack refresh process addresses this by refreshing the materialized cache as part of the Data Mart loading transaction.

Second, the current implementation uses a full refresh rather than a demonstrated incremental cache-maintenance mechanism. Refreshing the entire analytical dataset may become more expensive as data volumes increase.

Third, the performance comparison was conducted using the existing dataset and local PostgreSQL environment. The measured improvement may vary with data volume, query complexity, available memory and concurrent users.

Finally, the OLAP implementation provides SQL-based multidimensional analysis rather than a dedicated multidimensional cube engine. It supports the required reporting operations through relational queries and precomputed analytical results.

Future improvements could include incremental refresh strategies, additional query-performance testing, monitoring of cache freshness and indexing optimizations based on actual query workloads.

High-Cardinality Dimensions and Scalability

High-cardinality attributes, such as individual transaction identifiers, can increase the number of groups that analytical queries must process and may reduce query performance.

In PulseTrack, detailed impression, click and conversion records are maintained in the Gold warehouse, while the Data Mart layer provides aggregated datasets for commonly requested business analysis.

This separation reduces the need for OLAP queries to repeatedly process detailed transactional records. The monthly materialized cache further improves performance by storing precomputed campaign-level results.

The existing implementation also uses PostgreSQL indexes and SQL aggregation to support analytical queries.

For larger production workloads, additional improvements could include date-based partitioning, incremental aggregation, query-specific indexing and monitoring of frequently accessed high-cardinality attributes.

These improvements would require further testing under larger data volumes. The current performance benchmark demonstrates the benefit of materialized caching but does not establish scalability under all workload conditions.

8.6 OLAP Implementation Summary

The PulseTrack OLAP layer was successfully implemented using PostgreSQL semantic views, daily Data Marts and a monthly materialized cache.

The implementation supports multidimensional analysis through time hierarchies, aggregation, roll-up, drill-down, slice, dice, ranking and period-over-period comparisons.

Practical verification demonstrated that monthly, quarterly and yearly roll-up results reconciled correctly. Month-over-Month, Quarter-over-Quarter and Year-over-Year revenue comparisons were also executed successfully using the semantic analytical model.

The standard monthly view and its materialized cache each contained 7,200 rows, with zero differences identified during reconciliation.

Performance testing demonstrated approximately 1,754 times faster execution for the materialized cache in the measured comparison.

Overall, the implementation provides consistent analytical results and faster access to frequently requested monthly summaries. Its main areas for future improvement are refresh efficiency, scalability and continued performance monitoring.

8.7 Partner-Channel Allocation Conservation Test (19 October 2026)

An additional read-only SQL reconciliation compared daily partner metrics in mart.partner_daily against semantic.partner_channel_allocation_daily. Results were grouped by date_key and compared with a FULL OUTER JOIN to detect missing or extra dates. Absolute differences were evaluated for clicks, weighted conversion credit and attributed revenue with a tolerance of 0.000001. The SQL is included as Query 16 in sql/validation/verify_semantic_model.sql.

The test checked 546 reporting dates, returned zero mismatched dates, and found maximum differences of 0 for clicks, weighted conversion credit and attributed revenue. This provides direct evidence that the implemented partner-channel allocation preserves those source-mart measures across reporting dates and avoids inflation from allocating a partner to multiple reporting channels.

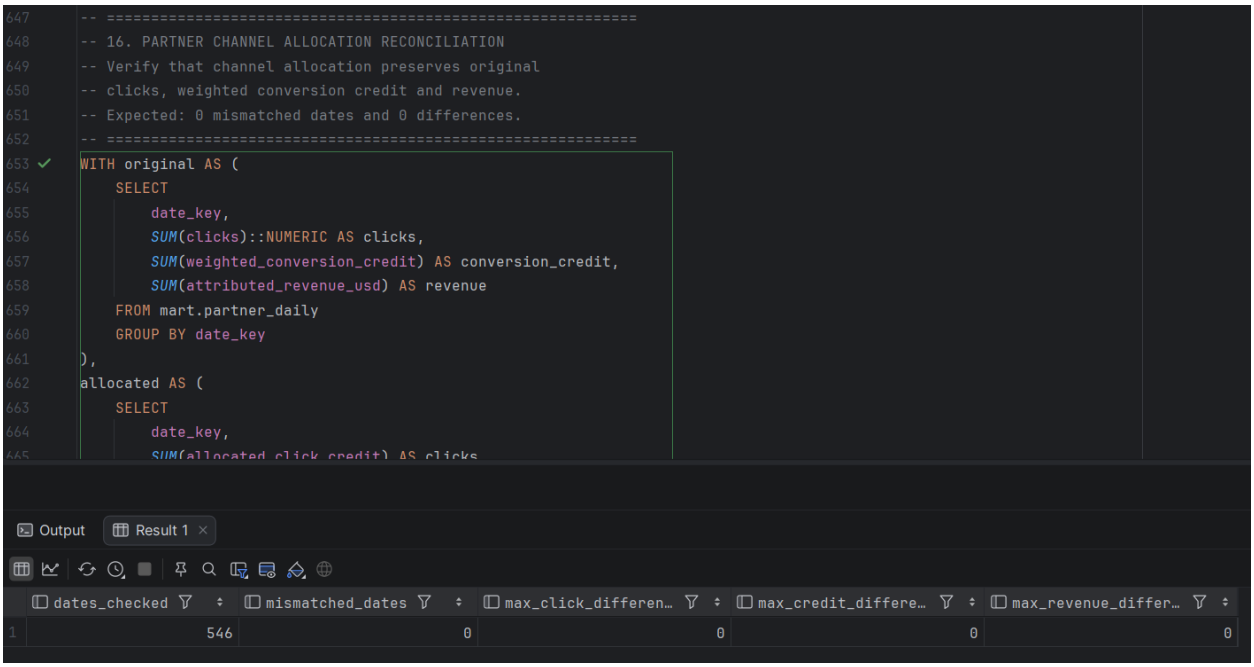


Figure 23: Partner-Channel Allocation Reconciliation Across 546 Reporting Dates

The result validates arithmetic conservation for this tested dataset. It does not independently establish the correctness of every business assumption behind the channel weights or generalize to data not included in the comparison.

Question 9 - Business Analytics and Insights

9.1 Business Analytics Overview

The PulseTrack analytical platform was used to investigate advertising performance, conversion behavior, revenue trends, partner attribution and financial adjustments.

Ten business-analysis queries were developed using the PostgreSQL semantic layer. These queries demonstrate how the completed data warehouse can support business decisions rather than only storing and transforming data.

The analyses use reporting measures such as impressions, clicks, conversions, click-through rate (CTR), conversion rate, adjusted revenue and attributed revenue.

The SQL queries are maintained in `sql/validation/pulsetrack_business_analytics.sql`. Their results provide evidence for identifying performance trends, investigating unusual outcomes and highlighting data-quality limitations that affect business reporting.

9.2 Analytical Questions and KPI Traceability

The ten business queries address the following analytical requirements.

No.	Business Analysis	Purpose
1	Top advertisers by quarter	Identify leading advertisers based on revenue
2	High CTR but low conversion rate	Detect campaigns attracting clicks without comparable conversions
3	Top revenue-attributed partners	Identify partners associated with attributed revenue
4	Month-over-Month revenue changes	Investigate changes in adjusted revenue
5	Campaign revenue contribution above 10%	Identify campaigns representing a significant share of total revenue
6	Creative underperformance	Compare creative CTR against its campaign
7	Account-manager assignment changes	Compare performance before and after historical reassignment
8	Partner revenue-attribution share	Examine the distribution of attributed revenue
9	Increasing impressions but declining conversions	Identify potential performance inconsistencies
10	Revenue adjustments occurring seven or more days later	Investigate delayed financial corrections

9.2.1 KPI Calculation and Data Lineage

The business analyses retrieve their results from reusable semantic views rather than querying raw source tables directly.

The underlying pipeline follows the established data flow:

Source Data → Bronze → Silver → Gold → Data Mart → Semantic Layer → Business Analysis

Bronze preserves original source records. Silver performs standardization and validation. Gold organizes the data into dimensional and fact structures, while the Data Mart layer provides summarized reporting datasets.

The semantic layer defines reusable business measures and calculations that are consumed by the analytical SQL queries.

For example, CTR is calculated from clicks and impressions, while adjusted revenue incorporates applicable revenue adjustments into the revenue calculation.

The Gold-to-Mart reconciliation checks, validated semantic results and pipeline audit records provide supporting evidence for the reliability and traceability of these calculations.

9.2.2 Interpretation of Analytical Results

The business-analysis results are interpreted within their reporting scope and data-quality constraints.

In particular, unresolved partner references must not be assigned to an unsupported partner identity. Historical account-manager comparisons must respect effective assignment dates, and changes in revenue should not automatically be treated as evidence of business causation.

These considerations help distinguish verified analytical findings from assumptions requiring further investigation.

9.3 Business Analytics Results and Insights

The ten business-analysis queries were used to investigate advertising performance, revenue generation and potential operational issues.

The following findings are based on the PulseTrack semantic reporting layer and demonstrate how warehouse data can support practical business decisions.

9.3.1 Analysis 1 – Top Advertisers by Quarter

Business Question: Which advertisers generated the highest adjusted revenue in each reporting quarter?

The analysis ranked advertisers based on their quarterly adjusted revenue.

Selected results included:

Reporting Quarter	Leading Advertiser	Adjusted Revenue (USD)
Q1 2025	QuickTax	3,539.19
Q2 2026	PawPlan	3,239.21

Business Insight:

The results demonstrate how advertiser performance can be compared across reporting quarters.

Identifying leading advertisers helps account managers monitor important revenue contributors and investigate changes in advertiser performance over time.

However, these values represent the leading advertisers within their respective quarters and should not be interpreted as their total revenue across the entire reporting period.

9.3.2 Analysis 2 – High CTR but Low Conversion Rate

Business Question: Which campaigns attract a high proportion of clicks but achieve relatively low conversion rates?

This analysis compared click-through rate (CTR) with conversion rate to identify campaigns where user engagement does not translate into comparable conversion outcomes.

A notable result was the SeniorSafe Signup campaign.

Campaign	CTR	Conversion Rate
SeniorSafe Signup	58.08%	1.48%

Business Insight:

The campaign demonstrated a high click-through rate of 58.08%, but its conversion rate was only 1.48%.

This difference suggests that attracting clicks does not necessarily lead to successful conversions.

The business team could investigate the campaign's landing page, audience targeting, offer suitability and conversion process.

The results identify a potential optimization opportunity but do not establish the cause of the low conversion rate.

9.3.3 Analysis 3 – Top Revenue-Attributed Partners

Business Question: Which identifiable traffic partners contributed the most attributed revenue?

The analysis ranked partners using attributed revenue from the semantic reporting layer.

One leading identified partner was Digital Solutions, associated with partner identifier 49552.

Partner	Partner ID	Attributed Revenue (USD)
Digital Solutions	49552	750.45

Business Insight:

The results provide visibility into revenue attributed to individual traffic partners, supporting partner performance monitoring and commercial decision-making.

However, the source data contains unresolved partner references that cannot be reliably assigned to known partners.

Therefore, rankings of identified partners should be interpreted separately from unresolved attribution. The analysis does not establish that the leading identified partner generated the highest revenue across all traffic, including unmatched records.

This distinction is important for preventing misleading partner-performance conclusions.

9.3.4 Analysis 4 – Month-over-Month Revenue Changes

Business Question: Which reporting periods experienced significant changes in adjusted revenue?

The analysis compared monthly adjusted revenue to identify periods with notable increases or decreases.

A significant decline was identified between May and June 2026.

Reporting Month	Adjusted Revenue (USD)
May 2026	28,218.16
June 2026	15,748.57
Revenue change	-12,469.59
MoM growth	-44.19%

Business Insight:

Adjusted revenue decreased by 44.19% between May and June 2026.

This result highlights a period requiring further business investigation. Possible areas to examine include campaign activity, conversion performance, revenue adjustments and source-data completeness.

Although the reporting periods contain calendar-date records, this does not guarantee that every expected source record was received. Therefore, the decline should not automatically be interpreted as a confirmed deterioration in business performance.

9.3.5 Analysis 5 – Campaign Revenue Concentration

Business Question: Did any individual campaign contribute more than 10% of total adjusted revenue?

The analysis calculated each campaign's contribution to overall adjusted revenue and applied a threshold of 10%.

The SQL query returned no campaigns exceeding the defined threshold.

Analysis Measure	Result
Revenue contribution threshold	Greater than 10%
Campaigns exceeding threshold	0
Query result	No matching campaigns

Business Insight:

Within the reporting scope and calculation rules used, no individual campaign contributed more than 10% of total adjusted revenue.

This suggests that the measured revenue was not dominated by a single campaign above the selected threshold.

The empty result is a valid analytical finding rather than a query failure.

However, it does not establish that revenue is evenly distributed across all campaigns. A more detailed concentration analysis would be required to assess the combined contribution of leading campaigns.

9.3.6 Analysis 6 – Creative Underperformance

Business Question: Which advertising creatives have a lower click-through rate than their associated campaign?

The analysis compared creative-level CTR against the overall CTR of the corresponding campaign.

One notable result was creative `CRE00277`.

Performance Measure	Result
Creative ID	CRE00277
Creative CTR	30.72%
Campaign CTR	52.56%
Approximate difference	-21.85 percentage points

Business Insight:

Creative CRE00277 recorded a substantially lower click-through rate than its associated campaign.

The difference indicates that the creative attracted a smaller proportion of clicks relative to its impressions than the campaign benchmark.

This finding could support further investigation of the creative's design, message, placement or audience targeting.

However, the comparison alone does not prove why the creative underperformed. Additional testing would be needed before deciding whether to replace or modify it.

9.3.7 Analysis 7 – Historical Account-Manager Assignment Changes

Business Question: How did advertiser performance differ before and after an account-manager reassignment?

The analysis investigated the LeanLife advertiser, whose account-manager assignment changed on 16 September 2025.

The historical comparison used the account-manager assignment timeline maintained through the Gold Slowly Changing Dimension Type 2 (SCD Type 2) implementation.

The analysis compared performance during 60-day periods before and after the reassignment.

Performance Measure	Before Reassignment	After Reassignment
Account Manager	Emma Hernandez (AM008)	Wei Williams (AM011)
Clicks	269	265
Eligible Conversions	5	8
Adjusted Revenue (USD)	663.38	1,232.13

Business Insight:

The results showed that clicks remained relatively similar across the two periods, while eligible conversions increased from 5 to 8 and adjusted revenue increased from USD 663.38 to USD 1,232.13.

The analysis demonstrates the importance of maintaining historical account-manager relationships. Without SCD Type 2, older business activity could incorrectly appear under the advertiser's current manager.

However, the performance difference should not be interpreted as proof that the reassignment caused the increase. Campaign conditions, traffic quality, timing and other factors may also have influenced the results.

9.3.8 Analysis 8 – Unresolved Partner Revenue Attribution

Business Question: What proportion of eligible base revenue was associated with unresolved partner attribution?

The analysis investigated the distribution of partner-attributed revenue and identified the amount that could not be assigned reliably to a known partner.

Attribution Measure	Result
Attribution category	UNRESOLVED
Unresolved attributed revenue (USD)	357,060.02
Share of eligible base revenue	76.52%

Business Insight:

Approximately 76.52% of eligible base revenue was categorized under unresolved partner attribution.

This represents a significant limitation for partner-level performance reporting.

The result indicates that partner rankings should be interpreted cautiously because a substantial amount of revenue cannot be associated confidently with an identified partner.

Further investigation should focus on source partner identifiers, matching rules and the completeness of attribution relationships.

The implementation preserves unresolved attribution explicitly instead of assigning revenue to a partner without sufficient supporting evidence.

	partner_id	partner_name	weighted_conversio…	attributed_revenue…	attribution_percen…	revenue_share_perc…
1	UNRESOLVED	Unresolved partner (no ver…	2784.1667	357060.02	76.4881	76.5184
2	49552	Digital Solutions	3.3333	750.45	0.0916	0.1608
3	85372	Performance Solutions	4	695.22	0.1099	0.149
4	63626	Growth Networks	4.4167	693.19	0.1213	0.1486
5	95139	Reach Labs	4.5	680.27	0.1236	0.1458
6	68515	Media Labs	3.3333	667.46	0.0916	0.143
7	96755	Reach Networks	3.9167	663	0.1076	0.1421
8	76046	Click Solutions	3.6667	646.66	0.1007	0.1386
9	14062	Lead Solutions	3.6667	645.73	0.1007	0.1384
10	48691	Digital Labs	4.5833	605.02	0.1259	0.1297
11	32037	Traffic Co	4.5	581.21	0.1236	0.1246
12	98358	Traffic Labs	2.8333	566.25	0.0778	0.1213
13	41584	Performance Co	2.5833	563.68	0.071	0.1208
14	77984	Performance Co	3.8333	539.44	0.1053	0.1156
15	33508	Click Partners	3.5	512.65	0.0962	0.1099

Figure 24: Business Analytics Query Result Showing Unresolved Partner Revenue Attribution

9.3.9 Analysis 9 – Increasing Impressions but Declining Conversions

Business Question: Which campaigns experienced increasing impressions while their conversion performance declined?

The analysis compared campaign activity across reporting periods to identify cases where greater exposure did not result in higher conversions.

One identified case involved the Sleepwell Lead Prospecting campaign.

Performance Measure	Earlier Period	Later Period
Impressions	36	57
Conversions	1	0

Business Insight:

The campaign's impressions increased from 36 to 57, while recorded conversions decreased from 1 to 0.

This demonstrates that increased advertising exposure does not necessarily result in improved conversion outcomes.

The finding could help the marketing team identify campaigns requiring further investigation into targeting, creative relevance, landing-page performance or conversion tracking.

However, the figures represent a relatively small number of impressions and conversions. The result should therefore be treated as an investigative signal rather than conclusive evidence of poor campaign performance.

9.3.10 Analysis 10 – Delayed Revenue Adjustments

Business Question: How many revenue adjustments were recorded with business adjustment dates at least seven days after their associated conversions?

The analysis compared conversion timestamps with revenue-adjustment timestamps to identify corrections recorded seven or more days after the original conversion.

Analysis Measure	Result
Adjustments meeting the seven-day condition	10,052
Net eligible adjustment amount (USD)	3,005.05

Business Insight:

The analysis identified 10,052 adjustment records whose business adjustment timestamps were at least seven days later than the corresponding conversion timestamps.

The net amount associated with eligible adjustments in this analysis was USD 3,005.05.

This demonstrates why financial reporting must support corrections made after the original conversion activity.

The warehouse maintains revenue adjustments separately from the original conversions, allowing adjusted revenue to reflect applicable corrections without duplicating conversion records.

Importantly, the seven-day difference measures the time between business events. It does not prove that the adjustment data was delivered to or ingested by the warehouse seven days late.

The analysis supports financial reconciliation and the investigation of historical revenue changes.

9.4 KPI Lineage and Business Decision Support

9.4.1 Business Analytics SQL Implementation

All ten business-analysis queries for Question 9 are organized in a dedicated SQL script, `sql/validation/pulsetrack_business_analytics.sql`.

The script contains the analytical queries corresponding to the ten business questions discussed in Section 9.3. Each query uses the PostgreSQL semantic layer to retrieve business measures and analytical results.

Maintaining the queries in a separate script improves code organization, supports repeatable execution and allows the analytical findings to be verified independently in DataGrip.

The SQL script is included in the project submission, while the report presents the corresponding results, interpretations and supporting evidence.

9.4.2 KPI Traceability Across Warehouse Layers

The PulseTrack business analytics implementation maintains a traceable data flow from original source records to the analytical results presented in Question 9.

The ten business-analysis queries access the semantic layer, which provides reporting measures derived from the Data Mart and underlying Gold warehouse.

The following table summarizes the lineage of important business measures.

The following table maps all ten business analyses to their semantic measures, reporting marts, Gold warehouse structures and original source datasets. Each analysis follows the Source → Bronze → Silver → Gold → Data Mart → Semantic Layer processing architecture.

Table: Business Analytics KPI-to-Source Traceability

Analysis	Semantic Measure / KPI	Data Mart and Gold Lineage	Silver, Bronze and Original Source
1. Top Advertisers	Quarterly adjusted revenue and ranking	`advertiser_daily`; Gold advertiser dimension, conversion and adjustment facts	Silver advertisers, conversion history and adjustments → Bronze relational records → advertiser, conversion and adjustment CSVs
2. High CTR, Low Conversion Rate	CTR and conversion rate	`campaign_daily`; Gold impression, click and conversion facts	Silver marketing transactions → Bronze relational records → impression, click and conversion CSVs
3. Top Partners	Attributed revenue and partner ranking	`partner_daily`; Gold partner dimension, conversion facts and attribution bridge	Silver partners, clicks and touchpoints → Bronze relational records → partner, click, conversion and touchpoint CSVs
4. Monthly Revenue Change	Adjusted revenue and MoM growth	`campaign_daily`; Gold date dimension, conversion and adjustment facts	Silver conversion history and adjustments → Bronze relational records → conversion and adjustment CSVs

5. Revenue Concentration	Campaign revenue contribution percentage	`campaign_daily`; Gold campaign dimension, conversion and adjustment facts	Silver campaign, conversion and adjustment data → Bronze relational records → campaign, conversion and adjustment CSVs
6. Creative Underperformance	Creative CTR compared with campaign CTR	`creative_daily` and `campaign_daily`; Gold impression and click facts, creative and campaign dimensions	Silver impressions, clicks and creatives → Bronze relational records → impression, click and creative CSVs
7. Account-Manager Changes	Historical clicks, eligible conversions and adjusted revenue	`account_manager_daily`; Gold SCD Type 2 advertiser dimension, account-manager dimension and transaction facts	Silver advertiser profiles, reference and transaction data → Bronze relational and JSONL records → advertiser, click, conversion and CRM JSONL sources
8. Partner Attribution Share	Attributed revenue and revenue contribution percentage	`partner_daily`; Gold partner dimension, conversion facts and attribution bridge	Silver partner, click, conversion and touchpoint data → Bronze relational records → corresponding source CSVs
9. Impressions vs Conversions	Impression count and conversion trend	`campaign_daily`; Gold date and campaign dimensions, impression and conversion facts	Silver impressions and conversion history → Bronze relational records → impression and conversion CSVs
10. Delayed Adjustments	Adjustment timing and eligible financial impact	`adjustment_daily`; Gold revenue-adjustment and conversion facts	Silver revenue adjustments and conversion history → Bronze relational records → adjustment and conversion CSVs

The semantic layer provides the final business-facing measures, while the Data Marts supply aggregated reporting data from the Gold warehouse.

The Gold layer maintains business relationships and fact-level measures. Silver contains standardized records, and Bronze preserves the source information required for reconciliation and investigation.

This mapping identifies the logical data lineage for each business analysis. Individual records can be investigated further using the relevant source identifiers, warehouse relationships and validation queries.

This layered approach separates raw data preservation, data cleansing, dimensional modeling and business reporting.

For example, a campaign's adjusted revenue can be traced from conversion and adjustment source records through their Bronze and Silver representations, into the Gold fact tables, and finally through the reporting marts to the semantic layer.

The reconciliation queries developed in Question 6 provide additional evidence that the reporting measures remain consistent with the underlying warehouse data.

9.4.3 Business Decision Support

The analytical findings demonstrate several practical uses of the PulseTrack warehouse.

Marketing teams can identify campaigns with high click-through rates but low conversion rates, investigate underperforming creatives and monitor unusual changes in campaign activity.

Account managers can compare advertiser performance over time while respecting historical assignment changes.

Partner-management teams can examine attributed revenue while clearly identifying unresolved partner relationships that require investigation.

Finance teams can monitor adjusted revenue, investigate corrections recorded after conversions and reconcile financial reporting with the underlying transaction history.

These analyses demonstrate how the data warehouse supports evidence-based decision-making. However, analytical results should be treated as indicators for further investigation rather than automatic proof of business causation.

9.5 Business Analytics Summary and Limitations

The PulseTrack data warehouse was used to execute ten business-analysis queries covering advertiser performance, campaign effectiveness, creative engagement, partner attribution, historical account-manager assignments, revenue trends and financial adjustments.

All ten queries are maintained in the dedicated SQL script `sql/validation/pulsetrack_business_analytics.sql`, allowing the analytical results to be reproduced and independently verified.

The analyses identified several findings requiring business attention, including campaigns with high click-through rates but low conversion rates, underperforming creatives, significant changes in monthly revenue and a substantial proportion of unresolved partner-attributed revenue.

Historical account-manager analysis demonstrated the value of Slowly Changing Dimension Type 2 in preserving accurate reporting relationships. Revenue-adjustment analysis also showed the importance of retaining financial corrections separately from original conversion records.

The implementation provides traceability from semantic KPIs through the Data Mart and Gold layers to the cleaned Silver and preserved Bronze source records.

However, the findings have limitations. Unresolved relationships reduce attribution completeness, reporting results depend on the available source data, and descriptive comparisons do not establish causal relationships.

Overall, the ten analytical queries demonstrate how the PulseTrack platform converts integrated warehouse data into actionable business insights while making important data-quality limitations visible.

Question 10 - Architecture Scenarios and Evaluation

10.1 Overview

The PulseTrack data warehousing architecture was evaluated against eight practical scenarios involving data growth, schema changes, duplicate delivery, delayed adjustments, historical corrections, pipeline failures, analytical performance and financial reconciliation.

The implemented platform uses PostgreSQL Bronze, Silver, Gold, Data Mart and semantic layers, with Python-based processing and Apache Airflow orchestration.

The following evaluation identifies how the existing design addresses each scenario, which components are involved, and what additional improvements would be required for a larger production environment.

10.2 Scenario-Based Architecture Evaluation

10.2.1 Scenario 1 – Source Data Volume Increases Tenfold

Scenario: The daily source-data volume increases by a factor of ten.

Existing Architecture:

The PulseTrack ingestion scripts use idempotent loading to avoid repeatedly inserting previously ingested data. Large transactional and event-log datasets are processed in batches, reducing the need to load all source records into Python memory at once.

The warehouse also separates detailed Gold facts from summarized Data Marts and provides a materialized monthly analytical cache for repeated reporting queries.

Architectural Response:

A tenfold increase would place greater demands on database storage, ingestion time, transformation processing and reporting refreshes.

The system could be improved by introducing incremental processing, date-based partitioning for large fact tables, better workload scheduling and additional monitoring of execution time and resource usage.

The full Data Mart and materialized-cache refresh strategy would require particular attention because rebuilding complete analytical datasets may become increasingly expensive.

Components Involved: Bronze ingestion, Silver transaction processing, Gold fact tables, Data Mart refresh, PostgreSQL indexing and Airflow scheduling.

Evidence and Limitations:

Existing processing uses batch-based techniques, and the OLAP performance test demonstrated the benefit of materialized caching. However, a tenfold volume stress test has not been performed. Therefore, the scalability improvements described above are architectural recommendations rather than verified performance results.

10.2.2 Scenario 2 – A JSON Source Changes Its Schema

Scenario: A JSONL source introduces new fields, changes an existing field structure or provides unexpected data types.

Existing Architecture:

The Bronze layer preserves original JSONL records as raw text, together with the source filename, line number and ingestion timestamp.

The Silver JSONL processing script parses the records, validates required identifiers and extracts selected fields into structured profile tables.

The implementation also retains the original parsed profile in JSONB format, allowing additional source attributes to be investigated without discarding the original document.

Invalid JSON or unsupported field structures can be identified during Silver processing and recorded in the profile-error table.

Architectural Response:

If the new schema remains compatible with the existing parser, additional fields can be preserved in the raw profile data without immediately changing every downstream table.

If the change affects required identifiers, nested structures or expected data types, the Silver transformation and validation rules would need to be reviewed and updated.

A production enhancement would be to introduce explicit schema-version tracking, automated schema-change detection and compatibility tests before deploying transformation changes.

Components Involved: JSONL source files, `bronze.jsonl_records`, `scripts/process_jsonl_profiles.py`, Silver profile tables and `silver.jsonl_profile_errors`.

Evidence and Limitations:

The implemented Bronze and Silver layers provide raw-data preservation, validation and error recording. However, automatic schema-version detection and end-to-end schema-evolution handling have not been implemented or tested.

10.2.3 Scenario 3 – The Same Source File Is Delivered Three Times

Scenario: An upstream system accidentally delivers the same source file three times.

Existing Architecture:

The PulseTrack Bronze ingestion layer implements idempotent loading to prevent repeated ingestion of already stored records.

PostgreSQL source tables use an `EXCEPT ALL` comparison to identify missing records while preserving legitimate duplicate occurrences from the original source.

JSONL and event-log ingestion use source filenames and line numbers to identify previously ingested lines. Records with matching source-file and line-number keys are skipped during repeated ingestion.

At the Silver layer, event-log processing uses unique event identifiers to prevent repeated event deliveries from creating duplicate business events.

Architectural Response:

If an unchanged file is processed repeatedly under the same filename, the existing Bronze logic prevents duplicate line insertion.

The implementation was verified by rerunning all three Bronze ingestion scripts. The reruns examined 958,166 source records or lines in total and inserted zero new Bronze records.

Silver event-log processing also prevents previously loaded event identifiers from being inserted again.

However, filename-and-line-number deduplication does not provide complete protection if the same file is delivered under a different filename. A production enhancement would introduce file-content checksums, delivery manifests and stable source-record identifiers where available.

Components Involved: Bronze ingestion scripts, Bronze source tables, PostgreSQL uniqueness constraints, Silver event processing and pipeline audit records.

Evidence and Limitations:

The unchanged-source reruns confirmed zero additional Bronze inserts. A separate Gold replay test also demonstrated idempotent processing of 50,000 existing impression records.

However, the Gold replay test was not a complete simulation of receiving the same external file three times. Cross-filename duplicate-file detection remains a recommended improvement.

10.2.4 Scenario 4 – A Revenue Adjustment Arrives Seven Days Late

Scenario: A correction to conversion revenue becomes available several days after the original conversion has already been processed and reported.

Existing Architecture:

PulseTrack stores revenue adjustments separately from original conversions.

The Bronze layer preserves adjustment source records, while Silver standardizes the adjustment amount, conversion reference and relevant timestamps.

The Gold warehouse maintains a dedicated revenue-adjustment fact table linked to the corresponding conversion where a valid relationship is available.

The Data Mart and semantic layers incorporate applicable adjustments into adjusted revenue calculations.

This design allows financial corrections to change reported revenue without creating duplicate conversion records.

Architectural Response:

When a new adjustment becomes available, the adjustment record should be ingested and validated, then propagated through the Gold, Data Mart and semantic layers.

The reporting aggregates and monthly materialized cache must then be refreshed to reflect the correction.

Historical reporting should preserve the original conversion and account for the applicable adjustment according to the defined revenue-reporting rules.

A production enhancement would support targeted incremental processing of affected conversions and reporting periods instead of refreshing all reporting tables.

Components Involved: Revenue-adjustment source records, Bronze and Silver adjustment tables, `gold.fact_revenue_adjustment`, conversion facts, Data Mart tables, semantic reporting views and the materialized cache.

Evidence and Limitations:

A controlled transaction-based test demonstrated that adding a USD 43.52 adjustment to an existing eligible conversion changed its adjusted revenue from USD 56.51 to USD 100.03 without increasing the conversion count.

The test was rolled back after verification, preserving the original warehouse data.

The business-analytics analysis also identified 10,052 adjustment records with business adjustment timestamps at least seven days after their associated conversion timestamps.

However, a seven-day difference between business timestamps does not necessarily mean the record was ingested seven days late. A complete end-to-end delayed-source-delivery and backfill test has not been demonstrated.

10.2.5 Scenario 5 – A Historical Account-Manager Assignment Is Corrected

Scenario: An advertiser's account-manager assignment is corrected after the related transactions have already been processed into the warehouse.

Existing Architecture:

PulseTrack implements Slowly Changing Dimension Type 2 (SCD Type 2) for the Gold advertiser dimension.

Instead of overwriting historical assignments, this approach maintains multiple versions of an advertiser record with effective-date information.

The historical versions allow transactions to be associated with the account manager responsible during the relevant business period.

This is important because replacing all historical assignments with the latest manager would produce misleading account-manager performance reports.

Architectural Response:

When a historical assignment is corrected, the warehouse must identify the relevant effective date and update the advertiser's historical dimension versions.

Affected transactions must then be associated with the appropriate historical version, while transactions outside the corrected period should retain their existing relationships.

The dependent Data Mart and semantic reporting results must also be refreshed so that historical account-manager reports reflect the correction.

The original source evidence and correction history should be retained for investigation.

Components Involved: Silver advertiser and profile data, Gold SCD Type 2 advertiser dimension, conversion facts, account-manager reporting mart, semantic analytical views and pipeline audit records.

Evidence and Limitations:

A controlled SCD Type 2 correction test changed the account-manager assignment for advertiser ADV0001 from AM025 to AM001, effective 28 January 2025.

The test demonstrated that six earlier conversions retained their historical assignment while 1,522 later conversions were associated with the corrected assignment. The advertiser's dimension history increased from two versions to three versions.

The test was rolled back after verification, leaving the production dataset unchanged.

The implementation therefore demonstrated historical correction behavior for a controlled case. However, a complete automated late-arriving correction workflow, including independent source delivery and incremental downstream rebuilding, has not been verified.

10.2.6 Scenario 6 – The ETL Pipeline Fails Halfway Through Processing

Scenario: The data pipeline fails after some processing stages have completed but before the remaining stages finish.

Existing Architecture:

The PulseTrack pipeline is orchestrated using Apache Airflow, with ten sequential tasks covering Bronze ingestion, Silver transformations, Gold warehouse loading, Data Mart refresh and validation.

Airflow records task execution status and provides logs to support failure investigation.

The database processing scripts also implement transaction controls and idempotent loading, allowing completed stages to be examined and processing to be repeated without unnecessarily creating duplicate records.

The `audit.pipeline_runs` table provides additional evidence through execution status, timestamps, error counts and error messages.

Architectural Response:

If a stage fails, the affected task should be marked as failed, and downstream tasks should not continue as though the full pipeline had completed successfully.

The failure should be investigated using Airflow logs, PostgreSQL errors and pipeline audit records.

After correcting the underlying problem, the failed stage and any required downstream stages should be rerun. Source-to-target reconciliation and analytical validation should then confirm that the resulting data remains consistent.

Transaction boundaries must be considered carefully. A failed operation can be rolled back within its database transaction, but a failure later in the overall workflow does not automatically undo all earlier completed tasks.

In particular, some Silver processing scripts commit individual batches. A failure during a later batch may leave earlier batches committed, making idempotent recovery and reconciliation important.

Components Involved: Apache Airflow DAG, Python ETL scripts, PostgreSQL transactions, `audit.pipeline_runs`, task logs and pipeline validation queries.

Evidence and Limitations:

Controlled tests examined failures during Gold loading and Data Mart refresh operations. Failed executions were recorded in the pipeline audit, and successful reruns were verified after correcting the issues.

A separate Airflow test deliberately introduced an incorrect database password. The resulting task failure was identified, and processing succeeded after the connection configuration was corrected.

The Airflow workflow successfully executed all eleven tasks, and the final Gold-to-Mart, attribution and OLAP cache validations passed.

These tests provide evidence of failure detection, auditing and manual recovery. However, they do not establish that every possible mid-pipeline failure can be recovered automatically without intervention.

10.2.7 Scenario 7 – A BI User Requests Five Years of Analytical Data

Scenario: A business intelligence user requests five years of historical advertising performance data for trend analysis and management reporting.

Existing Architecture:

The PulseTrack architecture separates detailed warehouse facts from summarized Data Mart tables and semantic analytical views.

The Gold layer maintains dimensional and transactional information, while the Data Mart layer provides daily summaries for campaigns, advertisers, creatives, partners, account managers and revenue adjustments.

The semantic layer supports analytical operations such as roll-up, drill-down, ranking and period-over-period comparisons.

A materialized monthly analytical cache is also available to improve performance when frequently requested results can be precomputed.

Architectural Response:

A five-year reporting request would first require confirmation that the complete historical source data is available and has been processed into the warehouse.

Once available, the date dimension and reporting tables could support aggregation across years, quarters and months.

For frequently repeated reports, materialized summaries could reduce query processing costs. Indexing, date-based partitioning and incremental refresh strategies would become increasingly important as the historical dataset grows.

The BI user should also be able to distinguish missing reporting periods from periods with genuinely zero business activity.

Components Involved: Historical source data, Gold date dimension and fact tables, daily Data Marts, semantic analytical views, monthly materialized cache and PostgreSQL query optimization.

Evidence and Limitations:

The existing OLAP implementation successfully demonstrated monthly, quarterly and yearly aggregation, as well as Month-over-Month, Quarter-over-Quarter and Year-over-Year comparisons.

Performance testing showed that retrieving 7,200 monthly analytical rows from the materialized cache required 0.547 milliseconds, compared with 959.571 milliseconds for the standard analytical view in the measured test.

However, the current verified reporting data primarily covers 2025 and 2026. A complete five-year dataset and five-year performance benchmark have not been demonstrated.

Therefore, five-year reporting is an architectural capability that would require sufficient historical data and additional scalability testing, rather than a completed five-year reporting test.

10.2.8 Scenario 8 – The Finance Team Disputes a Reported Revenue Amount

Scenario: The finance team identifies a reported revenue amount that does not match its expected financial records.

Existing Architecture:

PulseTrack preserves original conversion and revenue-adjustment records in Bronze and standardizes the relevant financial information in Silver.

The Gold warehouse stores conversion and revenue-adjustment facts separately. The Data Mart and semantic layers use these records to calculate base revenue, applicable adjustments and adjusted revenue.

The architecture also provides source-to-target reconciliation checks, data-quality reporting and pipeline execution audits.

Architectural Response:

The investigation should begin by identifying the disputed report, reporting period, campaign or advertiser, and the exact revenue measure being compared.

The finance team and data engineering team should then verify whether the report uses base revenue or adjusted revenue, which conversions are eligible for financial reporting, and which adjustment records fall within the reporting scope.

The reported amount can be traced backward through the semantic layer, Data Mart tables, Gold conversion and adjustment facts, Silver standardized records and original Bronze source data.

Special attention should be given to conversion status, adjustment timestamps, reporting-date rules, unmatched conversion references and corrections applied after the original conversion.

After identifying any discrepancy, the affected processing logic or source data should be corrected, the relevant reporting structures refreshed, and reconciliation checks rerun.

Components Involved: Bronze conversion and adjustment records, Silver conversion history and revenue adjustments, Gold financial fact tables, campaign and adjustment Data Marts, semantic revenue measures, validation scripts and pipeline audit records.

Evidence and Limitations:

The existing campaign revenue reconciliation verified USD 466,632.73 in eligible base revenue and USD 3,280.46 in applicable adjustments, producing USD 469,913.19 in adjusted revenue.

The automated validation script passed all fourteen Gold-to-Mart reconciliation checks, including checks involving revenue and adjustments.

A controlled late-adjustment test also demonstrated that a financial correction could change adjusted revenue without increasing the conversion count.

However, some adjustment records have unresolved conversion relationships, and differences in event dates, adjustment dates and eligibility rules may affect comparisons between reports.

The reconciliation results verify the implemented reporting calculations against their underlying warehouse data. They do not independently establish that every original financial transaction is correct.

Therefore, a disputed figure should be investigated using both source-level evidence and the documented business calculation rules.

10.3 Overall Architecture Evaluation and Recommendations

The PulseTrack data warehousing architecture was evaluated through eight practical scenarios covering scalability, schema evolution, duplicate delivery, late-arriving data, historical corrections, pipeline failures, analytical performance and financial reconciliation.

The implementation demonstrated several important architectural strengths.

The Bronze layer preserves original source information and supports idempotent ingestion. The Silver layer standardizes records, validates business data and preserves identified errors for investigation. The Gold layer provides dimensional modeling, historical advertiser relationships and detailed transactional facts.

The Data Mart and semantic layers support business reporting, financial reconciliation and multidimensional analysis. Apache Airflow provides workflow orchestration, while PostgreSQL audit records and automated validation queries support operational monitoring.

Controlled tests demonstrated duplicate-safe reruns, revenue-adjustment handling, historical account-manager corrections, failure detection and recovery. The completed pipeline also passed its analytical reconciliation checks.

However, the scenario evaluations identified several areas for future improvement.

Improvement Area	Recommended Enhancement
Scalability	Introduce incremental processing and partitioning for large transactional tables
Duplicate delivery	Add file checksums and delivery tracking to detect duplicate files with different names
Schema evolution	Implement automated schema-change detection and versioned validation rules
Late-arriving data	Support targeted reprocessing of affected historical records and reporting periods
Operational reliability	Strengthen automated recovery, checkpointing and end-to-end batch tracking
Analytical performance	Introduce incremental refresh strategies and monitor query performance as data volume grows
Data quality	Improve partner matching, unresolved reference investigation and financial reconciliation

These recommendations are based on the limitations observed in the current implementation and the requirements of the eight scenarios.

They represent future architectural improvements rather than capabilities that have already been fully implemented and tested.

Overall, the existing architecture provides a functional and validated foundation for data warehousing and analytical reporting, while the proposed improvements would support greater reliability and scalability in a production environment.

10.4 Final Conclusion

The PulseTrack Data Warehousing Capstone successfully demonstrated the design, implementation and validation of a layered analytical data platform using PostgreSQL, Python and Apache Airflow.

The platform integrated relational source tables, JSONL documents and event-log data through Bronze, Silver and Gold processing layers.

A total of 958,166 source records and file lines were preserved in Bronze. Silver transformations standardized the data, maintained conversion history and identified malformed event records and unresolved business relationships.

The Gold warehouse implemented dimensional and fact structures, including Slowly Changing Dimension Type 2 for historical advertiser relationships. Six Data Mart tables were developed to support business reporting, producing 687,318 aggregated reporting rows.

A PostgreSQL semantic layer provided reusable analytical measures, multidimensional operations and period-over-period comparisons. Performance testing demonstrated that the monthly materialized cache was substantially faster than the corresponding standard analytical view for the tested query.

The implementation also included ten business-analysis queries addressing advertising performance, revenue trends, attribution, historical account-manager assignments and financial adjustments.

Data quality and operational reliability were evaluated through reconciliation queries, pipeline audit records, controlled correction tests and failure-recovery testing. All fourteen Gold-to-Mart reconciliation checks, attribution reconciliation and OLAP cache validation passed.

The eight architecture scenarios demonstrated how the implemented system responds to common data engineering challenges and identified improvements required for more demanding production workloads.

Although limitations remain in areas such as incremental processing, unresolved source relationships and automated recovery, the project provides a working and verified end-to-end data warehousing implementation.

Overall, the capstone demonstrates how integrated data engineering, dimensional modeling, analytical processing and validation can transform heterogeneous source data into reliable and traceable information for business decision-making.